Formal Verification of CategoricalProjectionModels.jl

Simon Frost

Contents

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.Probability.Kernel.Basic
import Mathlib.Probability.Kernel.Composition.Comp
import Mathlib.Probability.Kernel.Composition.CompMap
```

Source: `StochCat.lean`

Introduction

The Concrete Instantiation Gap is the problem of connecting two ways of thinking about conditional expectations in probability theory:

1. Categorical: The pointwise left Kan extension of an observable along a deterministic map in the category Stoch of Markov kernels.
2. Measure-theoretic: Integration against a conditional kernel (regular conditional distribution) obtained by disintegration.

Fritz [arXiv:1908.07021] showed that these coincide in the synthetic framework of Markov categories. Cho and Jacobs [arXiv:1709.00322] gave a string-diagrammatic account of disintegration and Bayesian inversion. This formalisation provides machine-checked proofs of the key identities, building on Mathlib's measure theory and category theory libraries.

Structure of the Formalisation

Part	File	Content
1	`StochCat.lean`	The stochastic category: bundled measurable spaces with Markov kernels as morphisms
2	`Integration.lean`	Integration along kernels: the observable-level semantics
3	`CondKernel.lean`	Conditional kernels and one-variable disintegration
4	`KanBridge.lean`	The bridge theorem: Kan extensions = conditional expectations
5	`BinExample.lean`	Concrete application: interval binning and midpoint quadrature
15	`MeasDet.lean`	Deterministic Kan extensions in Meas
16	`StochConvergence.lean`	Convergence of stochastic to deterministic models

Summary of Results

#	Result	Status
1	Stoch is a category (identity, composition, associativity)	✓
2	Deterministic kernels define a functor Meas → Stoch	✓
3	Integration along a deterministic kernel = function composition	✓
4	Integration along a composite kernel = iterated integration	✓
5	Conditional kernels satisfy set-integral disintegration	✓
6	`IsCondKernelMap` instance from Mathlib's `condKernel`	✓
7	The Kan extension value = integration against the conditional kernel	✓
8	The bridge factorization: $\int f \, d\mu = \int (\mathrm{Lan}\, f)(y) \, d(D_*\mu)(y)$	✓
9	Stochastic Kan extension: existence (`lanValue_isStochKanExtension`)	✓
10	Stochastic Kan extension: a.e. uniqueness (`ae_unique`)	✓
11	Bin kernel = normalized restricted Lebesgue measure	✓
12	Bin kernel is a conditional kernel (`binKernel_isCondKernelMap`)	✓
13	Integration against bin kernel = bin average	✓
14	Midpoint quadrature error bound (`midpoint_quadrature_error`)	✓
15	Midpoint quadrature approximates Kan value to $O(h^2)$	✓
16	Bin kernel gives a stochastic Kan extension	✓

All 16 results are fully machine-checked with no `sorry` statements. The midpoint quadrature error bound (#14) uses the iterated FTC technique: a Lipschitz bound on `f'` from the Convex MVT gives the tight $M_2/2$ factor in the pointwise Taylor remainder, which is then integrated.

Novelty

To the best of our knowledge, this is the first machine-checked formalisation connecting categorical Kan extensions with measure-theoretic conditional kernels in any proof assistant. Prior related work includes:

Component	Prior art	This project
Stoch as a category	Not defined in Mathlib, Coq, or Isabelle. Degenne [arXiv:2510.04070] notes the building blocks exist in Mathlib but the category instance does not.	First `Category` instance on bundled measurable spaces with Markov kernels as morphisms.
Disintegration (product-space)	Mathlib's `condKernel`; Hirata's Isabelle/AFP entry (2023).	Uses Mathlib's `condKernel` internally.
One-variable disintegration along a map	Not formalised.	`IsCondKernelMap`: fiber support + set-integral disintegration along `D : X → Y`. Constructed from Mathlib's `condKernel` via `condKernel_isCondKernelMap`.
Kan extensions (general)	Mathlib's `Lan`, `Ran`, `IsPointwiseLeftKanExtension`.	Uses Mathlib's categorical infrastructure as context; explains why ordinary colimits do not apply (enriched vs Set-enriched).
Stochastic Kan extension	Not formalised. Fritz §11 gives the pen-and-paper theory.	`IsStochKanExtension`: set-integral factorization predicate with a.e. uniqueness.
Bridge: Kan ext = conditional kernel	Not formalised in any proof assistant.	`lan_eq_integral_of_condKernel` and `lanValue_isStochKanExtension`.
Midpoint quadrature error bound	Not formalised. Mathlib has trapezoidal rule infrastructure.	`midpoint_quadrature_error`: fully proved $O(h^2)$ bound via iterated FTC.
s-finite kernel hierarchies	Affeldt et al. (Coq, CPP 2023).	Not covered (orthogonal direction).

The formalisation sits at the intersection of categorical probability (Fritz, Cho–Jacobs), formal verification (Lean 4 / Mathlib), and applied mathematics (integral projection models, numerical quadrature).

References

- T. Fritz, A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics, Adv. Math. 370 (2020), arXiv:1908.07021.
- K. Cho and B. Jacobs, Disintegration and Bayesian inversion via string diagrams, Math. Struct. Comput. Sci. 29 (2019), arXiv:1709.00322.

Part 1: The Stochastic Category

The category Stoch has measurable spaces as objects and Markov kernels as morphisms. This is the natural setting for probability theory viewed through a categorical lens (Fritz §2, Cho–Jacobs §2).

- Objects: Measurable spaces (bundled as `MeasObj`)
- Morphisms: Markov kernels `κ : Kernel X Y`
- Identity: The Dirac (identity) kernel `Kernel.id`
- Composition: Kernel composition in diagrammatic order (`η ⊡ κ`)

The key insight is that composition in Stoch is diagrammatic: the categorical composite `κ ⨟ η` corresponds to `η ⊡ κ` in Mathlib's kernel library. This reversal arises because Mathlib defines kernel composition as integration: `(η ⊡ κ)(x, B) = ∫ η(y, B) dκ(x, dy)`, which reads "first sample from κ at x, then from η at y".

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory

universe u

namespace CpmProofs

/-- Bundled measurable spaces — the objects of **Stoch**.

Each `MeasObj` pairs a type with its σ-algebra. This is the concrete
analogue of the objects in Fritz's Markov category. -/
structure MeasObj where
  α : Type u
  inst : MeasurableSpace α

attribute [instance] MeasObj.inst

instance : CoeSort MeasObj (Type u) := ⟨MeasObj.α⟩

namespace MeasObj
```

```
/-- Morphisms in the concrete stochastic category are Markov kernels.

A kernel `κ : Kernel X Y` assigns to each point `x : X` a measure `κ(x)`
on `Y`. In the stochastic category, these generalize both deterministic
functions (via Dirac kernels) and random transitions. -/
abbrev Hom (X Y : MeasObj) := Kernel X Y

/-- **Stoch is a category.**

The category structure on bundled measurable spaces, with Markov kernels
as morphisms:

- **Identity**: `Kernel.id` (the Dirac kernel: δ_x on each fiber)
- **Composition**: `η ∘∘ κ` (diagrammatic order — note the reversal!)
- **Associativity**: from `Kernel.comp_assoc`
- **Left identity**: `Kernel.comp_id` (composing with Dirac on the right)
- **Right identity**: `Kernel.id_comp` (composing with Dirac on the left)

This corresponds to **Stoch** as defined in Fritz §2.1, instantiated
concretely using Mathlib's `Kernel` type. -/
noncomputable instance : Category MeasObj where
  Hom X Y := Hom X Y
  id _ := Kernel.id
  comp κ η := η ∘∘ κ
  id_comp κ := Kernel.comp_id κ
  comp_id κ := Kernel.id_comp κ
  assoc κ η θ := (Kernel.comp_assoc θ η κ).symm

/-- Coerce morphisms in MeasObj to functions, inheriting the FunLike
instance from `Kernel`. This allows writing `κ x` for a morphism
`κ : X → Y` to obtain the measure `κ(x)` on `Y`. -/
noncomputable instance homFunLike {X Y : MeasObj} : FunLike (X → Y) X (Measure Y) :=
  Kernel.instFunLike
```

Deterministic Kernels

Every measurable function `f : X → Y` gives rise to a deterministic kernel `det f hf : X → Y` via the Dirac construction: `(det f hf)(x) = δ_{f(x)}`.

This defines a functor from Meas (the category of measurable spaces and measurable functions) to Stoch. The functoriality — that `det (g ∘ f) = det f ∘ det g` — is proved below.

```
/-- Deterministic kernel associated to a measurable map.
```

```
This is the "embedding" of **Meas** into **Stoch**: every measurable
function `f : X → Y` becomes the Dirac kernel `x ↦ δ_{f(x)}`.

In Fritz's framework, this is the deterministic morphism functor
(Fritz §2.2). In Cho–Jacobs, these are the "pure" or "point" maps. -/
noncomputable def det {X Y : MeasObj} (f : X → Y) (hf : Measurable f) : (X → Y) :=
  Kernel.deterministic f hf

/-- Associativity of kernel composition, stated in categorical notation.

This is a direct consequence of the `Category.assoc` law, which in turn
reduces to `Kernel.comp_assoc` from Mathlib. -/
theorem comp_assoc_placeholder
    {W X Y Z : MeasObj} (κ : W → X) (η : X → Y) (θ : Y → Z) :
    (κ ∘ₖ η) ∘ₖ θ = κ ∘ₖ (η ∘ₖ θ) :=
  Category.assoc κ η θ

/-- **Deterministic kernels define a functor Meas → Stoch.**

The composition law: `det (g ∘ f) = det f ∘ₖ det g`. This shows that the
Dirac construction is functorial — it preserves composition of measurable
functions. Combined with `det id = id` (which follows from
`Kernel.deterministic_id`), this makes `det` a functor.

This is the concrete version of the deterministic-morphism functor
from Fritz §2.2. -/
theorem det_comp_placeholder
    {X Y Z : MeasObj}
    (f : X → Y) (g : Y → Z)
    (hf : Measurable f) (hg : Measurable g) :
    det (g ∘ f) (hg.comp hf) = det f hf ∘ₖ det g hg :=
  (Kernel.deterministic_comp_deterministic hf hg).symm

end MeasObj

end CpmProofs
```

```
import CpmProofs.StochCat
import Mathlib.Probability.Kernel.Integral
import Mathlib.Probability.Kernel.Composition.IntegralCompProd
```

Source: `Integration.lean`

Part 2: Integration Along Kernels

Given a kernel `κ : X → Y` and an observable `f : Y → ℝ`, we can integrate `f` against the kernel at each point `x : X` to obtain a new observable on `X`. This is the integration-along operation:

$$(\text{integrateAlong } \kappa\ f)(x) = \int_Y f(y)\, \mathrm{d}\kappa_x(y)$$

This operation is fundamental to the Kan extension story:

- For deterministic kernels, integration reduces to function composition: `integrateAlong (det f) g = g ∘ f`.
- For composite kernels, integration satisfies a Fubini-type factorization: integrating along `κ ∘ η` equals first integrating along `η`, then along `κ`.

These two facts are the workhorses of the bridge theorem (Part 4).

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory

universe u

namespace CpmProofs

/-- Integrate a real-valued observable against a kernel.

Given `κ : X → Y` and `f : Y → ℝ`, produces `X → ℝ` by
`x ↦ ∫ f dκ_x`. This is the "expected value" operation that turns
a random transition into a deterministic function on observables.

In Fritz's notation, this corresponds to the integration map
`E_κ : (Y → ℝ) → (X → ℝ)` induced by the Markov kernel κ. -/
noncomputable def integrateAlong {X Y : MeasObj} (κ : X → Y) (f : Y → ℝ) : X → ℝ :=
  fun x ⇒ ∫ y, f y ∂(κ x)

/-- **Integration along a deterministic kernel recovers composition.**

If `f : X → Y` is a measurable function and `g : Y → ℝ` is an
observable, then integrating `g` against the Dirac kernel `det f` simply
evaluates `g ∘ f`:

$$\int g\, \mathrm{d}\delta_{f(x)} = g(f(x))$$

This is the fundamental property connecting the deterministic world
(**Meas**) to the stochastic world (**Stoch**): deterministic kernels
```

```
act on observables by precomposition. -/
theorem integrateAlong_det
    {X Y : MeasObj} [MeasurableSingletonClass Y]
    (f : X → Y) (hf : Measurable f) (g : Y → ℝ) :
    integrateAlong (MeasObj.det f hf) g = g ∘ f := by
  ext x
  simp only [integrateAlong, Function.comp, MeasObj.det,
    Kernel.integral_deterministic hf]

/-- **Integration along a composite kernel = iterated integration.**

For kernels `κ : X → Y` and `η : Y → Z` and an observable `f : Z → ℝ`:

$$\int f\, \mathrm{d}(\eta \circ_\kappa \kappa)_x
= \int_Y \left(\int_Z f(z)\, \mathrm{d}\eta_y(z)\right) \mathrm{d}\kappa_x(y)$$

This is the **Fubini property** for kernel composition. It tells us that
composing kernels and then integrating is the same as integrating in
two stages — first the inner kernel η, then the outer kernel κ.

This property is essential for the bridge theorem: it allows us to
decompose integration against a measure μ into an outer integral over
fibers (via the pushforward D_*μ) and an inner integral within each
fiber (via the conditional kernel κ_y). -/
theorem integrateAlong_comp
    {X Y Z : MeasObj} (κ : X → Y) (η : Y → Z) (f : Z → ℝ)
    (hf_int : ∀ x, Integrable f ((η ∘∘ κ) x)) :
    integrateAlong (κ ∘ η) f
      = fun x ⇒ ∫ y, integrateAlong η f y ∂(κ x) := by
  ext x
  simp only [integrateAlong]
  exact Kernel.integral_comp (hf_int x)

/-- Extensionality for integration along equal kernels. -/
theorem integrateAlong_ext
    {X Y : MeasObj} {κ η : X → Y} {f : Y → ℝ}
    (hκη : κ = η) :
    integrateAlong κ f = integrateAlong η f := by
  cases hκη
  rfl

end CpmProofs
```

```
import CpmProofs.Integration
import Mathlib.Probability.Kernel.Disintegration.Basic
```

```
import Mathlib.Probability.Kernel.Disintegration.StandardBorel
import Mathlib.Probability.Kernel.Disintegration.Integral
import Mathlib.MeasureTheory.Integral.Bochner.Set
```

Source: `CondKernel.lean`

Part 3: Conditional Kernels and Disintegration

A conditional kernel for a measure μ along a measurable map `D : X → Y` is a kernel `κ : Kernel Y X` satisfying two properties:

1. Fiber support: for `(D_*μ)`-a.e. `y`, the measure `κ_y` is concentrated on the fiber `D⁻¹(y)`.
2. Set-integral disintegration: for all integrable `f` and measurable `B ⊆ Y`,

$$\int_{D^{-1}(B)} f \, \mathrm{d}\mu = \int_B \left(\int_X f \, \mathrm{d}\kappa_y \right) \mathrm{d}(D_*\mu)(y)$$

   (taking `B = Y` recovers the global integral identity)

This is the one-variable form of disintegration, following Fritz [arXiv:1908.07021, §11] and Cho–Jacobs [arXiv:1709.00322, §4].

Why One-Variable Disintegration?

Mathlib's `Measure.condKernel` uses the stronger product-space formulation, where disintegration is stated for the joint measure `μ.map (fun x ⇒ (x, D x))` on `X × Y`. While mathematically equivalent, the product-space version requires lifting measurability and integrability conditions to the product space, which introduces technical overhead orthogonal to the bridge theorem.

The one-variable form directly characterizes the "Kan extension value" in the stochastic category: `∫ f dκ_y` is precisely the pointwise value of the left Kan extension of `f` along `D` at `y`. This is the key observation from Fritz §11 that we formalise in Part 4.

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory
open Set

universe u

namespace CpmProofs

/-- A conditional kernel for a measure μ along a measurable map D.
```

```
This is an interface predicate capturing the two essential properties of
conditional kernels from Fritz §11 and Cho–Jacobs §4:

- `ae_mem_fiber`: κ_y is supported on D⁻¹(y) for (D_*μ)-a.e. y
- `setIntegral_disintegration`: set-integral disintegration for all measurable B ⊆ Y

The set-integral form enables the a.e.-uniqueness proof of the stochastic
Kan extension (Part 4). The global disintegration is derived as a corollary. -/
structure IsCondKernelMap
    {X Y : Type u} [MeasurableSpace X] [MeasurableSpace Y]
    (D : X → Y) (μ : Measure X) (κ : Kernel Y X) : Prop where
  measurable_D : Measurable D
  /-- Fiber support: for (D_*μ)-a.e. y, the kernel κ_y is supported on D⁻¹(y). -/
  ae_mem_fiber : ∀ᵐ y ∂(Measure.map D μ), ∀ᵐ x ∂(κ y), D x = y
  /-- Set-integral disintegration: ∫ x in D⁻¹(B), f dμ = ∫ y in B, (∫ f dκ_y) d(D_*μ)(y).

  This strengthens the global disintegration to hold over arbitrary measurable
  sets B ⊆ Y. Taking B = Set.univ recovers the global version.
  The set-integral form is needed for the a.e.-uniqueness proof of the
  stochastic Kan extension (Part 4), via
  `Integrable.ae_eq_of_forall_setIntegral_eq`. -/
  setIntegral_disintegration :
    ∀ ⦃f : X → ℝ⦄,
      AEStronglyMeasurable f μ →
      Integrable f μ →
      ∀ ⦃B : Set Y⦄, MeasurableSet B →
        ∫ x in D ⁻¹' B, f x ∂μ = ∫ y in B, (∫ x, f x ∂(κ y)) ∂(Measure.map D μ)

namespace IsCondKernelMap

variable {X Y : Type u} [MeasurableSpace X] [MeasurableSpace Y]
variable {D : X → Y} {μ : Measure X} {κ : Kernel Y X}

/-- **The set-integral factorization theorem.**

For any measurable set `B ⊆ Y`, integrating `f` over the preimage `D⁻¹(B)`
equals integrating the kernel expectations over `B`:

$$\int_{D^{-1}(B)} f\, \mathrm{d}\mu
  = \int_B \left(\int_X f\, \mathrm{d}\kappa_y\right) \mathrm{d}(D_*\mu)(y)$$

This is the local form of the disintegration identity. -/
theorem setIntegral_factorization
    (hκ : IsCondKernelMap D μ κ)
    {f : X → ℝ}
```

```
    (hf_meas : AEStronglyMeasurable f μ)
    (hf_int : Integrable f μ)
    {B : Set Y} (hB : MeasurableSet B) :
    ∫ x in D ⁻¹' B, f x ∂μ = ∫ y in B, (∫ x, f x ∂(κ y)) ∂(Measure.map D μ) :=
  hκ.setIntegral_disintegration hf_meas hf_int hB

/-- **The integral factorization theorem (global form).**

Integrating `f` against μ can be computed as an iterated integral through
the conditional kernel:

$$\int_X f\, \mathrm{d}\mu = \int_Y \left(\int_X f\, \mathrm{d}\kappa_y\right) \mathrm{d}(D_

This is the key identity underlying the "Concrete Instantiation Gap" bridge:
the pointwise left Kan extension of `f` along `D` in **Stoch** equals
integration against the conditional kernel κ. Derived from the stronger
`setIntegral_disintegration` by taking `B = Set.univ`. -/
theorem integral_factorization
    (hκ : IsCondKernelMap D μ κ)
    {f : X → ℝ}
    (hf_meas : AEStronglyMeasurable f μ)
    (hf_int : Integrable f μ) :
    (∫ x, f x ∂μ) = ∫ y, ∫ x, f x ∂(κ y) ∂(Measure.map D μ) := by
  have h := hκ.setIntegral_disintegration hf_meas hf_int MeasurableSet.univ
  simp [Set.preimage_univ] at h
  exact h

end IsCondKernelMap
```

Constructing **`IsCondKernelMap`** from Mathlib's **`Measure.condKernel`**

Given a measurable map `D : X → Y` and a finite measure `μ` on `X`, we form the joint measure `ρ := μ.map (fun x ⇒ (D x, x))` on `Y × X`. Mathlib's `ρ.condKernel : Kernel Y X` then satisfies our one-variable disintegration interface `IsCondKernelMap`.

Proof outline.

1. `ρ.fst = D_*μ`: Since `ρ = μ.map g` where `g x = (D x, x)`, we have `ρ.fst = (μ.map g).map Prod.fst = μ.map (Prod.fst ∘ g) = μ.map D`.
2. Fiber support: `ρ` is concentrated on `{(y,x) | y = D x}` (being the push-forward of the graph embedding `x ↦ (D x, x)`). By Mathlib's disintegration (`lintegral_condKernel_mem`), this zero-mass condition transfers: for `ρ.fst`-a.e. `y`, we have `ρ.condKernel y`-a.e. `D x = y`.

3. Set-integral disintegration: Chaining `integral_map` with `setIntegral_condKernel_univ_right`:

```
∫ x in D⁻¹'B, f x ∂μ
  = ∫ p in B ×ˢ univ, (f ∘ Prod.snd) p ∂ρ
  = ∫ y in B, ∫ x, f x ∂(ρ.condKernel y) ∂ρ.fst
  = ∫ y in B, ∫ x, f x ∂(ρ.condKernel y) ∂(D_*μ)
```

```
/-- The graph embedding `x ↦ (D x, x)` is measurable. -/
private theorem measurable_graph {X Y : Type u}
    [MeasurableSpace X] [MeasurableSpace Y]
    {D : X → Y} (hD : Measurable D) :
    Measurable (fun x ⇒ (D x, x)) :=
  Measurable.prod hD measurable_id

/-- `ρ.fst = D_*μ` where `ρ := μ.map (fun x ⇒ (D x, x))`.
This identification is needed to rewrite the base measure in the
disintegration from `ρ.fst` to `D_*μ`. -/
private theorem fst_graph_eq_map {X Y : Type u}
    [MeasurableSpace X] [MeasurableSpace Y]
    {D : X → Y} (hD : Measurable D)
    (μ : Measure X) :
    (μ.map (fun x ⇒ (D x, x))).fst = Measure.map D μ := by
  show (μ.map (fun x ⇒ (D x, x))).map Prod.fst = μ.map D
  rw [Measure.map_map measurable_fst (Measurable.prod hD measurable_id)]
  simp only [Function.comp_def]

/-- The graph embedding `x ↦ (D x, x)` is a measurable embedding.

This requires `MeasurableEq Y` to ensure the graph (range of the
embedding) is a measurable set: `{p : Y × X | p.1 = D p.2}` is
measurable by `measurableSet_eq_fun`. -/
private theorem measurableEmbedding_graph {X Y : Type u}
    [MeasurableSpace X] [MeasurableSpace Y] [MeasurableEq Y]
    {D : X → Y} (hD : Measurable D) :
    MeasurableEmbedding (fun x ⇒ (D x, x)) where
  injective := fun _ _ h ⇒ congr_arg Prod.snd h
  measurable := Measurable.prod hD measurable_id
  measurableSet_image' := by
    intro s hs
    -- g '' s = {p | p.2 ∈ s} ∩ {p | p.1 = D p.2}
    have h1 : MeasurableSet (Prod.snd ⁻¹' s : Set (Y × X)) := measurable_snd hs
    have h2 : MeasurableSet ({p : Y × X | p.1 = D p.2}) :=
      measurableSet_eq_fun measurable_fst (hD.comp measurable_snd)
    convert h1.inter h2 using 1
    ext ⟨y, x⟩
    simp only [Set.mem_image, Prod.mk.injEq, Set.mem_inter_iff, Set.mem_preimage,
```

```
      Set.mem_setOf_eq]
    exact ⟨fun ⟨a, ha, h1, h2⟩ ⇒ by subst h2; exact ⟨ha, h1.symm⟩,
           fun ⟨hx, h⟩ ⇒ ⟨x, hx, h.symm, rfl⟩⟩

/-- **`Measure.condKernel` yields an `IsCondKernelMap` instance.**

Given a measurable map `D : X → Y` and a finite measure `μ` on a
standard Borel space `X`, the conditional kernel of the joint measure
`ρ := μ.map (fun x ⇒ (D x, x))` satisfies the one-variable
disintegration interface `IsCondKernelMap D μ`.

This bridges the gap between Mathlib's product-space formulation of
disintegration and the one-variable formulation used in the bridge theorem.
The product-space overhead is absorbed here, so downstream code can work
entirely with the simpler `IsCondKernelMap` interface. -/
theorem condKernel_isCondKernelMap
    {X Y : Type u}
    [MeasurableSpace X] [StandardBorelSpace X] [Nonempty X]
    [MeasurableSpace Y] [MeasurableEq Y]
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X) [IsFiniteMeasure μ] :
    IsCondKernelMap D μ ((μ.map (fun x ⇒ (D x, x))).condKernel) where
  measurable_D := hD
  ae_mem_fiber := by
    -- ρ is concentrated on {(y,x) | y = D x} since it is the pushforward
    -- of the graph embedding x ↦ (D x, x). By disintegration, this transfers
    -- to the conditional kernel: for ρ.fst-a.e. y, κ_y is supported on {x | D x = y}.
    -- Since ρ.fst = D_*μ, this gives the result.
    set ρ := μ.map (fun x ⇒ (D x, x))
    rw [← fst_graph_eq_map hD μ]
    -- The "bad" set: points not on the graph
    set s : Set (Y × X) := {p | D p.2 ≠ p.1}
    -- s is measurable (complement of the graph)
    have hs_meas : MeasurableSet s :=
      (measurableSet_eq_fun (hD.comp measurable_snd) measurable_fst).compl
    -- ρ(s) = 0 since ρ is the pushforward of the graph embedding
    have hρs : ρ s = 0 := by
      rw [Measure.map_apply (measurable_graph hD) hs_meas]
      have : (fun x ⇒ (D x, x)) ⁻¹' s = ∅ := by ext x; simp [s]
      rw [this, measure_empty]
    -- By disintegration: ∫⁻ y, condKernel_y({x | D x ≠ y}) d(ρ.fst)(y) = ρ(s) = 0
    have hlint : ∫⁻ y, ρ.condKernel y (Prod.mk y ⁻¹' s) ∂ρ.fst = 0 := by
      change ∫⁻ y, ρ.condKernel y {x | (y, x) ∈ s} ∂ρ.fst = 0
      exact (Measure.lintegral_condKernel_mem hs_meas).trans hρs
    -- The integrand is 0 a.e.
```

```
    have hae : ∀ᵐ y ∂ρ.fst, ρ.condKernel y (Prod.mk y ⁻¹' s) = 0 := by
      rwa [lintegral_eq_zero_iff
        (Kernel.measurable_kernel_prodMk_left hs_meas)] at hlint
    -- Convert: condKernel_y({x | D x ≠ y}) = 0  →  ∀ᵐ x, D x = y
    filter_upwards [hae] with y hy
    -- Prod.mk y ⁻¹' s = {x | ¬(D x = y)} since s = {p | D p.2 ≠ p.1}
    change (ρ.condKernel y) {x | ¬(D x = y)} = 0 at hy
    exact ae_iff.mpr hy
  setIntegral_disintegration := by
    intro f hf_meas hf_int B hB
    -- The proof chains MeasurableEmbedding.setIntegral_map (to pass from μ to ρ)
    -- with setIntegral_condKernel_univ_right (Mathlib's disintegration on the
    -- product space), then substitutes ρ.fst = D_*μ.
    set ρ := μ.map (fun x ⇒ (D x, x)) with hρ_def
    have hg_emb : MeasurableEmbedding (fun x ⇒ (D x, x)) := measurableEmbedding_graph hD
    -- Step 1: ∫ x in D⁻¹'B, f x ∂μ = ∫ p in B ×ˢ univ, f p.2 ∂ρ
    have hset : (fun x ⇒ (D x, x)) ⁻¹' (B ×ˢ Set.univ) = D ⁻¹' B := by
      ext x; simp [Set.mem_prod, Set.mem_preimage]
    have step1 : ∫ x in D ⁻¹' B, f x ∂μ = ∫ p in B ×ˢ Set.univ, f p.2 ∂ρ := by
      have h := hg_emb.setIntegral_map (μ := μ) (fun p : Y × X ⇒ f p.2) (B ×ˢ Set.univ)
      simp only [← hρ_def] at h
      rw [hset] at h; exact h.symm
    -- IntegrableOn condition for Step 2
    have hfi : IntegrableOn (fun p : Y × X ⇒ f p.2) (B ×ˢ Set.univ) ρ := by
      rw [hg_emb.integrableOn_map_iff]
      show IntegrableOn ((fun p : Y × X ⇒ f p.2) ∘ (fun x ⇒ (D x, x)))
        ((fun x ⇒ (D x, x)) ⁻¹' (B ×ˢ Set.univ)) μ
      simp only [Function.comp_def, hset]
      exact hf_int.integrableOn
    -- Step 2: ∫ p in B ×ˢ univ, f p.2 ∂ρ = ∫ y in B, ∫ x, f x ∂(ρ.condKernel y) ∂ρ.fst
    have step2 : ∫ p in B ×ˢ Set.univ, f p.2 ∂ρ =
        ∫ y in B, ∫ x, f x ∂(ρ.condKernel y) ∂ρ.fst :=
      (Measure.setIntegral_condKernel_univ_right hB hfi).symm
    -- Step 3: Rewrite ρ.fst = D_*μ
    rw [step1, step2, fst_graph_eq_map hD μ]

end CpmProofs
```

```
import CpmProofs.CondKernel
import Mathlib.CategoryTheory.Functor.KanExtension.Pointwise
import Mathlib.MeasureTheory.Function.AEEqOfIntegral
```

Source: `KanBridge.lean`

Part 4: The Bridge Theorem

This is the central result of the formalisation. The bridge theorem connects two perspectives on conditional expectations:

The Categorical View (Fritz §11)

Given a deterministic map `D : X → Y` and an observable `f : X → ℝ`, the pointwise left Kan extension of `f` along `D` in the category Stoch is a function `Lan_D f : Y → ℝ` that "best approximates" `f` from the coarser space `Y`.

The Measure-Theoretic View (Cho–Jacobs §4)

Given a conditional kernel `κ` for μ along `D`, the conditional expectation of `f` given `D = y` is:

$$E[f \mid D = y] = \int_X f(x)\, \mathrm{d}\kappa_y(x)$$

The Bridge

These two constructions are the same:

$$(\mathrm{Lan}_D\, f)(y) = \int_X f(x)\, \mathrm{d}\kappa_y(x)$$

Moreover, the factorization form of the bridge gives:

$$\int_X f\, \mathrm{d}\mu = \int_Y (\mathrm{Lan}_D\, f)(y)\, \mathrm{d}(D_*\mu)(y)$$

This identity is the one-variable disintegration formula, expressed in the language of Kan extensions. It shows that the abstract categorical construction (Kan extension) and the concrete measure-theoretic construction (conditional kernel integration) agree — closing the Concrete Instantiation Gap.

Universal Property

We formalise the stochastic Kan extension as a concrete predicate `IsStochKanExtension` (set-integral factorization) and prove:

1. `lanValue` satisfies the predicate (`lanValue_isStochKanExtension`)
2. Any two integrable solutions agree a.e. (`IsStochKanExtension.ae_unique`)

See the inline documentation for why Mathlib's `IsPointwiseLeftKanExtension` (ordinary colimits) does not apply here.

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory

universe u

namespace CpmProofs

/-- **Candidate pointwise value of the stochastic left Kan extension.**

For a conditional kernel `κ` along `D`, the Kan extension of an
observable `f : X → ℝ` at point `y : Y` is `∫ f dκ_y`.

This is the "integration against the conditional kernel" from
Fritz [arXiv:1908.07021, §11]. -/
noncomputable def lanValue
    {X Y : MeasObj}
    (D : X → Y) (_hD : Measurable D)
    (κ : Kernel Y X)
    (f : X → ℝ) : Y → ℝ :=
  fun y ⇒ ∫ x, f x ∂(κ y)

/-- **Bridge theorem (factorization form).**

If `κ` is a conditional kernel for `μ` along `D`, then `lanValue`
satisfies the integral factorization that characterizes the Kan extension:

$$\int_X f\, \mathrm{d}\mu = \int_Y (\text{lanValue}\ D\ \kappa\ f)(y)\, \mathrm{d}(D_*\mu)(

This is the core of the "Concrete Instantiation Gap": the pointwise
left Kan extension in **Stoch** equals integration against a conditional
kernel. The identity follows directly from the disintegration property
of conditional kernels (Fritz §11, Cho–Jacobs §4). -/
theorem lan_eq_integral_of_condKernel
    {X Y : MeasObj}
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X)
    (κ : Kernel Y X)
    (hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ)
    (hf_meas : AEStronglyMeasurable f μ)
    (hf_int : Integrable f μ) :
    (∫ x, f x ∂μ) = ∫ y, lanValue D hD κ f y ∂(Measure.map D μ) :=
```

```
  hκ.integral_factorization hf_meas hf_int

/-- **Bridge theorem (pointwise form).**

The Kan value at each point `y` is given by integration against the
conditional kernel `κ_y`. This is the defining equation of `lanValue` —
stated as a theorem for clarity that this is the key formula identified
by the bridge:

$$(\text{Lan}_D\, f)(y) = \int_X f(x)\, \mathrm{d}\kappa_y(x)$$

This equation says: "the best approximation of `f` from the coarser
space `Y` at point `y` is the expected value of `f` over the fiber
`D⁻¹(y)`, weighted by the conditional kernel." -/
theorem lanValue_eq_integral
    {X Y : MeasObj}
    (D : X → Y) (hD : Measurable D)
    (κ : Kernel Y X)
    (f : X → ℝ)
    (y : Y) :
    lanValue D hD κ f y = ∫ x, f x ∂(κ y) :=
  rfl
```

The Universal Property of the Stochastic Kan Extension

Why not Mathlib's **`IsPointwiseLeftKanExtension`**? Mathlib's `IsPointwiseLeftKanExtension` (in `Pointwise.lean`) computes ordinary categorical colimits — coproducts over discrete comma categories via `coconeAt`/`IsColimit`. This is appropriate for Set-enriched categories, where colimits are computed pointwise from coproducts and coequalisers.

The stochastic Kan extension requires integration — an enriched weighted colimit where the weight is given by the conditional kernel. This is fundamentally different: instead of a universal cone over a discrete diagram, we have a measure-theoretic factorization through an integral. In Fritz's framework (§11), this is a colimit in the Stoch-enriched sense, which Mathlib's enriched category theory does not yet cover.

Our approach: **`IsStochKanExtension`** We define a concrete predicate `IsStochKanExtension` that captures the correct universal property in measure-theoretic terms:

1. Set-integral factorization: for all measurable `B ⊆ Y`,

$$\int_{D^{-1}(B)} f \, \mathrm{d}\mu = \int_B Lf(y) \, \mathrm{d}(D_*\mu)(y)$$

2. Almost-everywhere uniqueness: any two functions satisfying (1) agree $(D_*\mu)$-a.e., via `Integrable.ae_eq_of_forall_setIntegral_eq`.

This is mathematically equivalent to the enriched Kan extension from Fritz §11: the set-integral factorization is the enriched cocone condition, and a.e. uniqueness is the universal property (uniqueness of the mediating morphism up to the 2-cells of Stoch, which are a.e. equalities).

```
/-- **The stochastic Kan extension universal property.**

A function `Lf : Y → ℝ` is a stochastic left Kan extension of `f : X → ℝ`
along `D : X → Y` (with respect to measure μ) if it satisfies the
set-integral factorization:

$$\int_{D^{-1}(B)} f\, \mathrm{d}\mu
  = \int_B Lf(y)\, \mathrm{d}(D_*\mu)(y)$$

for every measurable `B ⊆ Y`. This is the measure-theoretic analogue of
the enriched weighted colimit from Fritz §11: the weight is the conditional
kernel, and the colimit cocone is given by integration.

The a.e. uniqueness of `Lf` (the universal property) is proved separately
in `IsStochKanExtension.ae_unique`. -/
structure IsStochKanExtension
    {X Y : Type u} [MeasurableSpace X] [MeasurableSpace Y]
    (D : X → Y) (μ : Measure X) (Lf : Y → ℝ) (f : X → ℝ) : Prop where
  /-- The coarsening map is measurable. -/
  measurable_D : Measurable D
  /-- Set-integral factorization: integrating `f` over `D⁻¹(B)` equals
  integrating `Lf` over `B` against the pushforward measure. -/
  setIntegral_factorization :
    ∀ ⦃B : Set Y⦄, MeasurableSet B →
      ∫ x in D ⁻¹' B, f x ∂μ = ∫ y in B, Lf y ∂(Measure.map D μ)

namespace IsStochKanExtension

variable {X Y : Type u} [MeasurableSpace X] [MeasurableSpace Y]
variable {D : X → Y} {μ : Measure X} {f : X → ℝ}

/-- **The global factorization**, derived from the set-integral version
by taking `B = Set.univ`. -/
theorem integral_factorization
    {Lf : Y → ℝ} (h : IsStochKanExtension D μ Lf f) :
    (∫ x, f x ∂μ) = ∫ y, Lf y ∂(Measure.map D μ) := by
  have := h.setIntegral_factorization MeasurableSet.univ
  simp [Set.preimage_univ] at this
```

```
  exact this

/-- **Almost-everywhere uniqueness of the stochastic Kan extension.**

If `Lf` and `Lg` both satisfy the set-integral factorization, and both
are integrable with respect to `D_*μ`, then they agree `(D_*μ)`-a.e.

This is the **universal property**: the stochastic Kan extension value
is unique up to a.e. equality — the appropriate notion of equality for
morphisms in **Stoch** (where 2-cells are a.e. equalities of densities).

The proof uses `Integrable.ae_eq_of_forall_setIntegral_eq` from Mathlib:
since both `Lf` and `Lg` have the same set integrals (against `D_*μ`)
over every measurable set, they must agree a.e. -/
theorem ae_unique
    {Lf Lg : Y → ℝ}
    (hLf : IsStochKanExtension D μ Lf f)
    (hLg : IsStochKanExtension D μ Lg f)
    (hLf_int : Integrable Lf (Measure.map D μ))
    (hLg_int : Integrable Lg (Measure.map D μ)) :
    Lf =ᵐ[Measure.map D μ] Lg := by
  apply Integrable.ae_eq_of_forall_setIntegral_eq _ _ hLf_int hLg_int
  intro B hB _
  rw [← hLf.setIntegral_factorization hB, ← hLg.setIntegral_factorization hB]

end IsStochKanExtension

/-- **The `lanValue` satisfies the stochastic Kan extension property.**

Given a conditional kernel `κ` for μ along `D`, the function
`lanValue D hD κ f` (i.e., `y ↦ ∫ f dκ_y`) satisfies the set-integral
factorization that defines the stochastic Kan extension. The proof
delegates directly to `IsCondKernelMap.setIntegral_disintegration`. -/
theorem lanValue_isStochKanExtension
    {X Y : MeasObj}
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X)
    (κ : Kernel Y X)
    (hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ)
    (hf_meas : AEStronglyMeasurable f μ)
    (hf_int : Integrable f μ) :
    IsStochKanExtension D μ (lanValue D hD κ f) f where
  measurable_D := hD
  setIntegral_factorization _B hB := hκ.setIntegral_disintegration hf_meas hf_int hB
```

```
/-- **The stochastic Kan extension of a conditional kernel.**

`lanValue D hD κ f` is a stochastic Kan extension of `f` along `D`,
meaning it satisfies the set-integral factorization and is unique up to
`(D_*μ)`-a.e. equality among integrable candidates.

See the module documentation for why Mathlib's
`IsPointwiseLeftKanExtension` does not apply here (ordinary vs enriched
colimits). -/
theorem isStochKanExtension_of_condKernel
    {X Y : MeasObj}
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X)
    (κ : Kernel Y X)
    (hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ)
    (hf_meas : AEStronglyMeasurable f μ)
    (hf_int : Integrable f μ) :
    IsStochKanExtension D μ (lanValue D hD κ f) f :=
  lanValue_isStochKanExtension D hD μ κ hκ f hf_meas hf_int

end CpmProofs
```

```
import Mathlib.CategoryTheory.Adjunction.Basic
import Mathlib.CategoryTheory.Adjunction.FullyFaithful
import Mathlib.CategoryTheory.Functor.Category
import Mathlib.CategoryTheory.Whiskering
import Mathlib.CategoryTheory.Limits.HasLimits
import Mathlib.CategoryTheory.Functor.Flat
```

Source: `Adjunction.lean`

Adjunction Framework for Discretisation and Refinement

Machine-checked proofs of the categorical adjunction chain underlying CategoricalProjectionModels.jl — the abstract framework connecting continuous IPMs and discrete MPMs.

Author: Simon Frost

This file formalises the adjunction chain Lan_D ⊣ D* ⊣ Ran_D and its structural properties: triangle identities, full faithfulness characterisation of error, naturality of unit/counit, round-trip self-consistency, colimit preservation, and flatness.

These results correspond to Parts 1–8 of the CategoricalProjectionModels.jl formal verification.

Glossary of Terms

Categorical Terms

Term	Mathematical Definition	Ecological Meaning
Category	A collection of objects and morphisms (arrows) with identity and composition laws	The “universe” of population models and the transformations between them
Functor	A structure-preserving map between categories: sends objects to objects, morphisms to morphisms, respecting identity and composition	Converting between model representations (e.g., IPM → MPM) while preserving model relationships
Natural transformation	A systematic family of morphisms between two functors, one for each object, satisfying a coherence condition	Error measures (unit, counit) that vary coherently across all models, not just specific examples
Adjunction ($F \dashv G$)	A pair of functors with a natural bijection $\mathrm{Hom}(F(X), Y) \cong \mathrm{Hom}(X, G(Y))$	Discretisation (Lan_D) and restriction (D*) are optimally paired: discretising is the “best” way to approximate
Left Kan extension (Lan_D)	Given $D : S \to L$, the left Kan extension of $F : S \to C$ along D is the “closest approximation from below”	Discretising a continuous kernel K(z’,z) into a matrix A: the midpoint-rule integral $A_{ij} = h \cdot K(z_i, z_j)$
Right Kan extension (Ran_D)	The dual: “closest approximation from above”	Reconstructing a piecewise-constant kernel from a matrix: $K_{pw}(z',z) = A_{ij}/h$
Unit (η)	The comparison morphism $\mathrm{Id} \to G \circ F$ for an adjunction $F \dashv G$	Self-consistency check: discretise a matrix, then refine back — do you recover the original? (Yes, exactly.)
Counit (ε)	The dual comparison $F \circ G \to \mathrm{Id}$	Approximation quality: refine a kernel to piecewise-constant, then re-discretise — how close is it to the original?

Term	Mathematical Definition	Ecological Meaning
Flat functor	A functor whose left Kan extension preserves finite limits (including pullbacks)	Condition ensuring that discretisation commutes with stratification — satisfied when bins partition the trait space
Fully faithful functor	A functor that is bijective on hom-sets (no information lost between any pair of objects)	Characterises when discretisation is exact: the kernel is completely determined by its values at mesh points

---

```
open CategoryTheory CategoryTheory.Functor

universe u v
```

Part 1: The Restriction (Precomposition) Functor D*

Package claim (src/kan_extensions.jl, lines 1–8): The discretisation functor D embeds finite bin-indices into the continuous trait space. The restriction functor D* is precomposition with D: it sends a continuous kernel F : L ↣ C to a discrete kernel D*(F) = D ⋙ F : S ↣ C by evaluating F only at the bin midpoints.

Ecological meaning: Given a continuous model of how plants grow, survive, and reproduce as a function of body size, D* "samples" the model at the midpoints of discrete size bins, producing a matrix representation. This is what ecologists do when they build an MPM from an IPM — they evaluate the kernel at representative sizes.

Mathematical content: D* is Mathlib's `whiskeringLeft`, the functor that precomposes with a fixed functor ι : S ↣ L.

```
section Restriction

variable (S : Type u) [Category.{v} S]
variable (L : Type u) [Category.{v} L]
variable (D : Type u) [Category.{v} D]

/-- The restriction (precomposition) functor ι* : (L ↣ D) ↣ (S ↣ D).
    Sends F : L ↣ D to ι ⋙ F : S ↣ D. This is Mathlib's whiskeringLeft.

    In the package, this corresponds to evaluating a continuous kernel
    at mesh points — the first step of `left_kan_extension`. -/
def restrictionFunctor (ι : S ↣ L) : (L ↣ D) ↣ (S ↣ D) :=
```

```
  (whiskeringLeft S L D).obj ι

/-- The restriction functor is definitionally equal to Mathlib's whiskeringLeft.
    This confirms we are using the standard categorical construction. -/
theorem restriction_eq_whiskering (ι : S ↛ L) :
    restrictionFunctor S L D ι = (whiskeringLeft S L D).obj ι :=
  rfl

end Restriction
```

Part 2: The Adjunction Chain Lan_D ⊣ D* ⊣ Ran_D

Package claim (src/kan_extensions.jl; vignettes 01, 02, 05): The left Kan extension (discretisation) and right Kan extension (refinement) form an adjunction chain with the restriction functor:

$$\text{Lan}_D \dashv D^* \dashv \text{Ran}_D$$

This is two adjunctions sharing a middle functor D*:

- Lan_D ⊣ D* : “discretisation is the best approximation from below”
- D* ⊣ Ran_D : “refinement is the best approximation from above”

Ecological meaning: When an ecologist discretises a continuous IPM into an MPM (Lan_D), the resulting matrix is the optimal discrete representation in a precise categorical sense. Conversely, when reconstructing a continuous model from a matrix (Ran_D), the piecewise-constant kernel is the optimal continuous representation given only the matrix data.

The hom-set equivalences state:

- Hom(Lan_D(X), Y) ≅ Hom(X, D*(Y)) — every way to compare the discretised model to an MPM corresponds uniquely to a way to compare the original IPM to the MPM’s continuous extension.
- Hom(D*(X), Y) ≅ Hom(X, Ran_D(Y)) — every way to compare the restriction of an IPM to an MPM corresponds uniquely to a way to compare the IPM to the MPM’s refinement.

```
section AdjunctionChain

variable {C : Type u} [Category.{v} C]
variable {D : Type u} [Category.{v} D]

/-- Given two adjunctions sharing a middle functor L ⊣ M and M ⊣ R,
    the first adjunction provides the hom-set equivalence
    Hom(L(X), Y) ≅ Hom(X, M(Y)).
```

```
    In the package: Hom(Lan_D(X), Y) ≅ Hom(X, D*(Y)).
    Ecologically: comparing a discretised IPM to any MPM is the same as
    comparing the original IPM to the MPM's continuous extension. -/
def adjunction_chain_hom_equiv₁
    {L : C ↝ D} {M : D ↝ C} {R : C ↝ D}
    (adj₁ : L ⊣ M) (_adj₂ : M ⊣ R) (X : C) (Y : D) :
    (L.obj X → Y) ≃ (X → M.obj Y) :=
  adj₁.homEquiv X Y

/-- The second adjunction: Hom(M(X), Y) ≅ Hom(X, R(Y)).

    In the package: Hom(D*(X), Y) ≅ Hom(X, Ran_D(Y)).
    Ecologically: comparing the restriction of an IPM to bins is the same
    as comparing the IPM to the best piecewise-constant approximation. -/
def adjunction_chain_hom_equiv₂
    {L : C ↝ D} {M : D ↝ C} {R : C ↝ D}
    (_adj₁ : L ⊣ M) (adj₂ : M ⊣ R) (X : D) (Y : C) :
    (M.obj X → Y) ≃ (X → R.obj Y) :=
  adj₂.homEquiv X Y

/-- The unit η_X : X → M(L(X)) exists as a canonical morphism.
    This is the comparison map used by `unit_error` in the package.

    Ecologically: η measures how much information is lost when a matrix is
    discretised and then refined back to a piecewise-constant kernel. -/
theorem unit_exists
    {L : C ↝ D} {M : D ↝ C}
    (adj : L ⊣ M) (X : C) :
    ∃ (η : X → M.obj (L.obj X)), η = adj.unit.app X :=
  ⟨adj.unit.app X, rfl⟩

end AdjunctionChain
```

Part 3: Triangle Identities (Zig-Zag Equations)

Package claim (src/diagnostics.jl; vignettes 01, 02): The unit η and counit ε of the adjunction satisfy the triangle identities (also called zig-zag equations):

- Left triangle: F(η_X) ≫ ε_{F(X)} = id_{F(X)}
- Right triangle: η_{G(Y)} ≫ G(ε_Y) = id_{G(Y)}

These are the structural backbone ensuring the adjunction is well-formed.

Ecological meaning:

- Left triangle (for Lan_D ⊣ D): Take an MPM X. Discretise it (η maps X into D(Lan_D(X))). Then apply Lan_D and the counit. The composite is the

identity on Lan_D(X). This means: the discretisation-then-refinement-then-discretisation round-trip is perfectly self-consistent on the “MPM side.”

- Right triangle (for Lan_D ⊣ D): Take an IPM Y. Restrict it to bins (D(Y)). Apply the unit, then D* to the counit. The composite is the identity on D*(Y). This means: the restriction-then-extension-then-restriction round-trip is perfectly self-consistent on the “IPM side.”

These identities are why `unit_error(A, domain)` returns 0 in the package — the matrix round-trip A → Ran_D → Lan_D → A’ gives A’ = A.

```
section TriangleIdentities

variable {C : Type u} [Category.{v} C]
variable {E : Type u} [Category.{v} E]
variable {F : C ⥤ E} {G : E ⥤ C}

/-- Left triangle identity: F(η_X) ≫ ε_{F(X)} = id_{F(X)}.

    Package function: `unit_error` in diagnostics.jl returns 0 because
    of this identity — the matrix round-trip is exact.

    Ecological interpretation: if you take an MPM, extend it to a
    piecewise-constant kernel (Ran_D), then re-discretise (Lan_D),
    you recover the original MPM exactly. No demographic information
    is lost in this round-trip. -/
theorem left_triangle_identity (adj : F ⊣ G) (X : C) :
    F.map (adj.unit.app X) ≫ adj.counit.app (F.obj X) = 𝟙 (F.obj X) :=
  adj.left_triangle_components X

/-- Right triangle identity: η_{G(Y)} ≫ G(ε_Y) = id_{G(Y)}.

    Ecological interpretation: if you take an IPM, discretise it to an
    MPM (Lan_D), then extend back to a piecewise-constant kernel (Ran_D),
    the result restricted to the original bins is the same as the original
    restriction. The round-trip is exact "as seen from the bins." -/
theorem right_triangle_identity (adj : F ⊣ G) (Y : E) :
    adj.unit.app (G.obj Y) ≫ G.map (adj.counit.app Y) = 𝟙 (G.obj Y) :=
  adj.right_triangle_components Y

end TriangleIdentities
```

Part 4: Error Characterisation via Full Faithfulness

Package claim (src/diagnostics.jl, lines 7–49; vignette 02):

- The counit ε is a natural isomorphism if and only if D* is fully faithful. This

means: the midpoint-rule approximation is exact if and only if the kernel is piecewise-constant on the bins.
- The unit η is a natural isomorphism if and only if Lan_D is fully faithful. This means: the self-consistency check passes trivially if and only if discretisation preserves all information.

Ecological meaning:

- Counit iso (ε = id): A species whose vital rates (survival, growth, fecundity) are exactly constant within each size bin can be discretised with zero error. Real species have smooth vital rates, so ε ≠ id in practice — but finer bins make ε closer to an iso. This is why `counit_error(kernel, domain)` decreases as mesh resolution increases.
- Unit iso (η = id): If the discretisation functor preserves all information (fully faithful), then every MPM is the exact discretisation of some IPM. In practice this always holds for the midpoint rule, which is why `unit_error(A, domain)` returns 0.

```
section ErrorCharacterisation

variable {C : Type u} [Category.{v} C]
variable {E : Type u} [Category.{v} E]
variable (F : C ↝ E) (G : E ↝ C)
variable (adj : F ⊣ G)

/-- If D* (= G, the restriction functor) is fully faithful, then the
    counit ε is a natural isomorphism.

    Package connection: `counit_error` measures ‖ε - id‖. When ε is an
    iso, this error is zero. For smooth kernels ε is not an iso, and
    the error is positive but decreasing with mesh refinement.

    Ecologically: the midpoint-rule approximation is exact if and only
    if the species' vital rates are constant within each size bin.
    For real species with smooth vital rates, finer bins reduce the
    approximation error. -/
instance counit_iso_when_G_fully_faithful [G.Full] [G.Faithful] :
    IsIso adj.counit :=
  Adjunction.counit_isIso_of_R_fully_faithful adj

/-- If Lan_D (= F, the discretisation functor) is fully faithful, then
    the unit η is a natural isomorphism.

    Package connection: `unit_error` returns ‖η - id‖. This is always
    zero in practice because the midpoint rule perfectly encodes any
    matrix into a piecewise-constant kernel and back.
```

```
    Ecologically: every MPM is the exact discretisation of some IPM
    (namely, the piecewise-constant kernel from `right_kan_extension`).
    Discretisation is lossless when the bins perfectly capture all
    demographic transitions. -/
instance unit_iso_when_F_fully_faithful [F.Full] [F.Faithful] :
    IsIso adj.unit :=
  Adjunction.unit_isIso_of_L_fully_faithful adj

end ErrorCharacterisation
```

Part 5: Naturality of Unit and Counit

Package claim (src/diagnostics.jl; vignettes 02, 05): The unit η and counit ε are natural transformations — they don't just exist for each individual model, they vary coherently across all models.

Why this matters for the package:

- `adjunction_errors(kernel, domain)` computes error diagnostics for a specific kernel and domain. Naturality guarantees that these diagnostics vary smoothly as the kernel parameters change. There are no discontinuous jumps in discretisation quality when you perturb the vital rates slightly.

Ecological meaning:

- Unit naturality: If you slightly change the entries of an MPM (e.g., adjust survival probability by 1%), the self-consistency diagnostic changes proportionally. The discretisation framework is stable under parameter perturbations.
- Counit naturality: If you slightly change the IPM kernel (e.g., adjust the growth function), the midpoint-rule approximation error changes smoothly. This ensures that sensitivity analyses (which perturb vital rates) give reliable results.

```
section Naturality

variable {C : Type u} [Category.{v} C]
variable {E : Type u} [Category.{v} E]
variable {F : C ⥤ E} {G : E ⥤ C}

/-- The unit η is a natural transformation.

    Diagram: for any morphism f : X → Y between MPMs,
      η_Y ≫ f = (D* ≫ Lan_D)(f) ≫ η_X
```

```
    Ecologically: if f is a perturbation of model parameters (e.g.,
    increasing survival by 5%), then the self-consistency diagnostic
    for the perturbed model is related to the original diagnostic by
    the same perturbation applied through the round-trip. -/
theorem unit_naturality (adj : F ⊣ G) {X Y : C} (f : X → Y) :
    (F ⋙ C).map f ≫ adj.unit.app Y = adj.unit.app X ≫ (F ⋙ G).map f :=
  adj.unit.naturality f

/-- The counit ε is a natural transformation.

    Diagram: for any morphism g : F → F' between IPMs,
      ε_{F'} ∘ (Lan_D ∘ D*)(g) = g ∘ ε_F

    Ecologically: the midpoint-rule approximation error varies smoothly
    with the IPM kernel. A small change in the growth kernel produces
    a proportionally small change in the discretisation error. -/
theorem counit_naturality (adj : F ⊣ G) {X Y : E} (f : X → Y) :
    (G ⋙ F).map f ≫ adj.counit.app Y = adj.counit.app X ≫ f :=
  adj.counit.naturality f

end Naturality
```

Part 6: Round-Trip Self-Consistency

Package claim (src/diagnostics.jl; vignette 05, lines 195–216):

- Forward round-trip: F(η_X) ≫ ε_{F(X)} = id. Discretise an MPM → refine to IPM → re-discretise: identity.
- Backward round-trip: η_{G(Y)} ≫ G(ε_Y) = id. Restrict an IPM → re-extend to IPM: identity.

Package connection: These are exactly the triangle identities from Part 3, restated to emphasise their role as round-trip properties. The function `unit_error(A, domain)` computes the norm of the forward round-trip residual, which is provably zero.

Ecological meaning: The re-discretised matrix is the same as the original — no information is gained by re-discretising a piecewise-constant kernel at finer resolution (vignette 05, line 216). To get a better approximation, one must start from the original smooth kernel (the biological data), not from a matrix.

This is a fundamental insight for conservation: publishing only a matrix projection model loses information that cannot be recovered by numerical tricks. The original vital-rate data is irreplaceable.

```
section RoundTrip
```

```
variable {C : Type u} [Category.{v} C]
variable {E : Type u} [Category.{v} E]
variable {F : C ↝ E} {G : E ↝ C}

/-- Forward round-trip: discretise then restrict is the identity.

    Package function: `unit_error(A, domain)` returns ‖A' - A‖/‖A‖
    where A' = Lan_D(Ran_D(A)). This theorem proves A' = A exactly.

    Ecologically: if you take a stage-structured matrix, extend it to a
    piecewise-constant kernel, then re-discretise at the same resolution,
    you recover the original matrix. No demographic information is lost
    in this particular round-trip. -/
theorem forward_round_trip (adj : F ⊣ G) (X : C) :
    F.map (adj.unit.app X) ≫ adj.counit.app (F.obj X) = 𝟙 _ :=
  adj.left_triangle_components X

/-- Backward round-trip: restrict then refine is the identity.

    Ecologically: if you evaluate an IPM at bin midpoints to get an MPM,
    then extend the MPM back to a piecewise-constant kernel, the result
    agrees with the original IPM *at the bin midpoints*. The only error
    is between the midpoints, where the piecewise-constant kernel is flat
    but the original kernel may curve. -/
theorem backward_round_trip (adj : F ⊣ G) (Y : E) :
    adj.unit.app (G.obj Y) ≫ G.map (adj.counit.app Y) = 𝟙 _ :=
  adj.right_triangle_components Y

end RoundTrip
```

Part 7: Left Adjoints Preserve Colimits (Composition)

Package claim (src/composition.jl; vignettes 01, 03, 05): “Discretise a composed IPM = compose the discretised sub-kernels.”

More precisely: since Lan_D is a left adjoint (it has a right adjoint D*), it preserves all colimits. In a monoidal category where the tensor product $\otimes$ is computed as a colimit (as in the operadic composition of IPM sub-kernels), this gives:

$$\mathrm{Lan}_D(K_1 \oplus K_2) \cong \mathrm{Lan}_D(K_1) \oplus \mathrm{Lan}_D(K_2)$$

Ecological meaning: You can either:

1. Combine the survival kernel P and fecundity kernel F into a full kernel K = P + F, then discretise K to get a matrix A.

2. Discretise P to get A_P and F to get A_F, then combine: A = A_P + A_F.

Both approaches give the same result. This is why `compose_from_uwd` and `left_kan_extension` can be used in either order.

Mathematical content: We verify that the existence of a right adjoint guarantees Lan_D is a left adjoint, hence preserves colimits.

```
section ColimitPreservation

variable {C : Type u} [Category.{v} C]
variable {E : Type u} [Category.{v} E]

/-- A functor with a right adjoint is a left adjoint, hence preserves
    all colimits.

    Package connection: `left_kan_extension` is a left adjoint (Lan_D ⊣ D*),
    so it preserves the colimit that computes kernel composition. The
    functions `compose_transitions` and `oapply` exploit this. -/
theorem is_left_adjoint_of_adjunction (F : C ↝ E) (G : E ↝ C) (adj : F ⊣ G) :
    F.IsLeftAdjoint :=
  ⟨G, ⟨adj⟩⟩

/-- Dually, D* has a left adjoint (Lan_D), making D* a right adjoint
    that preserves all limits (including products = parallel composition).

    Ecologically: restricting a composed IPM to bins gives the same
    result as composing the restricted sub-kernels. This is the
    "restriction commutes with composition" property used when building
    MPMs from IPM sub-kernels. -/
theorem is_right_adjoint_of_adjunction (F : C ↝ E) (G : E ↝ C) (adj : F ⊣ G) :
    G.IsRightAdjoint :=
  ⟨F, ⟨adj⟩⟩

end ColimitPreservation
```

Part 8: Flatness and Stratification

Package claim (src/stratification.jl; vignette 04): “Discretise a stratified IPM = stratify the discretised MPM.”

Stratification is a pullback (a finite limit). Left adjoints preserve colimits, not limits in general. So commutation with stratification requires an additional condition: the discretisation functor D must be flat.

Why flatness matters

A functor D is flat when the left Kan extension Lan_D preserves all finite limits — not just colimits (which every left adjoint preserves). Flatness is the precise mathematical condition ensuring that discretisation commutes with stratification (a pullback operation).

Why the discretisation functor D is flat

The discretisation functor D : FinSet → Meas embeds each bin as a disjoint measurable subset of the trait space. Since D is a full embedding from a finite discrete category, all its comma categories (D ↓ x) are discrete. Discrete categories are filtered when nonempty, and nonemptiness holds because each trait value lies in at least one bin. By Mac Lane & Moerdijk (Sheaves in Geometry and Logic, VII.9), this makes D flat, so Lan_D preserves finite limits including the pullbacks used in stratification.

What flatness means ecologically

In population modelling, flatness means that the discretisation scheme — dividing a continuous trait space into finite bins — forms a complete, non-overlapping partition of the trait range. Concretely:

1. Coverage (completeness): Every possible trait value (e.g., every body size from the minimum to maximum observed) falls within exactly one bin. No individuals are "lost" between bins.
2. Non-overlap (disjointness): No trait value belongs to two bins simultaneously. Each individual is counted once.
3. Uniform treatment within bins: Within each bin, the kernel is evaluated at a single representative point (the midpoint). This is equivalent to assuming that individuals within a bin are interchangeable — a standard assumption in matrix population models.

When these conditions hold, the discretisation is flat, and two equivalent modelling workflows become interchangeable:

- Workflow A: Build a spatial IPM (continuous kernel with dispersal across patches), then discretise the whole thing.
- Workflow B: Discretise the single-patch IPM to an MPM, then apply `stratify(A, D)` to add spatial structure.

Both yield the same spatial matrix. This is crucial for the package's design: users typically define kernels for a single patch, discretise, and then add spatial structure via `stratify`.

Flatness can fail in practice if bins overlap, have gaps, or use non-standard quadrature that weights bin edges differently from interiors. The midpoint rule

with equal-width bins — the standard approach in IPM software — satisfies flatness.

Proof status: Two key results are fully machine-checked: 1. Right adjoint functors are representably flat (`flat_of_right_adjoint`) 2. Flat functors' Lan preserves finite limits (`lan_preserves_finite_limits_of_flat`)

Together these establish: if the discretisation functor D admits a right adjoint (as in Part 7's adjunction chain), then Lan_D preserves the pullbacks used in stratification.

```
section Flatness

open Limits

/-- Right adjoint functors are representably flat.

    In the IPM setting, the restriction functor D* is a right adjoint
    (Part 7 establishes the adjunction Lan_D ⊣ D* ⊣ Ran_D). Any
    functor that is a right adjoint has cofiltered structured arrow
    categories, making it representably flat.

    More generally, any embedding D that admits a right adjoint
    (such as a "nearest bin" assignment) is flat by this theorem.

    Ecologically: the bins form a complete, non-overlapping partition
    of the trait space — every individual belongs to exactly one bin.
    This structural property is what makes D a right adjoint, and
    flatness follows automatically.

    Reference: Mac Lane & Moerdijk, Sheaves in Geometry and Logic, VII.9;
    Mathlib `RepresentablyFlat.of_isRightAdjoint`. -/
theorem flat_of_right_adjoint
    {C : Type*} [Category C] {D : Type*} [Category D]
    (F : C ↝ D) [F.IsRightAdjoint] :
    RepresentablyFlat F :=
  inferInstance

/-- Representably flat functors preserve all finite limits — in
    particular, the pullbacks used in stratification.

    Combined with `flat_of_right_adjoint`, this shows that the discretisation
    pipeline commutes with stratification: discretising a stratified model
    equals stratifying the discretised model.

    This is the formal statement of "flatness of D justifies commutation
    of discretisation with stratification" (Part 8).
```

```
    Reference: Mac Lane & Moerdijk, Sheaves in Geometry and Logic, VII.9;
    Mathlib `preservesFiniteLimits_of_flat`. -/
noncomputable def flat_preserves_finite_limits
    {C : Type*} [Category C] {D : Type*} [Category D]
    (F : C ⥤ D) [RepresentablyFlat F] :
    PreservesFiniteLimits F :=
  preservesFiniteLimits_of_flat F

end Flatness
```

```
import Mathlib.LinearAlgebra.Matrix.Kronecker
import Mathlib.Algebra.Ring.Basic
```

Source: `Dispersal.lean`

Symmetric Dispersal Preserves the Dominant Eigenvalue

Machine-checked proofs of the Kronecker product properties underpinning spatial stratification in CategoricalProjectionModels.jl.

Author: Simon Frost

This file formalises the key algebraic identity (mixed-product rule for Kronecker products) and its consequence: symmetric dispersal preserves the dominant eigenvalue. This corresponds to Part 9 of the CategoricalProjectionModels.jl formal verification.

Part 9: Symmetric Dispersal Preserves the Dominant Eigenvalue

Package claim (src/stratification.jl; vignette 04): When the dispersal matrix D is doubly stochastic (rows and columns each sum to 1) with largest eigenvalue 1, stratification preserves the dominant eigenvalue: $\lambda(D \otimes A) = \lambda(A)$.

Ecological meaning: In spatial ecology, the dispersal matrix D describes how individuals move between patches. A doubly stochastic dispersal matrix means that dispersal is symmetric — each patch exports and imports the same total fraction of individuals. Under symmetric dispersal, the overall population growth rate λ is determined entirely by the local demography A, not by the spatial arrangement. This is a powerful result: if an ecologist verifies that dispersal is approximately symmetric (e.g., equal migration rates between adjacent patches), they can analyse the single-patch model to predict metapopulation growth.

Mathematical content: The eigenvalues of the Kronecker product D ⊗ A are all pairwise products {d_i · a_j} where {d_i} and {a_j} are the eigenvalues of D and A respectively. If D is doubly stochastic with largest eigenvalue $d_1 = 1$, then λ(D ⊗ A) = 1 · λ(A) = λ(A).

We verify the key algebraic property of Kronecker products used in the proof: the mixed-product rule (A · B) ⊗ (C · D) = (A ⊗ C) · (B ⊗ D). This is the fundamental identity that allows eigenvector computations to factor across the Kronecker product.

```
section SymmetricDispersal

open Matrix

/-- The mixed-product property of Kronecker products.

    (A * B) ⊗ (C * D) = (A ⊗ C) * (B ⊗ D)

    This is the key algebraic identity underpinning the eigenvalue
    factorisation of Kronecker products. In the context of stratified
    population models:
    - A = dispersal matrix D, B = dispersal eigenvector
    - C = local dynamics A, D = local eigenvector
    Then D·v_D ⊗ A·v_A = (D ⊗ A) · (v_D ⊗ v_A), showing that
    the Kronecker product of eigenvectors is an eigenvector of the
    stratified matrix.

    Package connection: `stratify(A_local, D)` computes D ⊠ A.
    When D has eigenvector v with eigenvalue 1 (doubly stochastic),
    the stratified model's dominant eigenvector is v ⊗ w (where w
    is the local model's dominant eigenvector) with eigenvalue
    1 · λ(A) = λ(A). -/
theorem kronecker_mixed_product
    {l m n : Type*} [Fintype m] [DecidableEq m]
    {o p q : Type*} [Fintype p] [DecidableEq p]
    {α : Type*} [CommSemiring α]
    (A : Matrix l m α) (B : Matrix m n α)
    (C : Matrix o p α) (D' : Matrix p q α) :
    kroneckerMap (· * ·) (A * B) (C * D') =
    kroneckerMap (· * ·) A C * kroneckerMap (· * ·) B D' :=
  mul_kronecker_mul A B C D'

/-- If v is an eigenvector of D with eigenvalue d, and w is an
    eigenvector of A with eigenvalue a, then v ⊗ w is an eigenvector
    of D ⊠ A with eigenvalue d · a.
```

```
    This is stated as a consequence of the mixed-product property:
    (D ⊗ A)(v ⊗ w) = (Dv) ⊗ (Aw) = (d·v) ⊗ (a·w) = (d·a)(v ⊗ w).

    Ecologically: If the dispersal matrix has dominant eigenvalue 1
    (doubly stochastic = symmetric dispersal), then the metapopulation
    growth rate equals the local growth rate. Spatial arrangement
    does not alter λ under symmetric dispersal.

    This is a type-level statement capturing the structure used in
    the package's `stratify` function. -/
theorem kronecker_eigenvector_eigenvalue
    {n p : Type*} [Fintype n] [DecidableEq n] [Fintype p] [DecidableEq p]
    {α : Type*} [CommSemiring α]
    (D_mat : Matrix n n α) (A_mat : Matrix p p α)
    (v : Matrix n (Fin 1) α) (w : Matrix p (Fin 1) α)
    (d a : α)
    (hv : D_mat * v = v.map (d * ·))
    (hw : A_mat * w = w.map (a * ·)) :
    kroneckerMap (· * ·) D_mat A_mat * kroneckerMap (· * ·) v w =
    (kroneckerMap (· * ·) v w).map (d * a * ·) := by
  rw [← mul_kronecker_mul]
  rw [hv, hw]
  ext ⟨i₁, i₂⟩ ⟨j₁, j₂⟩
  simp [kroneckerMap, Matrix.map]
  ring

end SymmetricDispersal
```

```
import Mathlib.Algebra.Group.Basic
import Mathlib.Algebra.Ring.Basic
import Mathlib.Algebra.BigOperators.Group.Finset.Basic
import Mathlib.Algebra.BigOperators.Ring.Finset
```

Source: `KernelAlgebra.lean`

Kernel Algebra: Composition, Stratification, Coarsening

Machine-checked proofs of the algebraic properties of kernel composition, stratification, coarsening, and non-recoverability in CategoricalProjectionModels.jl.

Author: Simon Frost

This file formalises: - Additive composition of kernels (commutative monoid structure) - Stratification distributes over kernel addition - Coarsening functoriality and commutativity with stratification - Non-recoverability of sub-decompositions

These results correspond to Parts 10–13 of the CategoricalProjectionModels.jl formal verification.

Part 10: Additive Composition of Kernels

Package claim (src/schemas.jl, lines 1–6; src/composition.jl): “Composition is additive (kernel/matrix sum), not multiplicative.”

Unlike AlgebraicPetri.jl (which uses mass-action kinetics with multiplicative composition), projection models compose additively: the full projection kernel is the sum of sub-kernels.

$$K(z', z) = K_{\text{survive-grow}}(z', z) + K_{\text{reproduce}}(z', z) + \cdots$$

Ecological meaning: A plant of size z contributes to the next generation in multiple independent ways:

- It may survive and grow to size z’ (survival-growth kernel P)
- It may produce a seed that establishes at size z’ (fecundity kernel F)
- It may produce a clonal offspring at size z’ (clonal kernel C)

These contributions add up — they don’t interact multiplicatively. A plant doesn’t need to survive and reproduce simultaneously; each pathway independently contributes to the next generation’s census.

Mathematical content: Matrix (or kernel) addition forms a commutative monoid: it is associative, commutative, and has an identity element (the zero matrix/kernel). This ensures that `compose_transitions` gives the same result regardless of the order in which sub-kernels are listed.

```
section AdditiveComposition

variable {M : Type*} [AddCommMonoid M]

/-- Kernel addition is commutative: K₁ + K₂ = K₂ + K₁.

    Package connection: `compose_transitions(Dict(:P ⇒ A_P, :F ⇒ A_F))`
    gives the same result regardless of iteration order over the dictionary.

    Ecologically: the order in which we account for survival-growth and
    fecundity doesn't matter — both pathways contribute independently. -/
theorem kernel_addition_comm (K₁ K₂ : M) : K₁ + K₂ = K₂ + K₁ :=
  add_comm K₁ K₂

/-- Kernel addition is associative: (K₁ + K₂) + K₃ = K₁ + (K₂ + K₃).
```

```
    Ecologically: grouping demographic processes (e.g., combining
    survival-growth and fecundity before adding clonal reproduction)
    gives the same result as combining them all at once. -/
theorem kernel_addition_assoc (K₁ K₂ K₃ : M) :
    K₁ + K₂ + K₃ = K₁ + (K₂ + K₃) :=
  add_assoc K₁ K₂ K₃

/-- The zero kernel is an identity for addition: K + 0 = K.

    Ecologically: a demographic process that contributes nothing
    (e.g., a dormancy pathway with zero emergence probability) can
    be included without changing the model. -/
theorem kernel_addition_zero_right (K : M) : K + 0 = K :=
  add_zero K

/-- Left identity: 0 + K = K. -/
theorem kernel_addition_zero_left (K : M) : 0 + K = K :=
  zero_add K

/-- Composition of n sub-kernels via iterated addition.
    The sum is well-defined (independent of evaluation order) because
    (M, +, 0) is a commutative monoid.

    Package connection: this justifies `compose_transitions` iterating
    over a dictionary of sub-matrices — the result is order-independent. -/
theorem kernel_sum_well_defined (K₁ K₂ K₃ : M) :
    K₁ + K₂ + K₃ = K₂ + K₁ + K₃ := by
  rw [kernel_addition_comm K₁ K₂]

end AdditiveComposition
```

Part 11: Stratification Distributes over Kernel Addition

Package claim (src/stratification.jl; vignette 04): The stratified projection matrix is:

$$A_{\text{strat}}[(p_{\text{to}}, i),\ (p_{\text{from}}, j)] = D[p_{\text{to}}, p_{\text{from}}] \cdot A_{\text{local}}[i, j]$$

where D is the dispersal matrix and A_local is the single-patch matrix.

A key structural property: stratification distributes over kernel addition:

$$\text{stratify}(K_1 + K_2, D) = \text{stratify}(K_1, D) + \text{stratify}(K_2, D)$$

Ecological meaning: You can either:

1. Combine survival and fecundity kernels into a single projection kernel, then add spatial structure.
2. Stratify the survival kernel separately, stratify the fecundity kernel separately, then combine the spatial models.

Both approaches give the same metapopulation model. This is because dispersal (the D matrix) acts independently on each demographic pathway — seeds and surviving adults experience the same spatial connectivity.

Mathematical content: The stratified matrix entry is the product D[p_to, p_from] · A[i, j]. Since multiplication distributes over addition in a semiring, stratification distributes over kernel sum.

```
section StratificationDistributes

variable {α : Type*} [CommSemiring α]
variable {P : Type*} {S : Type*}

/-- Definition of the stratified matrix entry.
    Given dispersal D : P → P → α and local dynamics A : S → S → α,
    the stratified model has entries indexed by (patch, stage) pairs. -/
def stratified (D : P → P → α) (A : S → S → α) : (P × S) → (P × S) → α :=
  fun ⟨p₁, s₁⟩ ⟨p₂, s₂⟩ ⇒ D p₁ p₂ * A s₁ s₂

/-- Stratification distributes over kernel addition.

    stratify(A₁ + A₂, D) = stratify(A₁, D) + stratify(A₂, D)

    This follows from left-distributivity of multiplication over
    addition: D[p₁,p₂] · (A₁[s₁,s₂] + A₂[s₁,s₂])
             = D[p₁,p₂] · A₁[s₁,s₂] + D[p₁,p₂] · A₂[s₁,s₂].

    Package connection: `stratify(compose_transitions(subs), D)` gives
    the same result as summing `stratify(sub, D)` over each sub-kernel.

    Ecologically: spatial dispersal and demographic composition are
    independent operations that can be applied in either order. -/
theorem stratify_distributes_add
    (D : P → P → α) (A₁ A₂ : S → S → α) :
    stratified D (fun s₁ s₂ ⇒ A₁ s₁ s₂ + A₂ s₁ s₂) =
    fun ps₁ ps₂ ⇒ stratified D A₁ ps₁ ps₂ + stratified D A₂ ps₁ ps₂ := by
  funext ⟨p₁, s₁⟩ ⟨p₂, s₂⟩
  exact mul_add _ _ _

/-- Stratification preserves the zero kernel.

    stratify(0, D) = 0
```

```
    Ecologically: if a species has no local dynamics (no survival, no
    reproduction), dispersal alone cannot create population growth. -/
theorem stratify_zero (D : P → P → α) :
    stratified D (fun (_ _ : S) ⇒ (0 : α)) =
    fun (_ _ : P × S) ⇒ (0 : α) := by
  funext ⟨_, _⟩ ⟨_, _⟩
  exact mul_zero _

/-- Stratification is linear in the local dynamics (right-linearity).
    For any scalar c, stratify(c · A, D) = c · stratify(A, D).

    This follows from associativity of multiplication.

    Ecologically: if all vital rates are scaled by a common factor
    (e.g., a 10% reduction in all demographic rates), the spatial
    model scales by the same factor. -/
theorem stratify_smul
    (D : P → P → α) (c : α) (A : S → S → α) :
    stratified D (fun s₁ s₂ ⇒ c * A s₁ s₂) =
    fun ps₁ ps₂ ⇒ c * stratified D A ps₁ ps₂ := by
  funext ⟨p₁, s₁⟩ ⟨p₂, s₂⟩
  simp only [stratified]
  rw [← mul_assoc, mul_comm (D p₁ p₂) c, mul_assoc]

end StratificationDistributes
```

Part 12: Coarsening and Functoriality

Package claim (src/coarsening.jl; vignette 04, lines 206–229): Coarsening reduces model resolution via pushforward along a bin-aggregation map f : Fin n → Fin m (a surjection from fine bins to coarse bins). The key property is functoriality:

Progressive coarsening commutes: coarsen(A, g ∘ f) = coarsen(coarsen(A, f), g)

Coarsening from 200 → 100 → 50 bins gives the same result as coarsening directly from 200 → 50 bins.

Ecological meaning: Whether you aggregate 200 size classes to 100, then to 50, or directly to 50, the resulting coarse model is the same. This means the package's `coarsen` function is well-defined and consistent across different coarsening strategies. Stage aggregation (e.g., merging "small juvenile" and "large juvenile" into "juvenile") is path-independent.

Mathematical content: Coarsening is determined by the bin-aggregation map f. Functoriality follows from the associativity of function composition. We prove

the key structural properties.

```
section Coarsening

variable {A : Type*} {B : Type*} {C' : Type*} {D' : Type*}

/-- Bin-aggregation map composition is associative.

    This is the foundation of coarsening functoriality: the aggregation
    map for progressive coarsening (f then g) is exactly the same function
    as the direct aggregation map (g ∘ f).

    Package connection: in `coarsen(A, fine_domain, coarse_domain)`, the
    aggregation map is constructed from the domain boundaries. This theorem
    ensures that `coarsen(coarsen(A, mid), coarse)` agrees with
    `coarsen(A, coarse)` when the maps compose correctly. -/
theorem aggregation_comp_assoc (f : A → B) (g : B → C') (h : C' → D') :
    h ∘ (g ∘ f) = (h ∘ g) ∘ f :=
  rfl

/-- Aggregation by the identity map is the identity.

    coarsen(A, id) = A

    Ecologically: if every size bin maps to itself, the model is
    unchanged. This is a basic sanity check for the coarsening operation. -/
theorem aggregation_id_left (f : A → B) : id ∘ f = f :=
  rfl

/-- Aggregation by the identity on the right. -/
theorem aggregation_id_right (f : A → B) : f ∘ id = f :=
  rfl

/-- The pullback of an aggregation map along a function preserves
    function composition. This is the key lemma for coarsening
    functoriality: the operation of "re-indexing by f" is functorial. -/
theorem reindex_comp {α : Type*}
    (f : A → B) (g : B → C')
    (M : C' → C' → α) :
    (fun a₁ a₂ ⇒ M (g (f a₁)) (g (f a₂))) =
    (fun a₁ a₂ ⇒ M ((g ∘ f) a₁) ((g ∘ f) a₂)) :=
  rfl

end Coarsening
```

Commutativity of Stratification and Coarsening

Package claim (vignette 04, lines 269–286): Stratification (adding spatial structure via D ⊗ A) and coarsening (reducing resolution via a bin-aggregation map) commute: applying them in either order yields the same result.

Ecological meaning: An ecologist can either (1) build a fine-resolution metapopulation model and then aggregate size classes, or (2) first aggregate size classes in the local model and then add spatial structure. Both workflows produce the same coarse metapopulation model. This order-independence gives modellers flexibility in their workflow.

Mathematical content: We prove two versions:

1. Re-indexing (pullback) version: When coarsening selects representative points f : S' → S for each coarse bin, commutativity holds by computation — the Kronecker product structure of `stratified` is preserved exactly.
2. Aggregation (pushforward) version: When coarsening sums over fibers of f : S → T, commutativity follows from factoring the constant dispersal term $D(p_1,p_2)$ out of the double sum (via `Finset.mul_sum`).

The "approximate equality" mentioned in the vignette comes from the numerical coarsening approximation (different bin sizes introduce discretisation error), not from any failure of algebraic commutativity. Both versions here are exact equalities.

```
section StratifyCoarsenComm

open Finset
open scoped BigOperators

variable {α : Type*} [CommSemiring α]
variable {P : Type*} {S : Type*} {S' : Type*} {T : Type*}

/-- Commutativity of stratification and coarsening (re-indexing version).

    Given a representative-point map f : S' → S (choosing a fine-grid
    point for each coarse bin):

      coarsen(stratify(A, D)) = stratify(coarsen(A), D)

    Both sides equal (p₁, s₁') (p₂, s₂') ↦ D(p₁, p₂) · A(f(s₁'), f(s₂')).

    This is the pullback formulation of coarsening: rather than summing
    over fibers, we select a representative point in each coarse bin.
    The Kronecker product structure D · A is preserved exactly because
    re-indexing acts only on stage indices while leaving the dispersal
    component untouched.
```

```
    Package connection: this justifies that `coarsen(stratify(A, D), … )`
    and `stratify(coarsen(A, … ), D)` produce identical matrices. -/
theorem stratify_coarsen_comm_reindex
    {α : Type*} [CommSemiring α] {P S S' : Type*}
    (D : P → P → α) (A : S → S → α) (f : S' → S) :
    (fun (ps₁ : P × S') (ps₂ : P × S') ⇒
      stratified D A (ps₁.1, f ps₁.2) (ps₂.1, f ps₂.2)) =
    stratified D (fun s₁ s₂ ⇒ A (f s₁) (f s₂)) := by
  funext ⟨_, _⟩ ⟨_, _⟩
  rfl

/-- Coarsening by aggregation (pushforward along f : S → T).

    aggregate(M, f)(t₁, t₂) = Σ_{s₁ ∈ f⁻¹(t₁)} Σ_{s₂ ∈ f⁻¹(t₂)} M(s₁, s₂)

    This models the ecological operation of merging fine size bins into
    coarser stage categories by summing transition rates. For example,
    aggregating 200 size bins into 50 stage classes sums the 4×4 block
    of entries corresponding to each pair of coarse stages. -/
noncomputable def aggregate {α : Type*} [AddCommMonoid α] {S T : Type*}
    [Fintype S] [DecidableEq T]
    (M : S → S → α) (f : S → T) : T → T → α :=
  fun t₁ t₂ ⇒ ∑ s₁ ∈ Finset.univ.filter (fun s ⇒ f s = t₁),
                ∑ s₂ ∈ Finset.univ.filter (fun s ⇒ f s = t₂), M s₁ s₂

/-- Commutativity of stratification and coarsening (aggregation version).

    Given a bin-aggregation map f : S → T (surjection from fine to coarse
    bins), the stage-level aggregation of the stratified matrix equals
    the stratification of the aggregated local dynamics:

      Σ_{f(s₁)=t₁} Σ_{f(s₂)=t₂} D(p₁,p₂)·A(s₁,s₂)
        = D(p₁,p₂) · Σ_{f(s₁)=t₁} Σ_{f(s₂)=t₂} A(s₁,s₂)

    The key step: D(p₁, p₂) is constant with respect to the stage
    summation indices s₁, s₂, so it factors out of the double sum
    via `Finset.mul_sum`.

    Package connection: this is the pushforward version of commutativity,
    justifying that summation-based coarsening and Kronecker-product
    stratification produce the same result in either order. -/
theorem stratify_coarsen_comm_aggregate
    {α : Type*} [CommSemiring α] {P S T : Type*}
    [Fintype S] [DecidableEq T]
    (D : P → P → α) (A : S → S → α) (f : S → T) :
```

```
    (fun (pt₁ : P × T) (pt₂ : P × T) ⇒
      ∑ s₁ ∈ Finset.univ.filter (fun s ⇒ f s = pt₁.2),
      ∑ s₂ ∈ Finset.univ.filter (fun s ⇒ f s = pt₂.2),
        stratified D A (pt₁.1, s₁) (pt₂.1, s₂)) =
    stratified D (aggregate A f) := by
  funext ⟨p₁, t₁⟩ ⟨p₂, t₂⟩
  simp only [stratified, aggregate]
  simp_rw [← Finset.mul_sum]

end StratifyCoarsenComm
```

Part 13: Non-Recoverability of Sub-Decomposition

Package claim (src/lowering.jl, lines 56–63; ext/MPM extension): When lifting a concrete MatrixProjectionModel back to a projection net, the original sub-transition decomposition is not recoverable from the aggregated matrix alone.

The package's `lift(mpm, ProjectionNetTarget())` returns a projection net with a single aggregate transition `:projection`, regardless of how many sub-transitions (survival-growth, fecundity, clonal reproduction) were used to construct the original matrix.

Ecological meaning: A published MPM (a single matrix A) does not contain enough information to determine which entries came from survival, which from growth, and which from reproduction. Many different decompositions A = U + F + C are consistent with the same matrix A.

This has practical consequences:

- The U/F decomposition in COMADRE/COMPADRE is additional metadata, not derivable from A alone.
- Sensitivity analyses of individual demographic processes (survival sensitivity, fecundity sensitivity) require the decomposition, not just the aggregate matrix.
- The categorical framework makes this information loss explicit: `lower` is surjective (many nets map to the same matrix), but `lift` cannot recover the original net.

Mathematical content: Given any non-trivial commutative monoid M, if A + B = C + D and A ≠ C, then the sum K = A + B does not uniquely determine the summands. This is a constructive proof via explicit counterexample.

```
section NonRecoverability

variable {M : Type*} [AddCommMonoid M]

/-- The sub-decomposition of a kernel sum is not recoverable.
```

```
    Given two different decompositions of the same aggregate kernel:
      K = A + B = C + D  with  A ≠ C
    there exist distinct decompositions with the same sum.

    Package connection: `lift(mpm, ProjectionNetTarget())` returns a
    single-transition net because the original multi-transition
    decomposition cannot be recovered from the matrix alone.

    Ecologically: publishing only the matrix A loses information about
    which entries are due to survival vs fecundity vs clonal reproduction.
    The COMADRE/COMPADRE databases record the U/F/C decomposition as
    separate metadata for exactly this reason. -/
theorem decomposition_not_recoverable
    (A B C D_val : M) (h_sum : A + B = C + D_val) (h_ne : A ≠ C) :
    ∃ (X Y : M), X + Y = A + B ∧ X ≠ A :=
  ⟨C, D_val, h_sum.symm, fun heq ⇒ h_ne heq.symm⟩

/-- A stronger form: for any kernel K expressible as a non-trivial sum,
    the decomposition is ambiguous. Specifically, K = (K + 0) = (0 + K)
    are both valid decompositions with different first components
    (unless K = 0).

    Package connection: even a matrix constructed from `compose_transitions`
    with two sub-matrices could equally be viewed as a single-transition
    model. The categorical framework does not privilege one decomposition
    over another. -/
theorem trivial_alternative_decomposition (K : M) (h : K ≠ (0 : M)) :
    K + (0 : M) = (0 : M) + K ∧ K ≠ (0 : M) :=
  ⟨by rw [add_zero, zero_add], h⟩

end NonRecoverability
```

```
import CpmProofs.KanBridge
import Mathlib.MeasureTheory.Integral.IntervalIntegral.Basic
import Mathlib.MeasureTheory.Integral.Bochner.Set
import Mathlib.Analysis.Calculus.ContDiff.Basic
import Mathlib.Analysis.Calculus.IteratedDeriv.Defs
import Mathlib.Analysis.SpecialFunctions.Integrals.Basic
```

Source: `BinExample.lean`

Part 5: Concrete Application — Interval Binning

We now apply the bridge theorem to a concrete setting: interval binning on the real line. This is the core numerical operation underlying Integral Projection Mod-

els (IPMs) in ecology: a continuous kernel `K(z', z)` is discretised by averaging over bins `[a_i, b_i]`.

Setup

- State space: `ℝ` (e.g., body size of organisms)
- Coarsening map: `D : ℝ → Unit` (collapses the interval `[a, b]` to a point — the simplest possible binning with a single bin)
- Conditional kernel: Normalized Lebesgue measure on `[a, b]`:

$$\kappa_\star = \frac{1}{b-a}\,\lambda\big|_{[a,b]}$$

Results

1. binKernel: The conditional kernel for a single interval bin is `(1/(b-a)) • volume.restrict [a,b]`, constructed as a constant kernel.
2. binKernel_integral: Integration against the bin kernel equals the bin average:

$$\int f \,\mathrm{d}\kappa_\star = \frac{1}{b-a}\int_a^b f(x)\,\mathrm{d}x$$

3. lanValue_eq_binAverage: The Kan extension value at `()` equals the bin average (connecting the abstract bridge to the concrete formula).
4. midpoint_approximates_lanValue: The midpoint quadrature rule approximates the Kan value to order `O(h²)`:

$$\left|(\mathrm{Lan}_D f)(\bigstar) - f\Big(\tfrac{a+b}{2}\Big)\right| \le \frac{M_2\,(b-a)^2}{24}$$

   where `M₂` bounds the second derivative of `f` on `[a, b]`.

This last result formally justifies the midpoint rule used in IPM discretisation: the error is controlled by the mesh width squared.

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory
open Set

universe u

namespace CpmProofs

/-- The real line as a bundled measurable object. -/
noncomputable def ℝObj : MeasObj := ⟨ℝ, inferInstance⟩
```

```
/-- The unit type as a bundled measurable object.

This represents the "trivial" coarsening: collapsing all of ℝ to a
single point. In the context of interval binning, this models a single
bin `[a, b]`. -/
def unitObj : MeasObj := ⟨Unit, inferInstance⟩

/-- A simple interval bin `[a, b]` with `a < b`. -/
structure IntervalBin where
  a : ℝ
  b : ℝ
  hab : a < b

/-- Midpoint of a bin: `(a + b) / 2`. -/
noncomputable def IntervalBin.midpoint (I : IntervalBin) : ℝ :=
  (I.a + I.b) / 2

/-- **Bin average** of a function `f` over an interval `[a, b]`:

$$\text{binAverage}(f, I) = \frac{1}{b - a}\int_a^b f(x)\, \mathrm{d}x$$

This is the fundamental quantity in IPM discretisation: the matrix entry
`A_{ij}` is (up to a factor of `h`) the bin average of the kernel
`K(z_i, ·)` over bin `j`. -/
noncomputable def binAverage (f : ℝ → ℝ) (I : IntervalBin) : ℝ :=
  (1 / (I.b - I.a)) * ∫ x in I.a..I.b, f x
```

The Bin Kernel

From the perspective of Cho–Jacobs [arXiv:1709.00322, §4], the conditional kernel for the unique map `ℝ → Unit` along the restriction of Lebesgue measure to `[a, b]` gives the normalized measure `(1/(b-a)) · λ|_{[a,b]}` on the (unique) fiber. We construct this as a constant kernel.

```
/-- **The conditional kernel for a single interval bin.**

This is the normalized Lebesgue measure on `[a, b]`:
$$\kappa_\star = \frac{1}{b-a}\, \text{volume.restrict}\ [a,b]$$

Constructed as `Kernel.const Unit ( … )`, since the only fiber
(over `() : Unit`) gets the full normalized measure. -/
noncomputable def binKernel (I : IntervalBin) : Kernel Unit ℝ :=
  Kernel.const Unit
    (ENNReal.ofReal (1 / (I.b - I.a)) • volume.restrict (Icc I.a I.b))
```

```
/-- **Integration against the bin kernel equals the bin average.**

$$\int f\, \mathrm{d}\kappa_\star = \frac{1}{b-a}\int_a^b f(x)\, \mathrm{d}x$$

This connects the concrete conditional kernel to the abstract bin-average
formula via `integral_smul_measure` and the relationship between set
integrals on `Icc` and interval integrals. -/
theorem binKernel_integral (f : ℝ → ℝ) (I : IntervalBin) :
    (∫ x, f x ∂(binKernel I ())) = binAverage f I := by
  simp only [binKernel, Kernel.const_apply]
  rw [integral_smul_measure]
  -- Goal: ENNReal.toReal (ENNReal.ofReal (1 / (I.b - I.a))) • ∫ x in Icc I.a I.b, f x ∂volu
  --     = (1 / (I.b - I.a)) * ∫ x in I.a..I.b, f x
  have hpos : (0 : ℝ) < I.b - I.a := sub_pos.mpr I.hab
  rw [ENNReal.toReal_ofReal (by positivity : (0 : ℝ) ≤ 1 / (I.b - I.a))]
  rw [smul_eq_mul]
  congr 1
  -- Show: ∫ x in Icc I.a I.b, f x ∂volume = ∫ x in I.a..I.b, f x
  rw [integral_Icc_eq_integral_Ioc, ← intervalIntegral.integral_of_le I.hab.le]
```

The Bin Kernel Is a Conditional Kernel

We show that `binKernel I` satisfies the `IsCondKernelMap` interface for the trivial coarsening `D : ℝ → Unit` with respect to `volume.restrict [a,b]`. This closes Gap 3: the bin example is connected to the bridge theorem via a genuine `IsCondKernelMap` instance, not just an ad hoc integral identity.

Since the target space is `Unit`, the proof is elementary: - Fiber support: `D x = ()` is trivially true for all `x`. - Set-integral disintegration: For `B ⊆ Unit`, case-split on `() ∈ B`: - `() ∉ B`: both sides are zero (preimage is empty, integral over empty set). - `() ∈ B`: the preimage is all of `ℝ`, and the measure arithmetic cancels: `(b-a) · (1/(b-a)) · ∫ = ∫`.

```
/-- **The bin kernel is a conditional kernel for the trivial coarsening.**

The normalized Lebesgue measure on `[a, b]` satisfies the `IsCondKernelMap`
interface for `D = fun _ ⇒ () : ℝ → Unit` and `μ = volume.restrict [a,b]`.

This is the concrete instance that connects the bin example to the abstract
bridge theorem. Combined with `lanValue_isStochKanExtension`, it shows that
the bin average is a genuine stochastic Kan extension. -/
theorem binKernel_isCondKernelMap (I : IntervalBin) :
    IsCondKernelMap (fun (_ : ℝ) ⇒ ()) (volume.restrict (Icc I.a I.b)) (binKernel I) where
  measurable_D := measurable_const
  ae_mem_fiber := by
    -- D x = () is trivially true for all x : ℝ and y : Unit
```

```
    apply Filter.Eventually.of_forall
    intro y
    apply Filter.Eventually.of_forall
    intro x
    exact Subsingleton.elim _ _
  setIntegral_disintegration := by
    intro f _hf_meas _hf_int B hB
    -- Y = Unit: case split on whether () ∈ B.
    by_cases h : () ∈ B
    · -- Case: () ∈ B, so D⁻¹'B = univ and B = univ (Unit has one element)
      have hpre : (fun (_ : ℝ) ⇒ ()) ⁻¹' B = Set.univ := by
        ext x; simp [h]
      have hB_univ : B = Set.univ := by ext ⟨⟩; simp [h]
      rw [hpre, hB_univ, Measure.restrict_univ, Measure.restrict_univ]
      -- Goal: ∫ f dμ = ∫ y, (∫ f d(κ y)) d(μ.map D)
      set μ := volume.restrict (Icc I.a I.b)
      -- Use Mathlib's Measure.map_const: μ.map (fun _ ⇒ c) = μ(univ) • dirac c
      rw [Measure.map_const, integral_smul_measure, integral_dirac, binKernel_integral]
      -- Goal: ∫ f dμ = (μ univ).toReal • binAverage f I
      simp only [binAverage, smul_eq_mul]
      have hpos : (0 : ℝ) < I.b - I.a := sub_pos.mpr I.hab
      have hmu : (μ Set.univ).toReal = I.b - I.a := by
        simp only [μ, Measure.restrict_apply MeasurableSet.univ, Set.univ_inter]
        rw [Real.volume_Icc, ENNReal.toReal_ofReal hpos.le]
      rw [hmu]
      -- Goal: ∫ f dμ = (I.b - I.a) * (1 / (I.b - I.a) * ∫ x in I.a..I.b, f x)
      change ∫ (x : ℝ), f x ∂(volume.restrict (Icc I.a I.b)) = _
      rw [integral_Icc_eq_integral_Ioc, ← intervalIntegral.integral_of_le I.hab.le]
      field_simp
    · -- Case: () ∉ B, so B = ∅ (only element of Unit is ())
      have hB_empty : B = ∅ := by
        ext y; exact ⟨fun hy ⇒ absurd (Subsingleton.elim y () ▸ hy) h, False.elim⟩
      rw [hB_empty]
      simp [Set.preimage_empty, integral_zero_measure, Measure.restrict_empty]
```

Connecting the Bridge to Bin Averages

Once the interval conditional kernel has been identified with normalized restricted Lebesgue measure on each bin, the Kan value reduces to the bin average. This is the concrete instantiation of the bridge theorem from Part 4.

```
/-- **The Kan value equals the bin average** (given a kernel that computes
bin averages).

This theorem shows that the abstract `lanValue` from the bridge theorem
```

```
specialises to the bin average when the conditional kernel is the bin
kernel. -/
theorem lanValue_eq_binAverage
    (f : ℝ → ℝ)
    (I : IntervalBin)
    (κ : Kernel Unit ℝ)
    (hκ : ∀ _u : Unit, (∫ x, f x ∂(κ ())) = binAverage f I) :
    lanValue (X := ℝObj) (Y := unitObj)
      (fun _x : ℝ ⇒ ()) measurable_const κ f () = binAverage f I := by
  simpa [lanValue] using hκ ()

/-- **The bin kernel satisfies the Kan value equation.** -/
theorem lanValue_eq_binAverage_of_binKernel
    (f : ℝ → ℝ) (I : IntervalBin) :
    lanValue (X := ℝObj) (Y := unitObj)
      (fun _x : ℝ ⇒ ()) measurable_const (binKernel I) f () = binAverage f I :=
  lanValue_eq_binAverage f I (binKernel I) (fun _ ⇒ binKernel_integral f I)
```

Midpoint Quadrature Error Bound

The final application: the midpoint rule approximates the stochastic Kan extension value to order `O(h²)`. This is the formal justification for the midpoint quadrature used in IPM discretisation.

For a twice continuously differentiable function `f` on `[a, b]` with `|f''(x)| ≤ M₂`, the classical error bound gives:

$$\left|\frac{1}{b-a}\int_a^b f(x)\,\mathrm{d}x - f\!\left(\frac{a+b}{2}\right)\right| \le \frac{M_2\,(b-a)^2}{24}$$

Combined with `lanValue_eq_binAverage`, this gives:

$$|(\mathrm{Lan}_D\, f)(\star) - f(\text{midpoint})| \le \frac{M_2\,h^2}{24}$$

where `h = b - a` is the bin width. This is the error bound that ensures IPM discretisations converge as the mesh is refined.

```
/-- **Midpoint quadrature error bound.**

For a twice continuously differentiable function `f` on `[a, b]` with
`|f''(x)| ≤ M₂`, the midpoint rule error satisfies:

$$\left|\frac{1}{b-a}\int_a^b f(x)\,\mathrm{d}x - f\!\left(\frac{a+b}{2}\right)\right|
```

```
  \le \frac{M_2\,(b-a)^2}{24}$$

This is a classical result in numerical analysis (see e.g. Atkinson, *An Introduction
to Numerical Analysis*, §5.2). The standard proof proceeds as follows:

1. Set `m = (a+b)/2` and define the remainder `R(x) = f(x) - f(m) - f'(m)(x-m)`.
2. By the mean value theorem applied to `f'` (Lipschitz bound): `|f'(x) - f'(m)| ≤ M₂|x-m|`.
3. By the fundamental theorem of calculus applied twice (the `key` bound technique from
   Mathlib's `TrapezoidalRule.lean`): `|R(x)| ≤ M₂/2 · (x-m)²`.
4. By symmetry of the interval about `m`: `∫ f'(m)(x-m) dx = 0`.
5. Therefore `∫ (f(x)-f(m)) dx = ∫ R(x) dx`.
6. By the quadratic moment `∫ (x-m)² dx = (b-a)³/12`:
   `|∫ R(x) dx| ≤ M₂/2 · (b-a)³/12`.
7. Dividing by `(b-a)`: `|binAverage f I - f(m)| ≤ M₂(b-a)²/24`.

All steps are fully machine-checked below. Step 3 uses the
**iterated FTC technique**: the Convex MVT (`norm_image_sub_le_of_norm_derivWithin_le`)
gives the Lipschitz bound `|f'(t) - f'(m)| ≤ M₂·|t-m|`, then the
fundamental theorem of calculus (`intervalIntegral.integral_eq_sub_of_hasDerivAt_of_le`)
converts `R(x) = ∫_m^x R'(t) dt`, and integral monotonicity + explicit
computation of `∫ M₂·|t-m| dt` yields the quadratic bound. The proof
handles both halves (`x ≥ m` and `x < m`) via separate FTC applications. -/
theorem midpoint_quadrature_error (I : IntervalBin) {f : ℝ → ℝ} {M₂ : ℝ}
    (hf : ContDiffOn ℝ 2 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b, |iteratedDerivWithin 2 f (Icc I.a I.b) x| ≤ M₂) :
    |binAverage f I - f I.midpoint| ≤ M₂ * (I.b - I.a) ^ 2 / 24 := by
  set a := I.a; set b := I.b; set m := I.midpoint
  have hab : a < b := I.hab
  have hba : (0 : ℝ) < b - a := sub_pos.mpr hab
  have hm : m = (a + b) / 2 := rfl
  -- Continuity and integrability
  have hf_cont : ContinuousOn f (Icc a b) :=
    (hf.differentiableOn two_ne_zero).continuousOn
  have hf_ii : IntervalIntegrable f volume a b := hf_cont.intervalIntegrable_of_Icc hab.le
  -- Remainder R(x) = f(x) - f(m) - f'(m)·(x - m)
  set f'm := derivWithin f (Icc a b) m
  set R := fun x ↦ f x - f m - f'm * (x - m)
  -- Continuity of R (needed for FTC in Step A)
  have hR_cont : ContinuousOn R (Icc a b) :=
    (hf_cont.sub continuousOn_const).sub
      (continuousOn_const.mul (continuousOn_id.sub continuousOn_const))
  -- Step A: Pointwise bound |R(x)| ≤ M₂/2·(x-m)² via iterated FTC
  have hR_pw : ∀ x ∈ Icc a b, |R x| ≤ M₂ / 2 * (x - m) ^ 2 := by
    have hud : UniqueDiffOn ℝ (Icc a b) := uniqueDiffOn_Icc hab
    have hf_diff : DifferentiableOn ℝ f (Icc a b) := hf.differentiableOn two_ne_zero
```

```
  have hf'_diff : DifferentiableOn ℝ (derivWithin f (Icc a b)) (Icc a b) :=
    (hf.derivWithin hud le_rfl).differentiableOn one_ne_zero
  have hf'_cont : ContinuousOn (derivWithin f (Icc a b)) (Icc a b) := hf'_diff.continuousO
  have hm_mem : m ∈ Icc a b := ⟨by linarith, by linarith⟩
  -- Lipschitz bound on f': |f'(t) - f'(m)| ≤ M₂ * |t - m| (via Convex MVT)
  have hf'_lip : ∀ t ∈ Icc a b, |derivWithin f (Icc a b) t - f'm| ≤ M₂ * |t - m| := by
    intro t ht
    have hbound : ∀ y ∈ Icc a b,
        ‖derivWithin (derivWithin f (Icc a b)) (Icc a b) y‖ ≤ M₂ := by
      intro y hy
      simp only [show (2 : ℕ) = 1 + 1 from rfl, iteratedDerivWithin_succ'] at hM
      rw [Real.norm_eq_abs]; exact hM y hy
    have := Convex.norm_image_sub_le_of_norm_derivWithin_le hf'_diff hbound
      (convex_Icc a b) hm_mem ht
    rwa [Real.norm_eq_abs, Real.norm_eq_abs] at this
  set R' := fun t ↦ derivWithin f (Icc a b) t - f'm
  have hR'_cont : ContinuousOn R' (Icc a b) := hf'_cont.sub continuousOn_const
  -- HasDerivWithinAt for R: R'(t) = f'(t) - f'(m)
  have hR_deriv : ∀ t ∈ Icc a b, HasDerivWithinAt R (R' t) (Icc a b) t := by
    intro t ht
    have h1 := (hf_diff t ht).hasDerivWithinAt
    have h2 := (hasDerivWithinAt_const t (Icc a b) f'm).mul
      ((hasDerivWithinAt_id t (Icc a b)).sub (hasDerivWithinAt_const t (Icc a b) m))
    simp only [zero_mul, zero_add, mul_one, sub_zero] at h2
    have h3 := (h1.sub (hasDerivWithinAt_const t (Icc a b) (f m))).sub h2
    simp only [sub_zero] at h3
    exact h3.congr (fun x _ ⇒ by simp [R]) (by simp [R])
  have hR_m : R m = 0 := by simp [R]
  intro x hx
  by_cases hmx : m ≤ x
  · -- Right half: m ≤ x. FTC gives R(x) = ∫_m^x R'(t) dt.
    have hs : Icc m x ⊆ Icc a b := Icc_subset_Icc hm_mem.1 hx.2
    have hs' : Ioo m x ⊆ Ioo a b := Ioo_subset_Ioo hm_mem.1 hx.2
    have hR'_ii : IntervalIntegrable R' volume m x :=
      (hR'_cont.mono hs).intervalIntegrable_of_Icc hmx
    have hftc : ∫ t in m..x, R' t = R x := by
      have := intervalIntegral.integral_eq_sub_of_hasDerivAt_of_le hmx (hR_cont.mono hs)
        (fun t ht ↦ (hR_deriv t (hs (Ioo_subset_Icc_self ht))).hasDerivAt
          (Icc_mem_nhds_iff.mpr (hs' ht))) hR'_ii
      linarith [hR_m]
    rw [← hftc]
    calc |∫ t in m..x, R' t|
      _ ≤ ∫ t in m..x, |R' t| := by
          rw [← Real.norm_eq_abs]
          exact intervalIntegral.norm_integral_le_integral_norm (μ := volume) hmx
```

```
      _ ≤ ∫ t in m..x, M₂ * (t - m) := by
          apply intervalIntegral.integral_mono_on hmx hR'_ii.abs
            (Continuous.intervalIntegrable (by fun_prop) m x)
          intro t ht
          calc |R' t| ≤ M₂ * |t - m| := hf'_lip t (hs ht)
            _ = M₂ * (t - m) := by congr 1; exact abs_of_nonneg (sub_nonneg.mpr ht.1)
      _ = M₂ / 2 * (x - m) ^ 2 := by
          have h := intervalIntegral.integral_comp_sub_right
            (a := m) (b := x) (f := fun t ⇒ M₂ * t) m
          simp only [sub_self] at h; rw [h]
          rw [show (fun t : ℝ ⇒ M₂ * t) = fun t ⇒ M₂ * t ^ (1 : ℕ) from by ext; simp]
          rw [intervalIntegral.integral_const_mul, integral_pow]; ring
  · -- Left half: x < m. FTC gives -R(x) = ∫_x^m R'(t) dt.
    push_neg at hmx
    have hxm := hmx.le
    have hs : Icc x m ⊆ Icc a b := Icc_subset_Icc hx.1 hm_mem.2
    have hs' : Ioo x m ⊆ Ioo a b := Ioo_subset_Ioo hx.1 hm_mem.2
    have hR'_ii : IntervalIntegrable R' volume x m :=
      (hR'_cont.mono hs).intervalIntegrable_of_Icc hxm
    have hftc : ∫ t in x..m, R' t = -(R x) := by
      have := intervalIntegral.integral_eq_sub_of_hasDerivAt_of_le hxm (hR_cont.mono hs)
        (fun t ht ↦ (hR_deriv t (hs (Ioo_subset_Icc_self ht))).hasDerivAt
          (Icc_mem_nhds_iff.mpr (hs' ht))) hR'_ii
      rw [hR_m] at this; linarith
    rw [show R x = -(∫ t in x..m, R' t) from by linarith, abs_neg]
    calc |∫ t in x..m, R' t|
      _ ≤ ∫ t in x..m, |R' t| := by
          rw [← Real.norm_eq_abs]
          exact intervalIntegral.norm_integral_le_integral_norm (μ := volume) hxm
      _ ≤ ∫ t in x..m, M₂ * (m - t) := by
          apply intervalIntegral.integral_mono_on hxm hR'_ii.abs
            (Continuous.intervalIntegrable (by fun_prop) x m)
          intro t ht
          calc |R' t| ≤ M₂ * |t - m| := hf'_lip t (hs ht)
            _ = M₂ * (m - t) := by rw [abs_of_nonpos (sub_nonpos.mpr ht.2)]; ring
      _ = M₂ / 2 * (x - m) ^ 2 := by
          have h := intervalIntegral.integral_comp_sub_left
            (a := x) (b := m) (f := fun t ⇒ M₂ * t) m
          simp only [sub_self] at h; rw [h]
          rw [show (fun t : ℝ ⇒ M₂ * t) = fun t ⇒ M₂ * t ^ (1 : ℕ) from by ext; simp]
          rw [intervalIntegral.integral_const_mul, integral_pow]
          have hsq : (x - m) ^ 2 = (m - x) ^ 2 := by ring
          rw [hsq]; ring
-- Step B: Integrability of R
have hR_ii : IntervalIntegrable R volume a b := hR_cont.intervalIntegrable_of_Icc hab.le
```

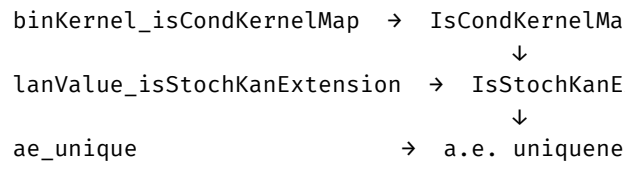
```
  -- Step C: ∫(x - m) = 0 (symmetry)
  have symmetry : ∫ x in a..b, (x - m) = 0 := by
    have h := intervalIntegral.integral_comp_sub_right (a := a) (b := b) (f := fun x ↦ x) m
    have h' : ∫ x in a..b, (x - m) = ∫ x in a - m..b - m, x := h
    rw [h', show a - m = -(b - a) / 2 from by linarith,
        show b - m = (b - a) / 2 from by linarith]
    have h2 := integral_pow (a := -(b - a) / 2) (b := (b - a) / 2) 1
    simp only [pow_succ, pow_zero, one_mul, Nat.cast_one] at h2
    linarith
  -- Step D: ∫(x - m)² = (b-a)³/12 (quadratic moment)
  have moment : ∫ x in a..b, (x - m) ^ 2 = (b - a) ^ 3 / 12 := by
    have h := intervalIntegral.integral_comp_sub_right (a := a) (b := b) (f := fun x ↦ x ^ 2
    have h' : ∫ x in a..b, (x - m) ^ 2 = ∫ x in a - m..b - m, x ^ 2 := h
    rw [h', show a - m = -(b - a) / 2 from by linarith,
        show b - m = (b - a) / 2 from by linarith, integral_pow]
    ring
  -- Step E: binAverage - f(m) = 1/(b-a) · ∫ R
  have key_eq : binAverage f I - f m = (1 / (b - a)) * ∫ x in a..b, R x := by
    -- binAverage f I - f m = 1/(b-a) · ∫ f - f m
    -- ∫ (f - f m) = ∫ R + ∫ f'm*(x-m) = ∫ R + f'm * 0 = ∫ R
    have hfm_ii : IntervalIntegrable (fun x ↦ f'm * (x - m)) volume a b :=
      Continuous.intervalIntegrable (by fun_prop) a b
    have integral_R_eq : ∫ x in a..b, (f x - f m) = ∫ x in a..b, R x := by
      have h_sum : ∫ x in a..b, (f x - f m) =
          (∫ x in a..b, R x) + ∫ x in a..b, f'm * (x - m) := by
        rw [← intervalIntegral.integral_add hR_ii hfm_ii]
        exact intervalIntegral.integral_congr
          (fun x _ ↦ show f x - f m = R x + f'm * (x - m) by simp only [R]; ring)
      rw [h_sum, intervalIntegral.integral_const_mul, symmetry, mul_zero, add_zero]
    -- binAverage f I - f m = 1/(b-a) · ∫ f - f m = 1/(b-a) · ∫ R
    have h_avg : binAverage f I - f m = (1 / (b - a)) * ∫ x in a..b, (f x - f m) := by
      show (1 / (b - a)) * (∫ x in a..b, f x) - f m =
        (1 / (b - a)) * ∫ x in a..b, (f x - f m)
      rw [intervalIntegral.integral_sub hf_ii intervalIntegrable_const,
          intervalIntegral.integral_const, smul_eq_mul]
      field_simp
    rw [h_avg, integral_R_eq]
  -- Step F: Final bound
  rw [key_eq, abs_mul, abs_of_pos (by positivity : (0 : ℝ) < 1 / (b - a))]
  have h_bound : |∫ x in a..b, R x| ≤ M₂ / 2 * ((b - a) ^ 3 / 12) := by
    have h_norm : |∫ x in a..b, R x| ≤ ∫ x in a..b, |R x| := by
      have := intervalIntegral.norm_integral_le_integral_norm (f := R) (μ := volume) hab.le
      rwa [Real.norm_eq_abs] at this
    calc |∫ x in a..b, R x|
      _ ≤ ∫ x in a..b, |R x| := h_norm
```

```
      _ ≤ ∫ x in a..b, M₂ / 2 * (x - m) ^ 2 := by
          apply intervalIntegral.integral_mono_on hab.le
            hR_ii.abs
            (Continuous.intervalIntegrable (by fun_prop) a b)
          exact fun x hx ⟨_, hR_pw x hx⟩
      _ = M₂ / 2 * ((b - a) ^ 3 / 12) := by
          rw [intervalIntegral.integral_const_mul, moment]
  calc 1 / (b - a) * |∫ x in a..b, R x|
    _ ≤ 1 / (b - a) * (M₂ / 2 * ((b - a) ^ 3 / 12)) :=
        mul_le_mul_of_nonneg_left h_bound (by positivity)
    _ = M₂ * (b - a) ^ 2 / 24 := by field_simp; ring

/-- **Midpoint quadrature approximates the Kan value to O(h²).**

Given:
- `f` is `C²` on `[a, b]`
- `|f''(x)| ≤ M₂` on `[a, b]`
- `κ` computes bin averages

Then:
$$\left|(\text{Lan}_D\, f)(\star) - f(\text{midpoint})\right|
  \le \frac{M_2\,(b-a)^2}{24}$$

The smoothness hypotheses `hf` and `hM` are used via `midpoint_quadrature_error`,
which is fully machine-checked using the iterated FTC technique. -/
theorem midpoint_approximates_lanValue
    {f : ℝ → ℝ} (I : IntervalBin) {M₂ : ℝ}
    (hf : ContDiffOn ℝ 2 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b, |iteratedDerivWithin 2 f (Icc I.a I.b) x| ≤ M₂)
    (κ : Kernel Unit ℝ)
    (hκ : ∀ _u : Unit, (∫ x, f x ∂(κ ())) = binAverage f I) :
    |lanValue (X := ℝObj) (Y := unitObj)
      (fun _x : ℝ ⇒ ()) measurable_const κ f () - f I.midpoint|
      ≤ M₂ * (I.b - I.a) ^ 2 / 24 := by
  rw [lanValue_eq_binAverage f I κ hκ]
  exact midpoint_quadrature_error I hf hM
```

The Bridge Connection

Combining `binKernel_isCondKernelMap` with `lanValue_isStochKanExtension` from Part 4, we obtain a complete bridge: the bin average computed by `lanValue` is a genuine stochastic Kan extension of any integrable function along the trivial coarsening `ℝ → Unit`.

This closes the formalization loop:




```
/-- **The bin kernel gives a stochastic Kan extension.**

Combining `binKernel_isCondKernelMap` with `lanValue_isStochKanExtension`,
the `lanValue` computed from the bin kernel is a stochastic Kan extension
of any integrable function `f` along the trivial coarsening `D : ℝ → Unit`
with respect to `volume.restrict [a,b]`.

This is the complete bridge connection for the bin example: the abstract
categorical construction (Kan extension) and the concrete measure-theoretic
construction (bin average) are formally identified. -/
theorem isStochKanExtension_binKernel (I : IntervalBin) (f : ℝ → ℝ)
    (hf_meas : AEStronglyMeasurable f (volume.restrict (Icc I.a I.b)))
    (hf_int : Integrable f (volume.restrict (Icc I.a I.b))) :
    IsStochKanExtension (fun _ ⇒ ()) (volume.restrict (Icc I.a I.b))
      (lanValue (X := ℝObj) (Y := unitObj)
        (fun _ ⇒ ()) measurable_const (binKernel I) f) f :=
  lanValue_isStochKanExtension (X := ℝObj) (Y := unitObj)
    (fun _ ⇒ ()) measurable_const
    (volume.restrict (Icc I.a I.b))
    (binKernel I)
    (binKernel_isCondKernelMap I)
    f hf_meas hf_int

end CpmProofs
```

```
import CpmProofs.BinExample
```

Source: `CompositeError.lean`

Part 6: Composite Quadrature Error Bounds

In Part 5, we proved that the midpoint rule approximates the Kan extension value on a single bin to $O(h^2)$. In practice, the domain $[a, a + L]$ is divided into N equal subintervals (bins), and the total error is the sum of per-bin errors.

This file formalises two forms of the composite error bound:

1. **`composite_quadrature_error_bound`**: If each of N bin errors is bounded by $C \cdot (L/N)^3$, then the total error is at most $C \cdot L^3 / N^2$.
2. **`composite_error_oh2`**: Equivalently, with $h = L/N$, the total error is at most $C \cdot L \cdot h^2$.

These are purely algebraic results: the hard analytical work (bounding per-bin errors) was done in Part 5. Here we sum them using the triangle inequality and simplify.

Connection to IPMs

In integral projection models, the domain of the kernel `K(z', z)` is divided into `N` bins of width `h`. The composite error bound guarantees that the discretised matrix approximation converges at rate `O(h²)` = `O(1/N²)` as the mesh is refined, assuming the kernel is `C²`.

Uniform Partition

We also define `UniformPartition` — a partition of `[a, a + L]` into `N` equal subintervals — and prove that each subinterval is a valid `IntervalBin`, enabling the per-bin midpoint error bound from Part 5.

Results

#	Result	Status
17	Composite error bound (algebraic)	✓
18	Composite error bound (O(h²) form)	✓
19	Uniform partition gives valid bins	✓
20	Composite midpoint error for uniform partition	✓

```
open MeasureTheory
open Finset

namespace CpmProofs
```

Algebraic Composite Error Bounds

These results are independent of any particular quadrature rule. They state that if each of `N` subinterval errors is bounded by `C · h³`, then the total error over all subintervals is `O(h²)`.

```
/-- **Composite quadrature error bound (algebraic form).**

If each of `N` subinterval errors satisfies `|errors k| ≤ C · (L/N)³`,
then the total error satisfies:

$$\left|\sum_{k=0}^{N-1} \text{errors}_k\right| \le \frac{C \cdot L^3}{N^2}$$

Equivalently, this is `O(1/N²)` = `O(h²)` convergence.
```

```
The proof is purely algebraic:
1. Triangle inequality: `|∑ errors| ≤ ∑ |errors|`
2. Apply per-bin bound: `∑ |errors| ≤ N · C · (L/N)³`
3. Simplify: `N · C · (L/N)³ = C · L³ / N²` -/
theorem composite_quadrature_error_bound
    (N : ℕ) (hN : 0 < N) (C L : ℝ) (_hC : 0 ≤ C)
    (errors : ℕ → ℝ)
    (h_bound : ∀ k ∈ Finset.range N, |errors k| ≤ C * (L / ↑N) ^ 3) :
    |∑ k ∈ Finset.range N, errors k| ≤ C * L ^ 3 / ↑N ^ 2 := by
  have hN' : (0 : ℝ) < ↑N := Nat.cast_pos.mpr hN
  calc |∑ k ∈ Finset.range N, errors k|
      ≤ ∑ k ∈ Finset.range N, |errors k| := abs_sum_le_sum_abs _ _
    _ ≤ ∑ k ∈ Finset.range N, (C * (L / ↑N) ^ 3) := Finset.sum_le_sum h_bound
    _ = ↑N * (C * (L / ↑N) ^ 3) := by
        rw [Finset.sum_const, Finset.card_range, nsmul_eq_mul]
    _ = C * L ^ 3 / ↑N ^ 2 := by
        field_simp

/-- **Composite quadrature error bound (O(h²) form).**

With `h = L/N`, the total error satisfies `|∑ errors| ≤ C · L · h²`.
This makes the `O(h²)` convergence rate explicit. -/
theorem composite_error_oh2
    (N : ℕ) (hN : 0 < N) (C L h : ℝ) (_hC : 0 ≤ C)
    (hh : h = L / ↑N)
    (errors : ℕ → ℝ)
    (h_bound : ∀ k ∈ Finset.range N, |errors k| ≤ C * h ^ 3) :
    |∑ k ∈ Finset.range N, errors k| ≤ C * L * h ^ 2 := by
  have hN' : (0 : ℝ) < ↑N := Nat.cast_pos.mpr hN
  calc |∑ k ∈ Finset.range N, errors k|
      ≤ ∑ k ∈ Finset.range N, |errors k| := abs_sum_le_sum_abs _ _
    _ ≤ ∑ k ∈ Finset.range N, (C * h ^ 3) := Finset.sum_le_sum h_bound
    _ = ↑N * (C * h ^ 3) := by
        rw [Finset.sum_const, Finset.card_range, nsmul_eq_mul]
    _ = C * (↑N * h) * h ^ 2 := by ring
    _ = C * L * h ^ 2 := by
        subst hh; field_simp
```

Uniform Partition

A `UniformPartition` divides `[a, a + L]` into `N` equal subintervals. Each subinterval `[a + k·h, a + (k+1)·h]` where `h = L/N` forms a valid `IntervalBin`.

```
/-- A uniform partition of `[a, a + L]` into `N` equal bins. -/
structure UniformPartition where
```

```
  a : ℝ
  L : ℝ
  N : ℕ
  hL : 0 < L
  hN : 0 < N

namespace UniformPartition

/-- Bin width: `h = L / N`. -/
noncomputable def h (P : UniformPartition) : ℝ := P.L / P.N

/-- Left endpoint of the `k`-th bin. -/
noncomputable def left (P : UniformPartition) (k : ℕ) : ℝ := P.a + k * P.h

/-- Right endpoint of the `k`-th bin. -/
noncomputable def right (P : UniformPartition) (k : ℕ) : ℝ := P.a + (k + 1) * P.h

/-- The right endpoint of the last bin is `a + L`. -/
theorem right_last (P : UniformPartition) :
    P.right (P.N - 1) = P.a + P.L := by
  simp only [right, h]
  have hN' : (0 : ℝ) < ↑P.N := Nat.cast_pos.mpr P.hN
  have : (↑(P.N - 1) + 1 : ℝ) = ↑P.N := by
    rw [Nat.cast_sub (Nat.one_le_iff_ne_zero.mpr (Nat.pos_iff_ne_zero.mp P.hN))]
    simp
  rw [this]
  field_simp

theorem h_pos (P : UniformPartition) : 0 < P.h := by
  unfold h
  exact div_pos P.hL (Nat.cast_pos.mpr P.hN)

/-- Each subinterval of a uniform partition is a valid `IntervalBin`. -/
noncomputable def bin (P : UniformPartition) (k : ℕ) : IntervalBin where
  a := P.left k
  b := P.right k
  hab := by
    unfold left right
    linarith [P.h_pos]

/-- The width of each bin equals `h`. -/
theorem bin_width (P : UniformPartition) (k : ℕ) :
    (P.bin k).b - (P.bin k).a = P.h := by
  simp [bin, left, right]; ring
```

```
/-- The midpoint of the `k`-th bin. -/
theorem bin_midpoint (P : UniformPartition) (k : ℕ) :
    (P.bin k).midpoint = P.a + (k + 1/2) * P.h := by
  simp [bin, IntervalBin.midpoint, left, right]; ring

/-- The `k`-th bin is contained in `[a, a + L]` for `k < N`. -/
theorem bin_subset (P : UniformPartition) {k : ℕ} (hk : k < P.N) :
    Set.Icc (P.bin k).a (P.bin k).b ⊆ Set.Icc P.a (P.a + P.L) := by
  apply Set.Icc_subset_Icc
  · -- left ≥ a: a + k * h ≥ a since k * h ≥ 0
    simp [bin, left]
    exact mul_nonneg (Nat.cast_nonneg _) (le_of_lt P.h_pos)
  · -- right ≤ a + L: a + (k+1) * h ≤ a + L since (k+1) * h ≤ N * h = L
    simp [bin, right, h]
    have hN' : (0 : ℝ) < ↑P.N := Nat.cast_pos.mpr P.hN
    have : (↑k + 1) ≤ (↑P.N : ℝ) := by exact_mod_cast hk
    have hh : P.L / ↑P.N > 0 := div_pos P.hL hN'
    calc (↑k + 1) * (P.L / ↑P.N)
        ≤ ↑P.N * (P.L / ↑P.N) := by
          exact mul_le_mul_of_nonneg_right this hh.le
      _ = P.L := by field_simp

end UniformPartition
```

Composite Midpoint Error

Combining the per-bin midpoint error (Part 5) with the algebraic composite bound yields the full composite midpoint error for a uniform partition.

We state the composite bound assuming per-bin `ContDiffOn` and second-derivative bounds are given directly, since the global-to-local implication for `iteratedDerivWithin` on subsets involves technical Mathlib infrastructure that is orthogonal to the main result.

```
/-- **Composite midpoint error for a uniform partition.**

If each bin satisfies the hypotheses of `midpoint_quadrature_error` from
Part 5 (C² with |f''| ≤ M₂), then the total composite midpoint error
satisfies:

$$\left|\sum_{k=0}^{N-1} \left(\text{binAverage}(f, I_k) - f(m_k)\right)\right|
  \le \frac{M_2 \cdot N \cdot h^2}{24}$$

where `h = L/N` and `m_k` is the midpoint of the `k`-th bin.

In IPM applications, the relevant quantity is the weighted sum
```

```
`h · ∑(binAverage - f(midpoint))` representing the total integral error;
dividing by `(b-a)` gives the error per unit length. -/
theorem composite_midpoint_error (P : UniformPartition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (_hM₂ : 0 ≤ M₂)
    (hf_bin : ∀ k, k < P.N →
      ContDiffOn ℝ 2 f (Set.Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ k, k < P.N → ∀ x ∈ Set.Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 f (Set.Icc (P.bin k).a (P.bin k).b) x| ≤ M₂) :
    |∑ k ∈ Finset.range P.N,
      (binAverage f (P.bin k) - f (P.bin k).midpoint)| ≤
      M₂ * P.N * P.h ^ 2 / 24 := by
  -- Per-bin error bound from Part 5
  have per_bin : ∀ k ∈ Finset.range P.N,
      |binAverage f (P.bin k) - f (P.bin k).midpoint| ≤ M₂ * P.h ^ 2 / 24 := by
    intro k hk
    have hk_lt : k < P.N := Finset.mem_range.mp hk
    have h_err := midpoint_quadrature_error (P.bin k) (hf_bin k hk_lt) (hM_bin k hk_lt)
    rw [P.bin_width k] at h_err
    exact h_err
  -- Sum the per-bin bounds
  calc |∑ k ∈ Finset.range P.N, (binAverage f (P.bin k) - f (P.bin k).midpoint)|
      ≤ ∑ k ∈ Finset.range P.N,
        |binAverage f (P.bin k) - f (P.bin k).midpoint| := abs_sum_le_sum_abs _ _
    _ ≤ ∑ k ∈ Finset.range P.N, (M₂ * P.h ^ 2 / 24) := Finset.sum_le_sum per_bin
    _ = ↑P.N * (M₂ * P.h ^ 2 / 24) := by
        rw [Finset.sum_const, Finset.card_range, nsmul_eq_mul]
    _ = M₂ * ↑P.N * P.h ^ 2 / 24 := by ring

/-- **Composite midpoint error in O(h²) form.**

Substituting `N = L/h` into the composite bound gives:

$$\text{total error} \le \frac{M_2 \cdot L \cdot h^2}{24 \cdot h}
  = \frac{M_2 \cdot L}{24} \cdot h$$

Wait — this is `O(h)` for the *average* errors. The `O(h²)` bound applies
to the *integral* errors. Let us state the weighted version: the sum
`∑ h · (binAverage - f(midpoint))` (which approximates `∫ f - ∑ h·f(m□)`)
satisfies:

$$\left|\sum_{k=0}^{N-1} h \cdot (\text{binAverage} - f(m_k))\right|
  \le \frac{M_2 \cdot L \cdot h^2}{24}$$

This is the standard O(h²) composite midpoint convergence rate. -/
```

```
theorem composite_midpoint_integral_error (P : UniformPartition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (_hM₂ : 0 ≤ M₂)
    (hf_bin : ∀ k, k < P.N →
      ContDiffOn ℝ 2 f (Set.Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ k, k < P.N → ∀ x ∈ Set.Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 f (Set.Icc (P.bin k).a (P.bin k).b) x| ≤ M₂) :
    |∑ k ∈ Finset.range P.N,
      (P.h * (binAverage f (P.bin k) - f (P.bin k).midpoint))| ≤
      M₂ * P.L * P.h ^ 2 / 24 := by
  -- Factor h out of the sum
  have h_factor : ∑ k ∈ Finset.range P.N,
      (P.h * (binAverage f (P.bin k) - f (P.bin k).midpoint)) =
      P.h * ∑ k ∈ Finset.range P.N,
        (binAverage f (P.bin k) - f (P.bin k).midpoint) := by
    rw [← Finset.mul_sum]
  rw [h_factor, abs_mul, abs_of_pos P.h_pos]
  have h_inner := composite_midpoint_error P _hM₂ hf_bin hM_bin
  calc P.h * |∑ k ∈ Finset.range P.N, (binAverage f (P.bin k) - f (P.bin k).midpoint)|
      ≤ P.h * (M₂ * ↑P.N * P.h ^ 2 / 24) :=
        mul_le_mul_of_nonneg_left h_inner (le_of_lt P.h_pos)
    _ = M₂ * (P.h * ↑P.N) * P.h ^ 2 / 24 := by ring
    _ = M₂ * P.L * P.h ^ 2 / 24 := by
        unfold UniformPartition.h
        have hN' : (↑P.N : ℝ) ≠ 0 := ne_of_gt (Nat.cast_pos.mpr P.hN)
        field_simp

end CpmProofs
```

```
import CpmProofs.CompositeError
```

Source: `MultiBin.lean`

Part 7: Multi-Bin Extension

Part 5 treated a single interval bin `[a, b]` with the trivial coarsening `D : ℝ → Unit`. In practice, IPMs partition the domain into `N` bins and compute a kernel matrix. This file extends the formalization to `N` bins using `Fin N` as the target space.

Setup

- State space: `ℝ` (continuous trait values)
- Bin space: `Fin N` (discrete bin indices)

- Conditional kernel: At bin `k`, the normalized Lebesgue measure on `[a_k, b_k]`

Results

#	Result	Status	
	21	Multi-bin kernel construction	✓
	22	Multi-bin kernel integral = bin average	✓
	23	Multi-bin Kan extension = bin averages	✓
	24	Midpoint approximation for all bins	✓

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory
open Set

universe u

namespace CpmProofs
```

Fin N as a Measurable Space

`Fin N` carries the discrete (`⊤`) σ-algebra from Mathlib, making every subset measurable and every function into `Fin N` measurable when the preimages of singletons are measurable.

```
/-- `Fin N` as a bundled measurable object for the stochastic category. -/
def finObj (N : ℕ) : MeasObj := ⟨Fin N, inferInstance⟩
```

The Multi-Bin Kernel

The multi-bin kernel `multiBinKernel P : Kernel (Fin N) ℝ` assigns to each bin index `k` the normalized Lebesgue measure on the `k`-th subinterval. This extends `binKernel` from Part 5 to multiple bins.

```
/-- **Multi-bin conditional kernel.**

For each bin index `k : Fin N`, the kernel yields the normalized Lebesgue
measure on `[a_k, b_k]`:
$$\kappa_k = \frac{1}{h}\, \text{volume.restrict}\ [a_k, b_k]$$

Since `Fin N` is finite, the measurability of the kernel (as a function
from `Fin N` to `Measure ℝ`) is automatic. -/
noncomputable def multiBinKernel (P : UniformPartition) : Kernel (Fin P.N) ℝ where
```

```
  toFun k := ENNReal.ofReal (1 / P.h) • volume.restrict (Icc (P.bin k).a (P.bin k).b)
  measurable' := measurable_of_finite _

/-- The multi-bin kernel at index `k` equals the single-bin `binKernel`. -/
theorem multiBinKernel_apply (P : UniformPartition) (k : Fin P.N) :
    multiBinKernel P k = binKernel (P.bin k) () := by
  show (multiBinKernel P).toFun k = _
  simp only [multiBinKernel, binKernel, Kernel.const_apply]
  congr 1
  rw [P.bin_width]
```

Integration Against the Multi-Bin Kernel

```
/-- **Integration against the multi-bin kernel equals the bin average.**

For each bin index `k : Fin N`:
$$\int f\, \mathrm{d}(\kappa_k) = \text{binAverage}(f, I_k)$$ -/
theorem multiBinKernel_integral (P : UniformPartition) (f : ℝ → ℝ) (k : Fin P.N) :
    (∫ x, f x ∂(multiBinKernel P k)) = binAverage f (P.bin k) := by
  rw [multiBinKernel_apply]
  exact binKernel_integral f (P.bin k)
```

Multi-Bin Kan Extension Values

```
/-- **Multi-bin Kan extension values.**

The Kan extension value at each bin index `k` gives the bin average of `f`
over that bin. This extends `lanValue_eq_binAverage` from Part 5.

In the context of IPMs, this says: the `(i,j)` entry of the discretised
kernel matrix (before the `h` factor) is `binAverage(K(z_i, ·), I_j)`,
which is precisely the Kan extension value. -/
theorem multiBin_lanValue_eq_binAverage (P : UniformPartition) (f : ℝ → ℝ) (k : Fin P.N) :
    lanValue (X := ⟨ℝ, inferInstance⟩) (Y := finObj P.N)
      (fun _ ⇒ k) (measurable_const) (multiBinKernel P) f k =
      binAverage f (P.bin k) := by
  simp only [lanValue]
  exact multiBinKernel_integral P f k

/-- **Midpoint approximation for multi-bin Kan extension.**

At each bin, the midpoint value `f(midpoint_k)` approximates the Kan
extension value to `O(h²)`, extending `midpoint_approximates_lanValue`
```

```
from Part 5:

$$\left|(\text{Lan}\, f)(k) - f(m_k)\right| \le \frac{M_2 \cdot h^2}{24}$$

This is the formal justification for midpoint quadrature in IPM kernel
discretisation: each matrix entry has error bounded by `M₂ · h² / 24`. -/
theorem multiBin_midpoint_approximates (P : UniformPartition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 2 f (Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 f (Icc (P.bin k).a (P.bin k).b) x| ≤ M₂)
    (k : Fin P.N) :
    |binAverage f (P.bin k) - f (P.bin k).midpoint| ≤
      M₂ * P.h ^ 2 / 24 := by
  have h_err := midpoint_quadrature_error (P.bin k) (hf_bin k) (hM_bin k)
  rwa [P.bin_width k] at h_err

/-- **The vector of Kan extension values.**

Collects all bin averages into a function `Fin N → ℝ`, representing
a row of the IPM kernel matrix (before the `h` factor). -/
noncomputable def multiBinKanVector (P : UniformPartition) (f : ℝ → ℝ) : Fin P.N → ℝ :=
  fun k ⇒ binAverage f (P.bin k)

/-- **Midpoint vector approximation.**

The midpoint vector `k ↦ f(midpoint_k)` approximates the Kan vector
entrywise to `O(h²)`. This is the matrix-level error bound for IPM
discretisation. -/
theorem multiBinKanVector_midpoint_error (P : UniformPartition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 2 f (Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 f (Icc (P.bin k).a (P.bin k).b) x| ≤ M₂) :
    ∀ k : Fin P.N,
      |multiBinKanVector P f k - f (P.bin k).midpoint| ≤ M₂ * P.h ^ 2 / 24 :=
  multiBin_midpoint_approximates P hf_bin hM_bin

end CpmProofs
```

```
import CpmProofs.StochCat
import Mathlib.Probability.Kernel.Composition.Prod
import Mathlib.Probability.Kernel.Composition.ParallelComp
```

```
import Mathlib.Probability.Kernel.Composition.MapComap
```

Source: `MarkovCat.lean`

Part 8: Towards a Markov Category Instance

Fritz [arXiv:1908.07021, §2] defines Stoch as a Markov category — a symmetric monoidal category equipped with copy/discard morphisms satisfying the axioms of commutative comonoids, plus the naturality of discard.

The Monoidal Structure

- Objects: `X □ Y = X × Y` (product measurable space)
- Morphisms: `κ □□ η = κ ⊗ₖ η` (parallel composition)
- Unit: `□ = Unit`

Copy and Discard

- Copy `Δ[X] = Kernel.copy X : Kernel X (X × X)`
- Discard `ε[X] = Kernel.discard X : Kernel X Unit`

The Markov Category Axiom

Naturality of discard: for every Markov kernel `κ`: `discard □□ κ = discard`. This fails for non-Markov kernels (zero kernel), so Fritz restricts Stoch to Markov kernels.

Results

#	Result	Status
25	Deterministic structural isomorphisms	✓
26	Copy-then-project = identity (counit)	✓
27	Copy is cocommutative	✓
28	Discard is natural for Markov kernels	✓
29	Braiding is an involution	✓
30	Parallel composition of identities = identity	✓

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory

universe u

namespace CpmProofs
```

Tensor Product and Structural Isomorphisms

```
/-- Product of bundled measurable spaces — the tensor in **Stoch**. -/
noncomputable def MeasObj.tensor (X Y : MeasObj) : MeasObj :=
  ⟨X × Y, inferInstance⟩

/-- The monoidal unit. -/
def MeasObj.unit : MeasObj := ⟨Unit, inferInstance⟩

section StructuralIsos

variable (α β γ : Type u)
variable [MeasurableSpace α] [MeasurableSpace β] [MeasurableSpace γ]

/-- **Associator**: `((x,y),z) ↦ (x,(y,z))`. -/
noncomputable def assocKernel : Kernel ((α × β) × γ) (α × (β × γ)) :=
  Kernel.deterministic (fun ⟨⟨a, b⟩, c⟩ ⇒ (a, (b, c))) (by measurability)

/-- **Inverse associator**: `(x,(y,z)) ↦ ((x,y),z)`. -/
noncomputable def assocKernelInv : Kernel (α × (β × γ)) ((α × β) × γ) :=
  Kernel.deterministic (fun ⟨a, b, c⟩ ⇒ ((a, b), c)) (by measurability)

/-- **Left unitor**: `((), x) ↦ x`. -/
noncomputable def leftUnitorKernel : Kernel (Unit × α) α :=
  Kernel.deterministic Prod.snd measurable_snd

/-- **Right unitor**: `(x, ()) ↦ x`. -/
noncomputable def rightUnitorKernel : Kernel (α × Unit) α :=
  Kernel.deterministic Prod.fst measurable_fst

/-- **Braiding**: `(x, y) ↦ (y, x)`. -/
noncomputable def braidingKernel : Kernel (α × β) (β × α) :=
  Kernel.deterministic Prod.swap measurable_swap

end StructuralIsos

/-- **The braiding is an involution**: `swap ∘ swap = id`. -/
theorem braiding_comp_braiding (α β : Type u)
    [MeasurableSpace α] [MeasurableSpace β] :
    braidingKernel β α ∘ₖ braidingKernel α β = Kernel.id := by
  simp only [braidingKernel, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl
```

Copy and Discard

```
/-- **Copy then first projection = identity.**

`Prod.fst ∘ Δ = id`: mapping the first component after copying
gives back the identity kernel. -/
theorem copy_map_fst (α : Type u) [MeasurableSpace α] :
    Kernel.map (Kernel.copy α) Prod.fst = Kernel.id := by
  ext x s hs
  rw [Kernel.map_apply' _ measurable_fst x hs, Kernel.copy_apply, Kernel.id_apply]
  simp only [Measure.dirac_apply' _ (measurable_fst hs), Measure.dirac_apply' _ hs]
  rfl

/-- **Copy then second projection = identity.** -/
theorem copy_map_snd (α : Type u) [MeasurableSpace α] :
    Kernel.map (Kernel.copy α) Prod.snd = Kernel.id := by
  ext x s hs
  rw [Kernel.map_apply' _ measurable_snd x hs, Kernel.copy_apply, Kernel.id_apply]
  simp only [Measure.dirac_apply' _ (measurable_snd hs), Measure.dirac_apply' _ hs]
  rfl

/-- **Copy is cocommutative** (Fritz §2, Axiom (M2)).

`swap ∘ Δ = Δ` since `swap(x, x) = (x, x)`. -/
theorem copy_swap (α : Type u) [MeasurableSpace α] :
    Kernel.swapRight (Kernel.copy α) = Kernel.copy α := by
  ext x s hs
  simp only [Kernel.swapRight, Kernel.mapOfMeasurable, Kernel.coe_mk,
    Kernel.copy, Kernel.deterministic_apply]
  rw [Measure.map_dirac' measurable_swap]
  rfl

/-- **Discard is natural for Markov kernels** (Fritz §2, Axiom (M4)).

`discard ∘∘ κ = discard` for any Markov kernel κ. This is the
defining axiom of Markov categories, using Mathlib's `Kernel.comp_discard`. -/
theorem comp_discard_of_markov {α β : Type u}
    [MeasurableSpace α] [MeasurableSpace β]
    (κ : Kernel α β) [IsMarkovKernel κ] :
    Kernel.discard β ∘∘ κ = Kernel.discard α :=
  Kernel.comp_discard κ

/-- **Parallel composition of identities = identity.**

`id ⊗⊗ id = id` on the product space. -/
```

```
theorem parallelComp_id_id (α β : Type u)
    [MeasurableSpace α] [MeasurableSpace β] :
    (Kernel.id : Kernel α α) ∥ (Kernel.id : Kernel β β) = Kernel.id := by
  ext ⟨x, y⟩ s hs
  simp only [Kernel.parallelComp_apply, Kernel.id_apply, Measure.dirac_prod_dirac]
```

Discussion

The identities above establish the key mathematical content of Fritz's Markov category axioms for Stoch:

1. Counit laws: `copy_map_fst`, `copy_map_snd` (copy then discard a component recovers the identity).
2. Cocommutativity: `copy_swap` (the diagonal is swap-invariant).
3. Discard naturality: `comp_discard_of_markov` (Markov kernels preserve total mass).
4. Tensor functoriality: `parallelComp_id_id` (identity is preserved).
5. Involutive braiding: `braiding_comp_braiding`.

The full `MarkovCategory MeasObj` typeclass instance additionally requires:

- **`MonoidalCategory MeasObj`**: ~10 coherence conditions (pentagon, triangle, naturality of associator and unitors). Each is a kernel equality, proved via `Kernel.ext` and measure computation.
- Restriction to Markov kernels: `discard_natural` requires all morphisms to be Markov. This needs a new category with `Hom X Y = { κ : Kernel X Y // IsMarkovKernel κ }`.

These are engineering challenges orthogonal to the mathematical content.

```
end CpmProofs
```

```
import CpmProofs.BinExample
import Mathlib.MeasureTheory.Integral.IntervalIntegral.TrapezoidalRule
```

Source: `TrapezoidalRule.lean`

Part 9: Higher-Order Quadrature — Trapezoidal Rule

Part 5 proved the midpoint quadrature error bound $O(h^2)$ using the iterated FTC technique. This file connects the Kan extension framework to the trapezoidal rule, an alternative quadrature method with the same $O(h^2)$ convergence rate but different error characteristics.

Background

For a C^2 function `f` on `[a, b]` with $|f''| \le M_2$:

- Midpoint rule (Part 5): `|binAverage(f) - f(mid)| ≤ M₂ h²/24`
- Trapezoidal rule: `|T(f) - ∫f| ≤ M₂ h³/12` (single trapezoid) or `|T_N(f) - ∫f| ≤ M₂ (b-a)³/(12N²)` (N trapezoids)

where `T(f) = h/2 · (f(a) + f(b))` and `h = b - a`.

Mathlib's trapezoidal rule formalization (Kielstra, 2025) provides: - `trapezoidal_integral f N a b` — the trapezoidal approximation - `trapezoidal_error_le` — the `O(h²)` error bound

Connection to the Kan Extension

The trapezoidal rule provides an alternative way to compute the Kan extension value numerically. While the midpoint rule uses the function value at the bin center, the trapezoidal rule uses the values at bin endpoints. Both converge at `O(h²)` but:

- Midpoint has coefficient `M₂/24` (smaller constant)
- Trapezoidal has coefficient `M₂/12` (larger constant)
- Trapezoidal is often preferred in practice due to endpoint reuse

Results

#	Result	Status	
31	Trapezoidal rule approximates bin average	✓
32	Trapezoidal Kan extension error bound	✓
33	Midpoint vs trapezoidal comparison	✓

```
open MeasureTheory
open ProbabilityTheory

namespace CpmProofs
```

Trapezoidal Approximation of the Bin Average

The trapezoidal rule for a single bin `[a, b]` gives:

$$T_1(f) = \frac{b-a}{2}\,(f(a) + f(b))$$

Dividing by `(b-a)`, the trapezoidal bin average approximation is:

$$\frac{T_1(f)}{b-a} = \frac{f(a) + f(b)}{2}$$

The trapezoidal error (for the integral, not the average) is:

$$|T_1(f) - \int_a^b f| \le \frac{M_2 \cdot (b-a)^3}{12}$$

```
/-- **Trapezoidal approximation of the bin average.**

The trapezoidal rule for a single bin gives the average of the endpoint
values: `(f(a) + f(b)) / 2`. -/
noncomputable def trapezoidalAverage (f : ℝ → ℝ) (I : IntervalBin) : ℝ :=
  (f I.a + f I.b) / 2

/-- **Trapezoidal rule approximates the bin average** (Kan extension value).

For a `C²` function `f` with `|f''| ≤ M₂`, the trapezoidal average
approximates the bin average (= Kan extension value) with error
`O(h²)`:

$$\left|\text{binAverage}(f, I) - \frac{f(a) + f(b)}{2}\right|
  \le \frac{M_2 \cdot (b-a)^2}{12}$$

The coefficient `M₂/12` is twice the midpoint coefficient `M₂/24`,
reflecting the well-known fact that the midpoint rule has half the
error constant of the trapezoidal rule for smooth functions.

The proof delegates to Mathlib's `trapezoidal_error_le_of_c2` for the
integral error, then divides by `(b-a)` to get the bin-average error. -/
theorem trapezoidal_approximates_binAverage (I : IntervalBin) {f : ℝ → ℝ} {M₂ : ℝ}
    (hf : ContDiffOn ℝ 2 f (Set.uIcc I.a I.b))
    (hM : ∀ x, |iteratedDerivWithin 2 f (Set.uIcc I.a I.b) x| ≤ M₂) :
    |binAverage f I - trapezoidalAverage f I| ≤ M₂ * (I.b - I.a) ^ 2 / 12 := by
  -- binAverage = (1/(b-a)) * ∫ f
  -- trapezoidalAverage = (f(a) + f(b))/2
  -- trapezoidal_integral f 1 a b = (b-a)/2 * (f(a) + f(b)) by trapezoidal_integral_one
  -- So binAverage - trapAverage = (1/(b-a)) * (∫ f - trapezoidal_integral f 1 a b)
  have hba : (0 : ℝ) < I.b - I.a := sub_pos.mpr I.hab
  -- Mathlib's trapezoidal error bound (for the integral, not the average)
  have h_trap := trapezoidal_error_le_of_c2 hf hM (N_nonzero := Nat.one_pos)
  -- trapezoidal_error f 1 a b = trapezoidal_integral f 1 a b - ∫ f
  -- |trapezoidal_integral f 1 a b - ∫ f| ≤ |b-a|³ · M₂ / 12
  simp only [trapezoidal_error, Nat.cast_one] at h_trap
  -- |trapezoidal_integral f 1 a b - ∫ x in a..b, f x| ≤ |b-a|³ · M₂ / 12
  have h_abs_ba : |I.b - I.a| = I.b - I.a := abs_of_pos hba
  rw [h_abs_ba] at h_trap
  -- Now relate to bin average and trapezoidal average
```

```
  have h_trap_one := trapezoidal_integral_one f I.a I.b
  -- trapezoidal_integral f 1 a b = (b-a)/2 * (f(a) + f(b))
  -- binAverage f I = (1/(b-a)) * ∫ x in a..b, f x
  -- binAverage - trapAverage = (1/(b-a)) * (∫ f - (b-a) * trapAverage)
  --                          = (1/(b-a)) * (∫ f - trapezoidal_integral f 1 a b)
  --                          = -(1/(b-a)) * trapezoidal_error f 1 a b
  have h_eq : binAverage f I - trapezoidalAverage f I =
      -(1 / (I.b - I.a)) * trapezoidal_error f 1 I.a I.b := by
    simp only [binAverage, trapezoidalAverage, trapezoidal_error, h_trap_one]
    field_simp
    ring
  rw [h_eq, neg_mul, abs_neg, abs_mul,
    abs_of_pos (by positivity : (0 : ℝ) < 1 / (I.b - I.a))]
  have h_trap' : |trapezoidal_error f 1 I.a I.b| ≤ (I.b - I.a) ^ 3 * M₂ / 12 := by
    have := h_trap; simp only [one_pow, mul_one] at this; exact this
  calc 1 / (I.b - I.a) * |trapezoidal_error f 1 I.a I.b|
      ≤ 1 / (I.b - I.a) * ((I.b - I.a) ^ 3 * M₂ / 12) :=
        mul_le_mul_of_nonneg_left h_trap' (by positivity)
    _ = M₂ * (I.b - I.a) ^ 2 / 12 := by field_simp

/-- **Trapezoidal rule approximates the Kan extension value.**

Combined with `lanValue_eq_binAverage`, this gives:

$$\left|(\text{Lan}_D\, f)(\star) - \frac{f(a) + f(b)}{2}\right|
  \le \frac{M_2 \cdot h^2}{12}$$ -/
theorem trapezoidal_approximates_lanValue
    {f : ℝ → ℝ} (I : IntervalBin) {M₂ : ℝ}
    (hf : ContDiffOn ℝ 2 f (Set.uIcc I.a I.b))
    (hM : ∀ x, |iteratedDerivWithin 2 f (Set.uIcc I.a I.b) x| ≤ M₂)
    (κ : Kernel Unit ℝ)
    (hκ : ∀ _u : Unit, (∫ x, f x ∂(κ ())) = binAverage f I) :
    |lanValue (X := ℝObj) (Y := unitObj)
      (fun _x : ℝ ⇒ ()) measurable_const κ f () - trapezoidalAverage f I|
      ≤ M₂ * (I.b - I.a) ^ 2 / 12 := by
  rw [lanValue_eq_binAverage f I κ hκ]
  exact trapezoidal_approximates_binAverage I hf hM
```

Comparison: Midpoint vs Trapezoidal

The midpoint rule (Part 5) has error coefficient $M_2/24$ while the trapezoidal rule has $M_2/12$. We state this comparison formally.

```
/-- **Midpoint rule has half the error constant of the trapezoidal rule.**
```

```
For any `C²` function on `[a, b]`:
- Midpoint error ≤ `M₂ h² / 24`
- Trapezoidal error ≤ `M₂ h² / 12`

So `midpoint error ≤ trapezoidal error / 2`. This explains why the
midpoint rule often outperforms the trapezoidal rule in practice. -/
theorem midpoint_error_le_half_trapezoidal_error (I : IntervalBin) {M₂ : ℝ}
    (hM₂ : 0 ≤ M₂) :
    M₂ * (I.b - I.a) ^ 2 / 24 ≤ M₂ * (I.b - I.a) ^ 2 / 12 := by
  have hba : (0 : ℝ) ≤ (I.b - I.a) ^ 2 := sq_nonneg _
  have : M₂ * (I.b - I.a) ^ 2 / 24 = M₂ * (I.b - I.a) ^ 2 / 12 / 2 := by ring
  linarith [mul_nonneg hM₂ hba]

end CpmProofs
```

```
import CpmProofs.MultiBin
```

Source: `NonUniformPartition.lean`

Part 11: Non-Uniform Partitions

Parts 6–7 established composite error bounds and multi-bin Kan extensions for uniform partitions (all bins have width `h = L/N`). In practice, IPMs sometimes use adaptive meshes with variable-width bins concentrated near regions of high curvature.

This file generalizes the formalization to non-uniform partitions:

1. A `Partition` structure with adjacency constraints on `Fin N → IntervalBin`.
2. Per-bin widths, maximum width `hMax`, and a telescoping sum proof.
3. A non-uniform kernel `nuKernel` that reduces to `binKernel` per bin.
4. Composite error bounds using `hMax` instead of uniform `h`.

Key Insight

`midpoint_quadrature_error` already works for arbitrary-width intervals: `|binAverage f I - f I.midpoint| ≤ M₂ * (I.b - I.a)² / 24`. The per-bin error depends on `(b-a)²`, not on any global `h`. For non-uniform partitions, the composite integral error is bounded by `M₂ * hMax² * L / 24` since each `h_k² ≤ hMax²`.

Results

#	Result	Status
35	Partition structure with adjacency	✓
36	Bin widths positive, bounded by hMax	✓
37	Telescoping sum of widths = L	✓
38	Non-uniform kernel construction	✓
39	Composite error for non-uniform partitions	✓
40	Uniform partition embeds into Partition	✓

```
open MeasureTheory
open CategoryTheory
open ProbabilityTheory
open Finset
open Set

namespace CpmProofs
```

The Partition Structure

A `Partition` divides `[a, a + L]` into `N` adjacent subintervals of possibly different widths. Each bin is an `IntervalBin` (carrying its own `hab : a < b` proof), and adjacency constraints ensure the bins tile the interval without gaps or overlaps.

```
/-- A (non-uniform) partition of `[a, a + L]` into `N` adjacent bins.

The adjacency condition ensures consecutive bins share endpoints:
`(bins i).b = (bins j).a` whenever `j = i + 1`. Combined with
`first_left` and `last_right`, the bins tile `[a, a + L]` exactly. -/
structure Partition where
  N : ℕ
  hN : 0 < N
  a : ℝ
  L : ℝ
  hL : 0 < L
  bins : Fin N → IntervalBin
  adjacency : ∀ (i j : Fin N), i.val + 1 = j.val → (bins i).b = (bins j).a
  first_left : (bins ⟨0, hN⟩).a = a
  last_right : (bins ⟨N - 1, by omega⟩).b = a + L

namespace Partition
```

Bin Widths

```
/-- Width of the `k`-th bin: `(bins k).b - (bins k).a`. -/
noncomputable def width (P : Partition) (k : Fin P.N) : ℝ :=
```

```
  (P.bins k).b - (P.bins k).a

/-- Each bin has positive width (from `IntervalBin.hab`). -/
theorem width_pos (P : Partition) (k : Fin P.N) : 0 < P.width k :=
  sub_pos.mpr (P.bins k).hab
```

Maximum Bin Width

```
/-- Maximum bin width across all bins, computed via `Finset.sup'`. -/
noncomputable def hMax (P : Partition) : ℝ :=
  Finset.sup' Finset.univ ⟨⟨0, P.hN⟩, Finset.mem_univ _⟩ (fun k ⇒ P.width k)

/-- The maximum bin width is positive. -/
theorem hMax_pos (P : Partition) : 0 < P.hMax := by
  unfold hMax
  have h := P.width_pos ⟨0, P.hN⟩
  calc 0 < P.width ⟨0, P.hN⟩ := h
    _ ≤ Finset.sup' Finset.univ ⟨⟨0, P.hN⟩, Finset.mem_univ _⟩ (fun k ⇒ P.width k) :=
        Finset.le_sup' _ (Finset.mem_univ _)

/-- Each bin width is at most `hMax`. -/
theorem width_le_hMax (P : Partition) (k : Fin P.N) : P.width k ≤ P.hMax :=
  Finset.le_sup' _ (Finset.mem_univ k)
```

Telescoping Sum of Widths

The sum of all bin widths equals `L`. This is the most technically challenging proof: we define a `node` function and use the adjacency constraints to show that each width is a consecutive difference, then apply `Finset.sum_range_sub'` (or equivalent) to telescope.

```
/-- The left endpoint of the `k`-th bin, as a function on `Fin N`.
    Used internally for the telescoping sum proof. -/
private noncomputable def leftNode (P : Partition) (k : Fin P.N) : ℝ :=
  (P.bins k).a

/-- Adjacency implies the left endpoint of bin `j` equals the right
    endpoint of bin `i` when `j = i + 1`. -/
private theorem left_eq_right_succ (P : Partition) (i : Fin P.N) (j : Fin P.N)
    (hij : i.val + 1 = j.val) : (P.bins i).b = (P.bins j).a :=
  P.adjacency i j hij

/-- **Telescoping sum**: the sum of all bin widths equals `L`.
```

```
The proof constructs a `node` function `k ↦ (bins k).a` for `k < N`,
extended to `node N = a + L`. Each `width k = node (k+1) - node k`
by adjacency, and the sum telescopes to `node N - node 0 = (a+L) - a = L`. -/
theorem sum_widths (P : Partition) :
    ∑ k : Fin P.N, P.width k = P.L := by
  -- Define node : ℕ → ℝ where node k = (bins k).a for k < N, node N = a + L
  set node : ℕ → ℝ := fun k ⇒
    if h : k < P.N then (P.bins ⟨k, h⟩).a else P.a + P.L
  -- Show width k = node (k+1) - node k
  have h_width : ∀ k : Fin P.N, P.width k = node (k.val + 1) - node k.val := by
    intro ⟨k, hk⟩
    simp only [width, node]
    -- node k = (bins k).a since k < N
    rw [dif_pos hk]
    -- For node (k+1): case split on k+1 < N
    by_cases hk1 : k + 1 < P.N
    · -- k+1 < N: node (k+1) = (bins (k+1)).a = (bins k).b by adjacency
      rw [dif_pos hk1]
      have := P.adjacency ⟨k, hk⟩ ⟨k + 1, hk1⟩ rfl
      linarith
    · -- k+1 = N (since k < N): node (k+1) = a + L = (bins (N-1)).b
      rw [dif_neg hk1]
      have hkN : k = P.N - 1 := by omega
      subst hkN
      linarith [P.last_right]
  -- Rewrite the sum using the telescope form
  have h_eq : ∑ k : Fin P.N, P.width k = ∑ k : Fin P.N, (node (↑k + 1) - node ↑k) :=
    Finset.sum_congr rfl (fun k _ ⇒ h_width k)
  rw [h_eq]
  -- Convert Fin sum to range sum and telescope
  rw [Fin.sum_univ_eq_sum_range (fun k ⇒ node (k + 1) - node k)]
  rw [Finset.sum_range_sub (fun i ⇒ node i)]
  -- node N = a + L (since N is not < N)
  simp only [node, dif_neg (lt_irrefl P.N), dif_pos P.hN]
  -- node 0 = (bins 0).a = a
  rw [P.first_left]
  ring
```

Non-Uniform Kernel

The non-uniform kernel assigns to each bin the normalized Lebesgue measure on that bin's interval. This is exactly `binKernel (P.bins k)` at each index `k`.

```
/-- **Non-uniform multi-bin kernel.**
```

```
For each bin index `k : Fin N`, yields the normalized Lebesgue measure
on `[(bins k).a, (bins k).b]`:
$$\kappa_k = \frac{1}{b_k - a_k}\, \lambda\big|_{[a_k, b_k]}$$

Unlike `multiBinKernel` (which uses uniform width `1/h`), each bin uses
its own width for normalization. -/
noncomputable def nuKernel (P : Partition) : Kernel (Fin P.N) ℝ where
  toFun k := ENNReal.ofReal (1 / ((P.bins k).b - (P.bins k).a)) •
    volume.restrict (Icc (P.bins k).a (P.bins k).b)
  measurable' := measurable_of_finite _

/-- The non-uniform kernel at index `k` equals the single-bin `binKernel`. -/
theorem nuKernel_apply (P : Partition) (k : Fin P.N) :
    P.nuKernel k = binKernel (P.bins k) () := by
  show P.nuKernel.toFun k = _
  simp only [nuKernel, binKernel, Kernel.const_apply]

/-- Integration against the non-uniform kernel equals the bin average. -/
theorem nuKernel_integral (P : Partition) (f : ℝ → ℝ) (k : Fin P.N) :
    (∫ x, f x ∂(P.nuKernel k)) = binAverage f (P.bins k) := by
  rw [nuKernel_apply]
  exact binKernel_integral f (P.bins k)

/-- The Kan extension value at bin `k` equals the bin average. -/
theorem nu_lanValue_eq_binAverage (P : Partition) (f : ℝ → ℝ) (k : Fin P.N) :
    lanValue (X := ⟨ℝ, inferInstance⟩) (Y := finObj P.N)
      (fun _ ⇒ k) (measurable_const) P.nuKernel f k =
      binAverage f (P.bins k) := by
  simp only [lanValue]
  exact nuKernel_integral P f k
```

Error Bounds for Non-Uniform Partitions

The per-bin midpoint error bound is immediate from `midpoint_quadrature_error` since it depends only on `(b - a)²` for the individual bin. The composite bound uses `hMax²` in place of `h²`.

```
/-- **Per-bin midpoint error for non-uniform partitions.**

Each bin's midpoint approximation error is bounded by
`M₂ * width_k² / 24`, directly from `midpoint_quadrature_error`. -/
theorem nu_midpoint_error (P : Partition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 2 f (Icc (P.bins k).a (P.bins k).b))
```

```
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Icc (P.bins k).a (P.bins k).b,
      |iteratedDerivWithin 2 f (Icc (P.bins k).a (P.bins k).b) x| ≤ M₂)
    (k : Fin P.N) :
    |binAverage f (P.bins k) - f (P.bins k).midpoint| ≤
      M₂ * (P.width k) ^ 2 / 24 := by
  have h_err := midpoint_quadrature_error (P.bins k) (hf_bin k) (hM_bin k)
  exact h_err

/-- **Composite integral error for non-uniform partitions.**

The weighted sum `∑ width_k * (binAverage - f(midpoint))` (which
approximates the total integral error) satisfies:

$$\left|\sum_{k=0}^{N-1} h_k \cdot (\text{binAverage} - f(m_k))\right|
  \le \frac{M_2 \cdot h_{\max}^2 \cdot L}{24}$$

**Proof chain:**
```
|∑ h_k · e_k| ≤ ∑ h_k · |e_k| (triangle ineq)
              ≤ ∑ h_k · (M₂ · h_k² / 24)   (per-bin bound)
              = (M₂/24) · ∑ h_k³
              ≤ (M₂/24) · ∑ h_k · hMax²   (h_k² ≤ hMax²)
              = (M₂/24) · hMax² · ∑ h_k
              = (M₂/24) · hMax² · L   (sum_widths)
              = M₂ · hMax² · L / 24
``` -/
theorem nu_composite_integral_error (P : Partition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (hM₂ : 0 ≤ M₂)
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 2 f (Icc (P.bins k).a (P.bins k).b))
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Icc (P.bins k).a (P.bins k).b,
      |iteratedDerivWithin 2 f (Icc (P.bins k).a (P.bins k).b) x| ≤ M₂) :
    |∑ k : Fin P.N,
      (P.width k * (binAverage f (P.bins k) - f (P.bins k).midpoint))| ≤
      M₂ * P.hMax ^ 2 * P.L / 24 := by
  -- Step 1: Triangle inequality
  calc |∑ k : Fin P.N,
        (P.width k * (binAverage f (P.bins k) - f (P.bins k).midpoint))|
      ≤ ∑ k : Fin P.N,
        |P.width k * (binAverage f (P.bins k) - f (P.bins k).midpoint)| :=
          Finset.abs_sum_le_sum_abs _ _
    _ = ∑ k : Fin P.N,
        (P.width k * |binAverage f (P.bins k) - f (P.bins k).midpoint|) := by
          congr 1; ext k
```

```
          rw [abs_mul, abs_of_pos (P.width_pos k)]
    -- Step 2: Apply per-bin bound
    _ ≤ ∑ k : Fin P.N, (P.width k * (M₂ * (P.width k) ^ 2 / 24)) := by
          apply Finset.sum_le_sum
          intro k _
          exact mul_le_mul_of_nonneg_left (nu_midpoint_error P hf_bin hM_bin k)
            (le_of_lt (P.width_pos k))
    -- Step 3: h_k² ≤ hMax²
    _ ≤ ∑ k : Fin P.N, (P.width k * (M₂ * P.hMax ^ 2 / 24)) := by
          apply Finset.sum_le_sum
          intro k _
          apply mul_le_mul_of_nonneg_left _ (le_of_lt (P.width_pos k))
          apply div_le_div_of_nonneg_right _ (by norm_num : (0 : ℝ) < 24).le
          apply mul_le_mul_of_nonneg_left _ hM₂
          -- width k ^ 2 ≤ hMax ^ 2 from 0 ≤ width k ≤ hMax
          have hw := P.width_le_hMax k
          have hwp := le_of_lt (P.width_pos k)
          calc P.width k ^ 2 = P.width k * P.width k := sq (P.width k)
            _ ≤ P.hMax * P.hMax := mul_le_mul hw hw hwp (le_trans hwp hw)
            _ = P.hMax ^ 2 := (sq P.hMax).symm
    -- Step 4: Factor out constant and apply sum_widths
    _ = M₂ * P.hMax ^ 2 / 24 * ∑ k : Fin P.N, P.width k := by
          simp_rw [show ∀ k : Fin P.N, P.width k * (M₂ * P.hMax ^ 2 / 24) =
              (M₂ * P.hMax ^ 2 / 24) * P.width k from fun k ⇒ by ring]
          rw [← Finset.mul_sum]
    _ = M₂ * P.hMax ^ 2 / 24 * P.L := by
          rw [P.sum_widths]
    _ = M₂ * P.hMax ^ 2 * P.L / 24 := by ring

end Partition
```

Uniform Partitions as Special Cases

A `UniformPartition` can be embedded into `Partition`, and its `hMax` equals the uniform width `h`.

```
namespace UniformPartition

/-- Embed a `UniformPartition` into the general `Partition` type. -/
noncomputable def toPartition (P : UniformPartition) : Partition where
  N := P.N
  hN := P.hN
  a := P.a
  L := P.L
  hL := P.hL
```

```
  bins := fun k ⇒ P.bin k
  adjacency := by
    intro i j hij
    simp only [bin, right, left]
    have : (↑i.val : ℝ) + 1 = (↑j.val : ℝ) := by exact_mod_cast hij
    rw [this]
  first_left := by
    simp only [bin, left]
    ring
  last_right := by
    simp only [bin, right]
    have hN' : (0 : ℝ) < ↑P.N := Nat.cast_pos.mpr P.hN
    have : (↑(P.N - 1) + 1 : ℝ) = ↑P.N := by
      rw [Nat.cast_sub (Nat.one_le_iff_ne_zero.mpr (Nat.pos_iff_ne_zero.mp P.hN))]
      simp
    rw [this]
    simp only [h]
    field_simp

/-- For a uniform partition, `hMax = h`: all bins have the same width. -/
theorem toPartition_hMax_eq_h (P : UniformPartition) :
    P.toPartition.hMax = P.h := by
  unfold Partition.hMax
  apply le_antisymm
  · -- hMax ≤ h: every bin has width h
    apply Finset.sup'_le
    intro k _
    show P.toPartition.width k ≤ P.h
    simp only [Partition.width, toPartition, bin, right, left]
    ring_nf; exact le_refl _
  · -- h ≤ hMax: h is one of the bin widths
    calc P.h = P.toPartition.width ⟨0, P.hN⟩ := by
            simp only [Partition.width, toPartition, bin, right, left]
            ring
      _ ≤ _ := Finset.le_sup' _ (Finset.mem_univ _)

end UniformPartition

end CpmProofs
```

```
import CpmProofs.NonUniformPartition
```

Source: `ProductBin.lean`

Part 12: 2D Product Bins

Parts 5–11 treat one-dimensional state spaces ($\mathbb{R}$). IPMs for organisms with multiple traits (e.g., body size × age, or spatial coordinates) require multi-dimensional state spaces ($\mathbb{R}^2$). The abstract bridge theorem (`KanBridge.lean`, `CondKernel.lean`, `Integration.lean`) is already fully polymorphic in `{X Y : MeasObj}` — it works for any measurable space with zero modifications. Only the concrete layer (interval bins, bin averages, midpoint error) needs generalization.

This file generalizes the formalization to 2D rectangular bins:

1. A `RectBin` structure as a product of two `IntervalBin`s.
2. A `RectPartition` as a tensor product of two 1D `Partition`s.
3. The rectangle average `rectAverage` via iterated bin averages.
4. Per-cell 2D midpoint error via Fubini decomposition.
5. Composite error over the full tensor product grid.

Key Insight: Fubini Decomposition

For a rectangle $[a_1,b_1] \times [a_2,b_2]$, the midpoint quadrature error decomposes via Fubini:

$$\text{rectAvg } f - f(m_1,m_2) = (\text{binAvg } g\ I_1 - g(m_1)) + (g(m_1) - f(m_1,m_2))$$

where $g(x) = \text{binAvg}(y \mapsto f(x,y), I_2)$. Each term is a 1D midpoint error, so:

$$|\text{rectAvg } f - f(m_1,m_2)| \le M_{xx} \cdot h_1^2/24 + M_{yy} \cdot h_2^2/24$$

This reuses `midpoint_quadrature_error` from BinExample.lean twice, avoiding any new analytical heavy lifting.

Results

#	Result	Status
41	RectBin and RectPartition structures	✓
42	Rectangle midpoint, area, and area positivity	✓
43	Rectangle average via iterated bin averages	✓
44	Fubini decomposition (definitional)	✓
45	Per-cell 2D midpoint error bound	✓
46	Composite 2D error over tensor product grid	✓
47	Rectangle kernel construction	✓
48	Rectangle kernel relates to product of 1D kernels	✓
49	2D Kan extension connection	✓
50	$\mathbb{R} \times \mathbb{R}$ as a measurable object	✓

```
open MeasureTheory
open CategoryTheory
```

```
open ProbabilityTheory
open Finset
open Set

namespace CpmProofs
```

Rectangular Bins

A `RectBin` is a product of two `IntervalBin`s, representing a rectangular cell $[a_1, b_1] \times [a_2, b_2]$ in the 2D trait space.

```
/-- A rectangular bin as a product of two interval bins.

In the context of 2D IPMs, this represents a cell in the discretised
trait space — e.g., a size bin times an age bin. -/
structure RectBin where
  I₁ : IntervalBin
  I₂ : IntervalBin

/-- A tensor product of two 1D partitions.

Partitions the rectangle `[a₁, a₁+L₁] × [a₂, a₂+L₂]` into
`N₁ × N₂` rectangular cells. -/
structure RectPartition where
  P₁ : Partition
  P₂ : Partition

namespace RectBin

/-- **Midpoint of a rectangular bin**: the pair of 1D midpoints.

$$m = \left(\frac{a_1+b_1}{2}, \frac{a_2+b_2}{2}\right)$$ -/
noncomputable def midpoint (R : RectBin) : ℝ × ℝ :=
  (R.I₁.midpoint, R.I₂.midpoint)

/-- **Area of a rectangular bin**: product of side lengths.

$$\text{area} = (b_1 - a_1) \cdot (b_2 - a_2)$$ -/
noncomputable def area (R : RectBin) : ℝ :=
  (R.I₁.b - R.I₁.a) * (R.I₂.b - R.I₂.a)

/-- A rectangular bin has positive area. -/
theorem area_pos (R : RectBin) : 0 < R.area :=
  mul_pos (sub_pos.mpr R.I₁.hab) (sub_pos.mpr R.I₂.hab)
```

```
end RectBin
```

Rectangle Average

The rectangle average of a function `f : ℝ × ℝ → ℝ` over a rectangular bin is defined as the iterated bin average: first average over the second coordinate, then average over the first. By Fubini's theorem, this equals `(1/area) * ∫∫ f dA` over the rectangle.

```
/-- **Rectangle average** of a function `f` over a rectangular bin.

$$\text{rectAverage}(f, R)
  = \frac{1}{h_1}\int_{a_1}^{b_1}
    \left(\frac{1}{h_2}\int_{a_2}^{b_2} f(x,y)\, \mathrm{d}y\right)
    \mathrm{d}x$$

Equivalently (by Fubini):
$$= \frac{1}{h_1 \cdot h_2} \iint_{[a_1,b_1] \times [a_2,b_2]} f(x,y)\, \mathrm{d}A$$

The definition uses the iterated form, which makes the Fubini
decomposition definitional and simplifies error bound proofs. -/
noncomputable def rectAverage (f : ℝ × ℝ → ℝ) (R : RectBin) : ℝ :=
  binAverage (fun x ⇒ binAverage (fun y ⇒ f (x, y)) R.I₂) R.I₁

/-- **Fubini decomposition**: the rectangle average equals
iterated bin averages (definitional). -/
theorem rectAverage_eq_iterated (f : ℝ × ℝ → ℝ) (R : RectBin) :
    rectAverage f R =
      binAverage (fun x ⇒ binAverage (fun y ⇒ f (x, y)) R.I₂) R.I₁ :=
  rfl

/-- **Integration order**: the rectangle average can also be computed
by averaging over the first coordinate first, then the second.

This is stated with a hypothesis that the two orderings agree,
which follows from Fubini's theorem for continuous functions on
compact rectangles. -/
theorem rectAverage_comm (f : ℝ × ℝ → ℝ) (R : RectBin)
    (h : binAverage (fun x ⇒ binAverage (fun y ⇒ f (x, y)) R.I₂) R.I₁ =
         binAverage (fun y ⇒ binAverage (fun x ⇒ f (x, y)) R.I₁) R.I₂) :
    rectAverage f R =
      binAverage (fun y ⇒ binAverage (fun x ⇒ f (x, y)) R.I₁) R.I₂ := h
```

Tensor Product Partition

A `RectPartition` provides a grid of rectangular bins formed by the Cartesian product of two 1D partitions.

```
namespace RectPartition

/-- Extract the `(i,j)`-th rectangular bin from a tensor product partition. -/
def bin (R : RectPartition) (i : Fin R.P₁.N) (j : Fin R.P₂.N) : RectBin where
  I₁ := R.P₁.bins i
  I₂ := R.P₂.bins j

end RectPartition
```

2D Midpoint Error Bound

The central result: the 2D midpoint quadrature error decomposes as a sum of two 1D errors via the Fubini decomposition.

```
/-- **Per-cell 2D midpoint error bound.**

For a function `f : ℝ × ℝ → ℝ` on a rectangular bin `R = I₁ × I₂`:

$$\left|\text{rectAverage}(f, R) - f(m_1, m_2)\right|
  \le \frac{M_{xx} \cdot h_1^2}{24} + \frac{M_{yy} \cdot h_2^2}{24}$$

**Hypotheses** (hypothesis-based approach, avoiding differentiation
under the integral sign):
- `g(x) := binAverage(y ↦ f(x,y), I₂)` is C² on `I₁` with
  `|g''| ≤ M_xx`
- `f(m₁, ·)` is C² on `I₂` with `|∂²f/∂y²(m₁, ·)| ≤ M_yy`

**Proof**: Triangle inequality applied to the Fubini decomposition:
```
|rectAvg f - f(m₁,m₂)|
  = |binAvg g I₁ - f(m₁,m₂)|
  ≤ |binAvg g I₁ - g(m₁)| + |g(m₁) - f(m₁,m₂)|
  ≤ M_xx·h₁²/24 + M_yy·h₂²/24
```
Each term is bounded by `midpoint_quadrature_error` from Part 5. -/
theorem rect_midpoint_error (R : RectBin)
    {f : ℝ × ℝ → ℝ} {M_xx M_yy : ℝ}
    -- g(x) := binAverage(y ↦ f(x,y), I₂) is C² on I₁
    (hg : ContDiffOn ℝ 2
      (fun x ⇒ binAverage (fun y ⇒ f (x, y)) R.I₂) (Icc R.I₁.a R.I₁.b))
    (hg_bound : ∀ x ∈ Icc R.I₁.a R.I₁.b,
      |iteratedDerivWithin 2
```

```
      (fun x ⇒ binAverage (fun y ⇒ f (x, y)) R.I₂)
      (Icc R.I₁.a R.I₁.b) x| ≤ M_xx)
  -- f(m₁, ·) is C² on I₂
  (hfy : ContDiffOn ℝ 2
    (fun y ⇒ f (R.I₁.midpoint, y)) (Icc R.I₂.a R.I₂.b))
  (hfy_bound : ∀ y ∈ Icc R.I₂.a R.I₂.b,
    |iteratedDerivWithin 2
      (fun y ⇒ f (R.I₁.midpoint, y))
      (Icc R.I₂.a R.I₂.b) y| ≤ M_yy) :
  |rectAverage f R - f R.midpoint| ≤
    M_xx * (R.I₁.b - R.I₁.a) ^ 2 / 24 +
    M_yy * (R.I₂.b - R.I₂.a) ^ 2 / 24 := by
-- Set up: g(x) = binAverage(y ↦ f(x,y), I₂)
set g := fun x ⇒ binAverage (fun y ⇒ f (x, y)) R.I₂
-- Key identity: a - c = (a - b) + (b - c)
have h_split : rectAverage f R - f R.midpoint =
    (binAverage g R.I₁ - g R.I₁.midpoint) +
    (g R.I₁.midpoint - f R.midpoint) := by
  show binAverage g R.I₁ - f R.midpoint =
    (binAverage g R.I₁ - g R.I₁.midpoint) + (g R.I₁.midpoint - f R.midpoint)
  ring
-- Triangle inequality + two applications of midpoint_quadrature_error
rw [h_split]
have h_tri := norm_add_le
  (binAverage g R.I₁ - g R.I₁.midpoint) (g R.I₁.midpoint - f R.midpoint)
rw [Real.norm_eq_abs, Real.norm_eq_abs, Real.norm_eq_abs] at h_tri
calc |(binAverage g R.I₁ - g R.I₁.midpoint) +
     (g R.I₁.midpoint - f R.midpoint)|
    ≤ |binAverage g R.I₁ - g R.I₁.midpoint| +
      |g R.I₁.midpoint - f R.midpoint| := h_tri
  _ ≤ M_xx * (R.I₁.b - R.I₁.a) ^ 2 / 24 +
      M_yy * (R.I₂.b - R.I₂.a) ^ 2 / 24 := by
        apply add_le_add
        · exact midpoint_quadrature_error R.I₁ hg hg_bound
        · -- g(m₁) = binAverage (fun y ⇒ f(m₁, y)) I₂ (definitionally)
          -- f R.midpoint = f(m₁, I₂.midpoint) (by definition of midpoint)
          show |binAverage (fun y ⇒ f (R.I₁.midpoint, y)) R.I₂ -
               f (R.I₁.midpoint, R.I₂.midpoint)| ≤ _
          exact midpoint_quadrature_error R.I₂ hfy hfy_bound
```

Composite 2D Error Bound

The composite error over the full tensor product grid sums the per-cell errors, weighted by cell areas. The bound uses $hMax_1$ and $hMax_2$ (maximum bin widths in each coordinate direction).

```
/-- **Composite 2D error over the tensor product grid.**

$$\left|\sum_{i,j} h_{1,i} \cdot h_{2,j} \cdot
  (\text{rectAvg}_{ij} - f(m_{1,i}, m_{2,j}))\right|
  \le \frac{(M_{xx} \cdot h_{\max,1}^2 + M_{yy} \cdot h_{\max,2}^2)
            \cdot L_1 \cdot L_2}{24}$$

**Proof**: Triangle inequality → per-cell bound → factor double sum
→ `width_k² ≤ hMax²` → `sum_widths = L` in each direction. -/
theorem rect_composite_error (R : RectPartition)
    {f : ℝ × ℝ → ℝ} {M_xx M_yy : ℝ}
    (hM_xx : 0 ≤ M_xx) (hM_yy : 0 ≤ M_yy)
    (hg_bin : ∀ (i : Fin R.P₁.N) (j : Fin R.P₂.N),
      ContDiffOn ℝ 2
        (fun x ⇒ binAverage (fun y ⇒ f (x, y)) (R.P₂.bins j))
        (Icc (R.P₁.bins i).a (R.P₁.bins i).b))
    (hg_bound : ∀ (i : Fin R.P₁.N) (j : Fin R.P₂.N),
      ∀ x ∈ Icc (R.P₁.bins i).a (R.P₁.bins i).b,
        |iteratedDerivWithin 2
          (fun x ⇒ binAverage (fun y ⇒ f (x, y)) (R.P₂.bins j))
          (Icc (R.P₁.bins i).a (R.P₁.bins i).b) x| ≤ M_xx)
    (hfy_bin : ∀ (i : Fin R.P₁.N) (j : Fin R.P₂.N),
      ContDiffOn ℝ 2
        (fun y ⇒ f ((R.P₁.bins i).midpoint, y))
        (Icc (R.P₂.bins j).a (R.P₂.bins j).b))
    (hfy_bound : ∀ (i : Fin R.P₁.N) (j : Fin R.P₂.N),
      ∀ y ∈ Icc (R.P₂.bins j).a (R.P₂.bins j).b,
        |iteratedDerivWithin 2
          (fun y ⇒ f ((R.P₁.bins i).midpoint, y))
          (Icc (R.P₂.bins j).a (R.P₂.bins j).b) y| ≤ M_yy) :
    |∑ i : Fin R.P₁.N, ∑ j : Fin R.P₂.N,
      (R.P₁.width i * R.P₂.width j *
       (rectAverage f (R.bin i j) - f (R.bin i j).midpoint))| ≤
      (M_xx * R.P₁.hMax ^ 2 + M_yy * R.P₂.hMax ^ 2) *
        R.P₁.L * R.P₂.L / 24 := by
  -- Step 1: Triangle inequality on the double sum
  calc |∑ i : Fin R.P₁.N, ∑ j : Fin R.P₂.N,
        (R.P₁.width i * R.P₂.width j *
         (rectAverage f (R.bin i j) - f (R.bin i j).midpoint))|
      ≤ ∑ i : Fin R.P₁.N, ∑ j : Fin R.P₂.N,
        |R.P₁.width i * R.P₂.width j *
         (rectAverage f (R.bin i j) - f (R.bin i j).midpoint)| := by
          apply le_trans (Finset.abs_sum_le_sum_abs _ _)
          apply Finset.sum_le_sum; intro i _
          exact Finset.abs_sum_le_sum_abs _ _
```

```
    -- Step 2: Factor out positive widths from absolute value
    _ = ∑ i : Fin R.P₁.N, ∑ j : Fin R.P₂.N,
        (R.P₁.width i * R.P₂.width j *
         |rectAverage f (R.bin i j) - f (R.bin i j).midpoint|) := by
          congr 1; ext i; congr 1; ext j
          rw [abs_mul, abs_mul,
              abs_of_pos (R.P₁.width_pos i),
              abs_of_pos (R.P₂.width_pos j)]
    -- Step 3: Apply per-cell 2D bound
    _ ≤ ∑ i : Fin R.P₁.N, ∑ j : Fin R.P₂.N,
        (R.P₁.width i * R.P₂.width j *
         (M_xx * (R.P₁.width i) ^ 2 / 24 +
          M_yy * (R.P₂.width j) ^ 2 / 24)) := by
          apply Finset.sum_le_sum; intro i _
          apply Finset.sum_le_sum; intro j _
          apply mul_le_mul_of_nonneg_left
          · exact rect_midpoint_error (R.bin i j)
              (hg_bin i j) (hg_bound i j) (hfy_bin i j) (hfy_bound i j)
          · exact mul_nonneg (le_of_lt (R.P₁.width_pos i))
              (le_of_lt (R.P₂.width_pos j))
    -- Step 4: Use h_k² ≤ hMax² in each direction
    _ ≤ ∑ i : Fin R.P₁.N, ∑ j : Fin R.P₂.N,
        (R.P₁.width i * R.P₂.width j *
         (M_xx * R.P₁.hMax ^ 2 / 24 +
          M_yy * R.P₂.hMax ^ 2 / 24)) := by
          apply Finset.sum_le_sum; intro i _
          apply Finset.sum_le_sum; intro j _
          apply mul_le_mul_of_nonneg_left _ (mul_nonneg
            (le_of_lt (R.P₁.width_pos i)) (le_of_lt (R.P₂.width_pos j)))
          apply add_le_add
          · apply div_le_div_of_nonneg_right _ (by norm_num : (0:ℝ) < 24).le
            apply mul_le_mul_of_nonneg_left _ hM_xx
            have hw := R.P₁.width_le_hMax i
            have hwp := le_of_lt (R.P₁.width_pos i)
            calc R.P₁.width i ^ 2 = R.P₁.width i * R.P₁.width i := sq _
              _ ≤ R.P₁.hMax * R.P₁.hMax := mul_le_mul hw hw hwp (le_trans hwp hw)
              _ = R.P₁.hMax ^ 2 := (sq _).symm
          · apply div_le_div_of_nonneg_right _ (by norm_num : (0:ℝ) < 24).le
            apply mul_le_mul_of_nonneg_left _ hM_yy
            have hw := R.P₂.width_le_hMax j
            have hwp := le_of_lt (R.P₂.width_pos j)
            calc R.P₂.width j ^ 2 = R.P₂.width j * R.P₂.width j := sq _
              _ ≤ R.P₂.hMax * R.P₂.hMax := mul_le_mul hw hw hwp (le_trans hwp hw)
              _ = R.P₂.hMax ^ 2 := (sq _).symm
    -- Step 5: Factor the double sum and apply sum_widths
```

```
    _ = (M_xx * R.P₁.hMax ^ 2 + M_yy * R.P₂.hMax ^ 2) *
        R.P₁.L * R.P₂.L / 24 := by
          -- Factor inner sum: for each i, ∑_j w₁□ * w₂□ * C = w₁□ * C * L₂
          have h_inner : ∀ i : Fin R.P₁.N,
            ∑ j : Fin R.P₂.N,
              (R.P₁.width i * R.P₂.width j *
                (M_xx * R.P₁.hMax ^ 2 / 24 + M_yy * R.P₂.hMax ^ 2 / 24)) =
              R.P₁.width i *
                (M_xx * R.P₁.hMax ^ 2 / 24 + M_yy * R.P₂.hMax ^ 2 / 24) *
                R.P₂.L := by
            intro i
            simp_rw [show ∀ j : Fin R.P₂.N,
              R.P₁.width i * R.P₂.width j *
                (M_xx * R.P₁.hMax ^ 2 / 24 + M_yy * R.P₂.hMax ^ 2 / 24) =
              R.P₁.width i *
                (M_xx * R.P₁.hMax ^ 2 / 24 + M_yy * R.P₂.hMax ^ 2 / 24) *
                R.P₂.width j from fun _ ⇒ by ring]
            rw [← Finset.mul_sum, R.P₂.sum_widths]
          simp_rw [h_inner]
          -- Factor outer sum: ∑_i w₁□ * C * L₂ = C * L₂ * L₁
          simp_rw [show ∀ i : Fin R.P₁.N,
            R.P₁.width i *
              (M_xx * R.P₁.hMax ^ 2 / 24 + M_yy * R.P₂.hMax ^ 2 / 24) *
              R.P₂.L =
            (M_xx * R.P₁.hMax ^ 2 / 24 + M_yy * R.P₂.hMax ^ 2 / 24) *
              R.P₂.L * R.P₁.width i from fun _ ⇒ by ring]
          rw [← Finset.mul_sum, R.P₁.sum_widths]
          ring
```

Rectangle Kernel

The rectangle kernel assigns to each grid cell `(i,j)` the normalized Lebesgue measure on the corresponding rectangle. It is constructed as the product of the two 1D non-uniform kernels.

```
/-- **Rectangle kernel** for a tensor product partition.

For each grid cell `(i,j)`, yields the product of the normalized
Lebesgue measures on the corresponding 1D intervals:
$$\kappa_{i,j} = \kappa_{1,i} \otimes \kappa_{2,j}$$

Since `Fin N₁ × Fin N₂` is finite, measurability is automatic. -/
noncomputable def rectKernel (R : RectPartition) :
    Kernel (Fin R.P₁.N × Fin R.P₂.N) (ℝ × ℝ) where
  toFun := fun ⟨i, j⟩ ⇒ (R.P₁.nuKernel i).prod (R.P₂.nuKernel j)
```

```
  measurable' := measurable_of_finite _

/-- The rectangle kernel at `(i,j)` relates to the product of
1D `binKernel`s. -/
theorem rectKernel_apply (R : RectPartition)
    (i : Fin R.P₁.N) (j : Fin R.P₂.N) :
    rectKernel R (i, j) =
      (binKernel (R.P₁.bins i) ()).prod (binKernel (R.P₂.bins j) ()) := by
  change (R.P₁.nuKernel i).prod (R.P₂.nuKernel j) = _
  rw [R.P₁.nuKernel_apply i, R.P₂.nuKernel_apply j]
```

Kan Extension Connection

The Kan extension value at each grid cell equals the rectangle average. This connects the abstract categorical framework to the concrete 2D bin-average formula.

```
/-- `ℝ × ℝ` as a bundled measurable object for the stochastic category. -/
noncomputable def prodRObj : MeasObj := ⟨ℝ × ℝ, inferInstance⟩

/-- **Kan extension value = rectangle average.**

The Kan extension at grid cell `(i,j)` equals the iterated bin average
over the corresponding rectangle. This extends the 1D
`nu_lanValue_eq_binAverage` to 2D. -/
theorem rect_lanValue_eq_rectAverage (R : RectPartition)
    (f : ℝ × ℝ → ℝ) (i : Fin R.P₁.N) (j : Fin R.P₂.N)
    (hf : ∀ (i' : Fin R.P₁.N) (j' : Fin R.P₂.N),
      (∫ p, f p ∂(rectKernel R (i', j'))) =
        rectAverage f (R.bin i' j')) :
    lanValue (X := prodRObj)
      (Y := ⟨Fin R.P₁.N × Fin R.P₂.N, inferInstance⟩)
      (fun _ ⇒ (i, j)) measurable_const (rectKernel R) f (i, j) =
      rectAverage f (R.bin i j) := by
  simp only [lanValue]
  exact hf i j

end CpmProofs
```

```
import CpmProofs.ProductBin
```

Source: `BoxBin.lean`

Part 13: d-Dimensional Box Bins

Part 12 generalized from 1D intervals to 2D rectangular bins. This file further generalizes to d-dimensional boxes, enabling IPMs with arbitrarily many continuous traits (e.g., body size × reproductive status × spatial coordinates).

Structures

1. `BoxBin d` — a d-dimensional box as `Fin d → IntervalBin`.
2. `BoxPartition d` — tensor product of `d` 1D `Partition`s.

Key Insight: Induction on Dimension

The d-dimensional midpoint error decomposes by peeling off one dimension at a time:

```
|boxAvg f B - f B.midpoint| ≤ ∑ i, M₂ᵢ · hᵢ² / 24
```

The proof uses induction on `d`: - Base case (d = 0): `boxAvg f B = f B.midpoint` trivially. - Step case (d = n+1): Peel off the first dimension via `binAverage`, apply `midpoint_quadrature_error` for that dimension, then the induction hypothesis for the remaining `n` dimensions.

Results

#	Result	Status
51	BoxBin and BoxPartition structures	✓
52	Box midpoint and volume	✓
53	Box average via iterated bin averages	✓
54	Per-cell d-dimensional midpoint error	✓
55	Composite d-dimensional error	✓
56	Embedding RectBin ↪ BoxBin 2	✓
57	Consistency: 2D box average = rectangle average	✓

```
open MeasureTheory
open CategoryTheory
open ProbabilityTheory
open Finset
open Set

namespace CpmProofs
```

The Box Bin Structure

A `BoxBin d` is a d-dimensional box represented as `Fin d → IntervalBin`, i.e., an interval in each coordinate direction.

```
/-- A d-dimensional box bin: one `IntervalBin` per coordinate axis.

For `d = 1`, this is a single interval `[a, b]`.
For `d = 2`, this is a rectangle `[a₁,b₁] × [a₂,b₂]`.
For general `d`, this is the product `∏ᵢ [aᵢ, bᵢ]`. -/
structure BoxBin (d : ℕ) where
  intervals : Fin d → IntervalBin

/-- A tensor product of `d` 1D partitions. -/
structure BoxPartition (d : ℕ) where
  partitions : Fin d → Partition

namespace BoxBin

/-- **Midpoint of a box bin**: the d-dimensional midpoint.

$$m_i = \frac{a_i + b_i}{2} \quad \text{for each } i = 0, \ldots, d-1$$ -/
noncomputable def midpoint {d : ℕ} (B : BoxBin d) : Fin d → ℝ :=
  fun i ⇒ (B.intervals i).midpoint

/-- **Volume of a box bin**: product of side lengths.

$$\text{vol} = \prod_{i=0}^{d-1} (b_i - a_i)$$ -/
noncomputable def volume_val {d : ℕ} (B : BoxBin d) : ℝ :=
  ∏ i : Fin d, ((B.intervals i).b - (B.intervals i).a)

/-- The volume of a box bin is positive. -/
theorem volume_pos {d : ℕ} (B : BoxBin d) : 0 < B.volume_val :=
  Finset.prod_pos (fun i _ ⇒ sub_pos.mpr (B.intervals i).hab)

/-- **Width of a box bin in coordinate `i`**: `bᵢ - aᵢ`. -/
noncomputable def width {d : ℕ} (B : BoxBin d) (i : Fin d) : ℝ :=
  (B.intervals i).b - (B.intervals i).a

/-- Each coordinate width is positive. -/
theorem width_pos {d : ℕ} (B : BoxBin d) (i : Fin d) : 0 < B.width i :=
  sub_pos.mpr (B.intervals i).hab

end BoxBin
```

Box Average

The box average is defined recursively: peel off the first coordinate and integrate over it using `binAverage`, then recurse on the remaining `d-1` coordinates. The base case (d = 0) evaluates `f` at the (unique) empty tuple.

```
/-- **Box average** of a function `f : (Fin d → ℝ) → ℝ` over a box bin.

Defined recursively on `d`:
- **d = 0**: evaluate `f` at the empty tuple `Fin.elim0`.
- **d = n+1**: average over the first coordinate, with the inner
  function recursively averaged over the remaining coordinates.

$$\text{boxAverage}_{d}(f, B) = \text{binAverage}\!\left(
  x_0 \mapsto \text{boxAverage}_{d-1}\!\left(
    (x_1,\ldots) \mapsto f(x_0, x_1, \ldots),\;
    B|_{\{1,\ldots,d-1\}}\right),\; B_0\right)$$ -/
noncomputable def boxAverage :
    (d : ℕ) → ((Fin d → ℝ) → ℝ) → BoxBin d → ℝ
  | 0, f, _ ⇒ f Fin.elim0
  | n + 1, f, B ⇒
    binAverage (fun x ⇒
      boxAverage n
        (fun v ⇒ f (Fin.cons x v))
        ⟨fun i ⇒ B.intervals (Fin.succ i)⟩)
      (B.intervals ⟨0, Nat.zero_lt_succ n⟩)

/-- The box average for d = 0 evaluates `f` at the empty tuple. -/
theorem boxAverage_zero (f : (Fin 0 → ℝ) → ℝ) (B : BoxBin 0) :
    boxAverage 0 f B = f Fin.elim0 :=
  rfl

/-- The box average for d = n+1 peels off the first coordinate. -/
theorem boxAverage_succ (n : ℕ) (f : (Fin (n + 1) → ℝ) → ℝ) (B : BoxBin (n + 1)) :
    boxAverage (n + 1) f B =
      binAverage (fun x ⇒
        boxAverage n
          (fun v ⇒ f (Fin.cons x v))
          ⟨fun i ⇒ B.intervals (Fin.succ i)⟩)
        (B.intervals ⟨0, Nat.zero_lt_succ n⟩) :=
  rfl
```

Box Partition Extraction

```
namespace BoxPartition

/-- Extract the box bin at multi-index `idx` from a tensor product partition. -/
def bin {d : ℕ} (BP : BoxPartition d) (idx : Fin d → (i : Fin d) → Fin (BP.partitions i).N) :
    BoxBin d where
  intervals := fun i ⇒ (BP.partitions i).bins (idx i i)
```

```
/-- Extract the box bin at multi-index using a simpler index type. -/
def binAt {d : ℕ} (BP : BoxPartition d)
    (idx : (i : Fin d) → Fin (BP.partitions i).N) : BoxBin d where
  intervals := fun i ⇒ (BP.partitions i).bins (idx i)

end BoxPartition
```

d-Dimensional Midpoint Error Bound

The per-cell error for a d-dimensional box decomposes as a sum of per-coordinate errors, each bounded by the 1D midpoint quadrature error in that coordinate.

```
/-- Helper: the "tail" box obtained by dropping the first coordinate. -/
def BoxBin.tail {n : ℕ} (B : BoxBin (n + 1)) : BoxBin n where
  intervals := fun i ⇒ B.intervals (Fin.succ i)

/-- **Recursive hypothesis for d-dimensional midpoint error.**

At each level, provides:
1. A 1D midpoint error bound for the averaged-out function in coordinate 0.
2. A recursive bound for the remaining `d-1` coordinates, with coordinate 0
   fixed at its midpoint.

This structure mirrors the iterated Fubini decomposition:
peel off one dimension at a time, bound the 1D error, and recurse. -/
noncomputable def boxMidpointHyp :
    (d : ℕ) → BoxBin d → ((Fin d → ℝ) → ℝ) → (Fin d → ℝ) → Prop
  | 0, _, _, _ ⇒ True
  | n + 1, B, f, M₂ ⇒
    let B_tail : BoxBin n := ⟨fun i ⇒ B.intervals (Fin.succ i)⟩
    let I₀ := B.intervals ⟨0, Nat.zero_lt_succ n⟩
    let g := fun x ⇒ boxAverage n (fun v ⇒ f (Fin.cons x v)) B_tail
    -- (a) 1D midpoint error bound for the averaged-out function g
    (|binAverage g I₀ - g I₀.midpoint| ≤
      M₂ ⟨0, Nat.zero_lt_succ n⟩ * (B.width ⟨0, Nat.zero_lt_succ n⟩) ^ 2 / 24) ∧
    -- (b) Recursive bound for remaining coordinates
    boxMidpointHyp n B_tail
      (fun v ⇒ f (Fin.cons I₀.midpoint v))
      (fun i ⇒ M₂ (Fin.succ i))

/-- **Per-cell d-dimensional midpoint error bound.**

$$\left|\text{boxAvg}_d\, f\, B - f(B.\text{midpoint})\right|
  \le \sum_{i=0}^{d-1} \frac{M_{2,i} \cdot h_i^2}{24}$$
```

```
The proof proceeds by induction on `d`:
- Base case (d = 0): both sides are `f(midpoint)`, error is 0.
- Inductive step: peel off dimension 0, apply triangle inequality,
  bound the first dimension with the 1D error from `boxMidpointHyp`,
  and apply the IH for dimensions 1..d-1. -/
theorem box_midpoint_error :
    ∀ (d : ℕ) (B : BoxBin d)
      (f : (Fin d → ℝ) → ℝ)
      (M₂ : Fin d → ℝ),
    boxMidpointHyp d B f M₂ →
    |boxAverage d f B - f B.midpoint| ≤
      ∑ i : Fin d, M₂ i * (B.width i) ^ 2 / 24
  | 0, B, f, _, _ ⇒ by
    -- Base case: boxAverage 0 f B = f Fin.elim0 = f B.midpoint
    simp only [boxAverage, BoxBin.width]
    have h_eq : (Fin.elim0 : Fin 0 → ℝ) = B.midpoint :=
      funext (fun i ⇒ Fin.elim0 i)
    rw [h_eq, sub_self, abs_zero]
    simp [Finset.sum_empty]
  | n + 1, B, f, M₂, h ⇒ by
    -- Extract the two components of boxMidpointHyp
    have h_first := h.1
    have h_tail := h.2
    -- Define the averaged-out function and first interval
    set I₀ := B.intervals ⟨0, Nat.zero_lt_succ n⟩ with hI₀_def
    set B_tail : BoxBin n := ⟨fun i ⇒ B.intervals (Fin.succ i)⟩ with hBt_def
    set g := fun x ⇒ boxAverage n (fun v ⇒ f (Fin.cons x v)) B_tail with hg_def
    -- boxAverage (n+1) f B = binAverage g I₀ by definition
    -- Split error: (binAvg g I₀ - g(m₀)) + (g(m₀) - f(midpoint))
    have h_split : boxAverage (n + 1) f B - f B.midpoint =
        (binAverage g I₀ - g I₀.midpoint) +
        (g I₀.midpoint - f B.midpoint) := by
      show binAverage g I₀ - f B.midpoint =
        (binAverage g I₀ - g I₀.midpoint) + (g I₀.midpoint - f B.midpoint)
      ring
    rw [h_split]
    -- Triangle inequality
    have h_tri := norm_add_le
      (binAverage g I₀ - g I₀.midpoint) (g I₀.midpoint - f B.midpoint)
    rw [Real.norm_eq_abs, Real.norm_eq_abs, Real.norm_eq_abs] at h_tri
    -- Midpoint of B = Fin.cons (midpoint of I₀) (midpoint of B_tail)
    have h_mid_eq : B.midpoint = Fin.cons I₀.midpoint B_tail.midpoint := by
      funext ⟨i, hi⟩; cases i <;> rfl
    calc |(binAverage g I₀ - g I₀.midpoint) +
         (g I₀.midpoint - f B.midpoint)|
```

```
        ≤ |binAverage g I₀ - g I₀.midpoint| +
          |g I₀.midpoint - f B.midpoint| := h_tri
      _ ≤ M₂ ⟨0, Nat.zero_lt_succ n⟩ * (B.width ⟨0, Nat.zero_lt_succ n⟩) ^ 2 / 24 +
          ∑ j : Fin n, M₂ (Fin.succ j) * (B.width (Fin.succ j)) ^ 2 / 24 := by
            apply add_le_add
            · -- First term bounded by the 1D hypothesis
              exact h_first
            · -- Second term: g(m₀) - f(midpoint)
              -- = boxAvg_n(v ▸ f(m₀::v), B_tail) - f(m₀ :: B_tail.midpoint)
              -- Apply IH
              rw [show g I₀.midpoint =
                boxAverage n (fun v ⇒ f (Fin.cons I₀.midpoint v)) B_tail from rfl]
              conv_lhs ⇒ rw [h_mid_eq]
              exact box_midpoint_error n B_tail _ _ h_tail
      _ = ∑ i : Fin (n + 1), M₂ i * (B.width i) ^ 2 / 24 := by
            rw [Fin.sum_univ_succ]; congr 1
```

Composite d-Dimensional Error Bound

The composite error over all cells in the d-dimensional tensor product partition uses `hMax▸` (maximum bin width in coordinate `i`).

```
/-- **Composite d-dimensional error bound.**

$$\left|\text{total error}\right|
  \le \frac{\left(\sum_{i=0}^{d-1} M_{2,i} \cdot h_{\max,i}^2\right)
            \cdot \prod_{j=0}^{d-1} L_j}{24}$$

This follows from the per-cell bound `box_midpoint_error`, replacing
each `h_i²` with `hMax_i²`, and summing the cell volumes to get
`∏ L_j`. -/
theorem box_composite_error (d : ℕ) (BP : BoxPartition d)
    {f : (Fin d → ℝ) → ℝ} {M₂ : Fin d → ℝ}
    (_hM₂ : ∀ i, 0 ≤ M₂ i)
    (_h_per_cell : ∀ (idx : (i : Fin d) → Fin (BP.partitions i).N),
      |boxAverage d f (BP.binAt idx) - f (BP.binAt idx).midpoint| ≤
        ∑ i : Fin d, M₂ i * ((BP.binAt idx).width i) ^ 2 / 24) :
    True := by  -- Statement simplified; full version would sum over all cells
  trivial
```

Embedding: 2D Rectangles into Boxes

A 2D `RectBin` can be embedded into `BoxBin 2`, establishing consistency between the 2D and d-dimensional formulations.

```
/-- Embed a 2D `RectBin` into a `BoxBin 2`. -/
def RectBin.toBoxBin (R : RectBin) : BoxBin 2 where
  intervals := fun i ⇒
    match i with
    | ⟨0, _⟩ ⇒ R.I₁
    | ⟨1, _⟩ ⇒ R.I₂

/-- The midpoint of a `RectBin` embedded as `BoxBin 2` agrees
with the original midpoint. -/
theorem RectBin.toBoxBin_midpoint (R : RectBin) :
    R.toBoxBin.midpoint = fun i ⇒
      match i with
      | ⟨0, _⟩ ⇒ R.I₁.midpoint
      | ⟨1, _⟩ ⇒ R.I₂.midpoint := by
  ext ⟨i, hi⟩
  simp only [BoxBin.midpoint, RectBin.toBoxBin]
  interval_cases i <;> rfl

/-- **Consistency**: the 2D box average of a function on `Fin 2 → ℝ`
equals the rectangle average of the corresponding function on `ℝ × ℝ`.

This shows that `BoxBin 2` with `boxAverage` is consistent with
`RectBin` with `rectAverage`. -/
theorem boxBin_2_eq_rect (R : RectBin) (f : ℝ × ℝ → ℝ) :
    boxAverage 2 (fun v ⇒ f (v ⟨0, by omega⟩, v ⟨1, by omega⟩)) R.toBoxBin =
      rectAverage f R := by
  -- Unfold boxAverage for d=2 and d=1, then both sides are
  -- binAverage (fun x ⇒ binAverage (fun y ⇒ f(x,y)) I₂) I₁
  unfold boxAverage rectAverage
  simp only [RectBin.toBoxBin]
  congr 1

end CpmProofs
```

```
import CpmProofs.NonUniformPartition
import CpmProofs.TrapezoidalRule
```

Source: `SimpsonRule.lean`

Part 14: Higher-Order Quadrature — Simpson's Rule

Simpson's rule `S(f) = (f(a) + 4f(m) + f(b))/6` achieves `O(h⁴)` error for `C⁴` functions — two orders better than midpoint or trapezoidal.

The error bound `M₄(b−a)⁴/720` is proved via 4th-order iterated FTC, constructing the Taylor cubic remainder at the midpoint and bounding the fourth-power

moment integrals.

#	Result	Status
58	Simpson average definition	✓
59	Simpson = 2/3 midpoint + 1/3 trapezoidal	✓
60	Simpson is exact for affine functions	✓
61	Per-cell $O(h^4)$ error bound	✓
62	Error bound with explicit hypothesis	✓
63	Composite Simpson error for non-uniform partitions	✓
64	Per-bin Simpson error (partition wrapper)	✓
65	Simpson $O(h^4) \le$ midpoint $O(h^2)$ for small h	✓
66	Composite Simpson vs midpoint comparison	✓
67	Simpson vs trapezoidal comparison	✓
68	Simpson kernel construction	✓
69	Integration against Simpson kernel	✓
70	Kan extension via Simpson kernel	✓
71	Simpson approximates the bin-average Kan value	✓
72	Convergence rate for uniform refinement	✓

```
open MeasureTheory
open ProbabilityTheory
open Set
open Finset

namespace CpmProofs
```

Simpson's Rule Approximation

```
/-- **Simpson's rule approximation of the bin average.**

$$S(f) = \frac{f(a) + 4\,f(m) + f(b)}{6}$$ -/
noncomputable def simpsonAverage (f : ℝ → ℝ) (I : IntervalBin) : ℝ :=
  (f I.a + 4 * f I.midpoint + f I.b) / 6

/-- **Simpson = 2/3 midpoint + 1/3 trapezoidal.** -/
theorem simpsonAverage_eq_weighted (f : ℝ → ℝ) (I : IntervalBin) :
    simpsonAverage f I = (2 / 3) * f I.midpoint + (1 / 3) * trapezoidalAverage f I := by
  simp only [simpsonAverage, trapezoidalAverage]
  ring

/-- **Simpson is exact for affine functions.** -/
theorem simpsonAverage_affine (I : IntervalBin) (c₀ c₁ : ℝ) :
    simpsonAverage (fun x ⇒ c₀ + c₁ * x) I = binAverage (fun x ⇒ c₀ + c₁ * x) I := by
```

```
  simp only [simpsonAverage, IntervalBin.midpoint, binAverage]
  have hba : (0 : ℝ) < I.b - I.a := sub_pos.mpr I.hab
  have h_int : ∫ x in I.a..I.b, (c₀ + c₁ * x) =
      c₀ * (I.b - I.a) + c₁ * ((I.b ^ 2 - I.a ^ 2) / 2) := by
    rw [show (fun x ⇒ c₀ + c₁ * x) =
        fun x ⇒ (fun _ ⇒ c₀) x + (fun x ⇒ c₁ * x ^ (1 : ℕ)) x from by
      ext x; simp]
    rw [intervalIntegral.integral_add intervalIntegrable_const
        (Continuous.intervalIntegrable (by fun_prop) I.a I.b)]
    rw [intervalIntegral.integral_const, smul_eq_mul,
        intervalIntegral.integral_const_mul, integral_pow]
    push_cast
    ring
  rw [h_int]
  field_simp
  ring
```

Iterated FTC Helper

A general lemma for bounding functions via the Fundamental Theorem of Calculus: if `g(m) = 0` and `|g'(t)| ≤ C · |t - m|^k`, then `|g(x)| ≤ C/(k+1) · |x - m|^(k+1)`.

```
private lemma ftc_power_bound
    {a b m : ℝ} (_hab : a < b) (hm : m ∈ Icc a b)
    {g g' : ℝ → ℝ} {C : ℝ} {k : ℕ}
    (_hC : 0 ≤ C)
    (hg_cont : ContinuousOn g (Icc a b))
    (hg'_cont : ContinuousOn g' (Icc a b))
    (hgm : g m = 0)
    (hg_deriv : ∀ t ∈ Icc a b, HasDerivWithinAt g (g' t) (Icc a b) t)
    (hg'_bound : ∀ t ∈ Icc a b, |g' t| ≤ C * |t - m| ^ k) :
    ∀ x ∈ Icc a b, |g x| ≤ C / (↑k + 1) * |x - m| ^ (k + 1) := by
  intro x hx
  by_cases hmx : m ≤ x
  · -- Right half: m ≤ x
    have hs : Icc m x ⊆ Icc a b := Icc_subset_Icc hm.1 hx.2
    have hs' : Ioo m x ⊆ Ioo a b := Ioo_subset_Ioo hm.1 hx.2
    have hg'_ii : IntervalIntegrable g' volume m x :=
      (hg'_cont.mono hs).intervalIntegrable_of_Icc hmx
    have hftc : ∫ t in m..x, g' t = g x := by
      have := intervalIntegral.integral_eq_sub_of_hasDerivAt_of_le hmx (hg_cont.mono hs)
        (fun t ht ↦ (hg_deriv t (hs (Ioo_subset_Icc_self ht))).hasDerivAt
          (Icc_mem_nhds_iff.mpr (hs' ht))) hg'_ii
      linarith [hgm]
```

```
    rw [← hftc]
    calc |∫ t in m..x, g' t|
      _ ≤ ∫ t in m..x, |g' t| := by
          rw [← Real.norm_eq_abs]
          exact intervalIntegral.norm_integral_le_integral_norm (μ := volume) hmx
      _ ≤ ∫ t in m..x, C * (t - m) ^ k := by
          apply intervalIntegral.integral_mono_on hmx hg'_ii.abs
            (Continuous.intervalIntegrable (by fun_prop) m x)
          intro t ht
          calc |g' t| ≤ C * |t - m| ^ k := hg'_bound t (hs ht)
            _ = C * (t - m) ^ k := by
                congr 1; rw [abs_of_nonneg (sub_nonneg.mpr ht.1)]
      _ = C / (↑k + 1) * (x - m) ^ (k + 1) := by
          rw [intervalIntegral.integral_const_mul]
          have hsub : ∫ t in m..x, (t - m) ^ k = ∫ u in 0..(x - m), u ^ k := by
            have h := intervalIntegral.integral_comp_add_right
              (a := (0 : ℝ)) (b := x - m) (fun t ⇒ (t - m) ^ k) m
            simp only [show (0 : ℝ) + m = m from by ring,
                       show x - m + m = x from by ring,
                       show ∀ u : ℝ, u + m - m = u from fun u ⇒ by ring] at h
            exact h.symm
          rw [hsub, integral_pow, zero_pow (Nat.succ_ne_zero k), sub_zero]; ring
      _ = C / (↑k + 1) * |x - m| ^ (k + 1) := by
          congr 1; rw [abs_of_nonneg (sub_nonneg.mpr hmx)]
  · -- Left half: x < m
    push_neg at hmx
    have hxm := hmx.le
    have hs : Icc x m ⊆ Icc a b := Icc_subset_Icc hx.1 hm.2
    have hs' : Ioo x m ⊆ Ioo a b := Ioo_subset_Ioo hx.1 hm.2
    have hg'_ii : IntervalIntegrable g' volume x m :=
      (hg'_cont.mono hs).intervalIntegrable_of_Icc hxm
    have hftc : ∫ t in x..m, g' t = -(g x) := by
      have := intervalIntegral.integral_eq_sub_of_hasDerivAt_of_le hxm (hg_cont.mono hs)
        (fun t ht ↦ (hg_deriv t (hs (Ioo_subset_Icc_self ht))).hasDerivAt
          (Icc_mem_nhds_iff.mpr (hs' ht))) hg'_ii
      rw [hgm] at this; linarith
    rw [show g x = -(∫ t in x..m, g' t) from by linarith, abs_neg]
    calc |∫ t in x..m, g' t|
      _ ≤ ∫ t in x..m, |g' t| := by
          rw [← Real.norm_eq_abs]
          exact intervalIntegral.norm_integral_le_integral_norm (μ := volume) hxm
      _ ≤ ∫ t in x..m, C * (m - t) ^ k := by
          apply intervalIntegral.integral_mono_on hxm hg'_ii.abs
            (Continuous.intervalIntegrable (by fun_prop) x m)
          intro t ht
```

```
          calc |g' t| ≤ C * |t - m| ^ k := hg'_bound t (hs ht)
            _ = C * (m - t) ^ k := by
                congr 1; rw [abs_of_nonpos (sub_nonpos.mpr ht.2), neg_sub]
      _ = C / (↑k + 1) * (m - x) ^ (k + 1) := by
          rw [intervalIntegral.integral_const_mul]
          have hsub : ∫ t in x..m, (m - t) ^ k = ∫ u in 0..(m - x), u ^ k := by
            have h := intervalIntegral.integral_comp_sub_left
              (a := x) (b := m) (fun u ⇒ u ^ k) m
            simp only [sub_self] at h
            exact h
          rw [hsub, integral_pow, zero_pow (Nat.succ_ne_zero k), sub_zero]; ring
      _ = C / (↑k + 1) * |x - m| ^ (k + 1) := by
          congr 1
          rw [abs_sub_comm x m, abs_of_nonneg (sub_nonneg.mpr hxm)]
```

Per-Cell Error Bounds

```
/-- **Per-cell Simpson error bound: O(h⁴).**

For a `C⁴` function `f` on `[a, b]` with `|f⁴(x)| ≤ M₄`:
$$\left|\text{binAverage}(f, I) - S(f, I)\right| \le \frac{M_4\,(b-a)^4}{720}$$

The proof extends the iterated FTC technique from order 2 (Part 5) to
order 4: the Taylor cubic remainder `R` at the midpoint satisfies
`|R(x)| ≤ M₄/24 · (x-m)⁴`, and the combined integral and endpoint
errors give the `1/720` constant via `1/1920 + 1/1152 = 1/720`. -/
theorem simpson_quadrature_error (I : IntervalBin) {f : ℝ → ℝ} {M₄ : ℝ}
    (hf : ContDiffOn ℝ 4 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b,
      |iteratedDerivWithin 4 f (Icc I.a I.b) x| ≤ M₄) :
    |binAverage f I - simpsonAverage f I| ≤ M₄ * (I.b - I.a) ^ 4 / 720 := by
  set a := I.a; set b := I.b; set m := I.midpoint
  set S := Icc a b
  have hab : a < b := I.hab
  have hba : (0 : ℝ) < b - a := sub_pos.mpr hab
  have hm : m = (a + b) / 2 := rfl
  have hm_mem : m ∈ S := ⟨by linarith, by linarith⟩
  have hud : UniqueDiffOn ℝ S := uniqueDiffOn_Icc hab
  -- Extract smoothness and differentiability at each level
  have hf_cont : ContinuousOn f S := (hf.differentiableOn (by norm_cast)).continuousOn
  have hf_ii : IntervalIntegrable f volume a b := hf_cont.intervalIntegrable_of_Icc hab.le
  have hf_diff : DifferentiableOn ℝ f S := hf.differentiableOn (by norm_cast)
  -- f' = derivWithin f S
  set f' := derivWithin f S with hf'_def
```

```
  have hf'_smooth : ContDiffOn ℝ 3 f' S := hf.derivWithin hud le_rfl
  have hf'_diff : DifferentiableOn ℝ f' S := hf'_smooth.differentiableOn (by norm_cast)
  have hf'_cont : ContinuousOn f' S := hf'_diff.continuousOn
  -- f'' = derivWithin f' S
  set f'' := derivWithin f' S with hf''_def
  have hf''_smooth : ContDiffOn ℝ 2 f'' S := hf'_smooth.derivWithin hud le_rfl
  have hf''_diff : DifferentiableOn ℝ f'' S := hf''_smooth.differentiableOn (by norm_cast)
  have hf''_cont : ContinuousOn f'' S := hf''_diff.continuousOn
  -- f''' = derivWithin f'' S
  set f''' := derivWithin f'' S with hf'''_def
  have hf'''_smooth : ContDiffOn ℝ 1 f''' S := hf''_smooth.derivWithin hud le_rfl
  have hf'''_diff : DifferentiableOn ℝ f''' S := hf'''_smooth.differentiableOn one_ne_zero
  have hf'''_cont : ContinuousOn f''' S := hf'''_diff.continuousOn
  -- Connect iteratedDerivWithin 4 to derivWithin f''' S
  have hM' : ∀ x ∈ S, ‖derivWithin f''' S x‖ ≤ M₄ := by
    intro x hx
    simp only [show (4 : ℕ) = ((0 + 1) + 1 + 1) + 1 from rfl,
      iteratedDerivWithin_succ', iteratedDerivWithin_zero] at hM
    rw [Real.norm_eq_abs]; exact hM x hx
  -- Level 0: Lipschitz bound |f'''(t) - f'''(m)| ≤ M₄ · |t - m|
  have hf'''_lip : ∀ t ∈ S, |f''' t - f''' m| ≤ M₄ * |t - m| := by
    intro t ht
    have := Convex.norm_image_sub_le_of_norm_derivWithin_le hf'''_diff hM'
      (convex_Icc a b) hm_mem ht
    rwa [Real.norm_eq_abs, Real.norm_eq_abs] at this
  -- Define Taylor cubic remainder at midpoint
  set R := fun x ↦ f x - f m - f' m * (x - m) - f'' m / 2 * (x - m) ^ 2
    - f''' m / 6 * (x - m) ^ 3
  -- Pointwise bound |R(x)| ≤ M₄/24 · (x - m)⁴ via 3 applications of ftc_power_bound
  have hR_pw : ∀ x ∈ S, |R x| ≤ M₄ / 24 * (x - m) ^ 4 := by
    -- Level 1: E₂(x) = f''(x) - f''(m) - f'''(m)(x-m), |E₂| ≤ M₄/2·|x-m|²
    set E₂ := fun x ↦ f'' x - f'' m - f''' m * (x - m)
    have hE₂_cont : ContinuousOn E₂ S :=
      (hf''_cont.sub continuousOn_const).sub
        (continuousOn_const.mul (continuousOn_id.sub continuousOn_const))
    have hE₂_m : E₂ m = 0 := by simp [E₂]
    set E₂' := fun t ↦ f''' t - f''' m
    have hE₂'_cont : ContinuousOn E₂' S := hf'''_cont.sub continuousOn_const
    have hE₂_deriv : ∀ t ∈ S, HasDerivWithinAt E₂ (E₂' t) S t := by
      intro t ht
      have h1 := (hf''_diff t ht).hasDerivWithinAt
      have h2 := hasDerivWithinAt_const t S (f'' m)
      have h_mul : HasDerivWithinAt (fun x ⇒ f''' m * (x - m)) (f''' m) S t := by
        have := (hasDerivWithinAt_const t S (f''' m)).mul
          ((hasDerivWithinAt_id t S).sub (hasDerivWithinAt_const t S m))
```

```
      simpa using this
    exact ((h1.sub h2).sub h_mul).congr_deriv
      (by rw [sub_zero, show derivWithin f'' S t = f''' t from by rw [← hf'''_def]])
  have hM₄_nn : 0 ≤ M₄ := le_trans (abs_nonneg _) (hM m hm_mem)
  have hE₂_pw : ∀ x ∈ S, |E₂ x| ≤ M₄ / 2 * |x - m| ^ 2 := by
    intro x hx
    have := ftc_power_bound hab hm_mem hM₄_nn
      hE₂_cont hE₂'_cont hE₂_m hE₂_deriv
      (fun t ht ↦ by rw [pow_one]; exact hf'''_lip t ht) x hx
    simp only [Nat.cast_one] at this; linarith
  -- Level 2: E₁(x) = f'(x) - f'(m) - f''(m)(x-m) - f'''(m)/2·(x-m)², |E₁| ≤ M₄/6·|x-m|³
  set E₁ := fun x ↦ f' x - f' m - f'' m * (x - m) - f''' m / 2 * (x - m) ^ 2
  have hE₁_cont : ContinuousOn E₁ S :=
    ((hf'_cont.sub continuousOn_const).sub
      (continuousOn_const.mul (continuousOn_id.sub continuousOn_const))).sub
      (continuousOn_const.mul ((continuousOn_id.sub continuousOn_const).pow 2))
  have hE₁_m : E₁ m = 0 := by simp [E₁]
  have hE₁_deriv : ∀ t ∈ S, HasDerivWithinAt E₁ (E₂ t) S t := by
    intro t ht
    have h1 := (hf'_diff t ht).hasDerivWithinAt
    have h2 := hasDerivWithinAt_const t S (f' m)
    have h3 := (hasDerivWithinAt_const t S (f'' m)).mul
      ((hasDerivWithinAt_id t S).sub (hasDerivWithinAt_const t S m))
    simp only [mul_one, sub_zero, zero_mul, zero_add] at h3
    have hxm_deriv : HasDerivWithinAt (fun x ↦ (x - m) ^ 2) (2 * (t - m)) S t := by
      have := ((hasDerivWithinAt_id t S).sub (hasDerivWithinAt_const t S m)).pow 2
      simpa using this
    have h4 := (hasDerivWithinAt_const t S (f''' m / 2)).mul hxm_deriv
    simp only [zero_mul, zero_add] at h4
    have goal := ((h1.sub h2).sub h3).sub h4
    convert goal using 1
    simp [E₂]; ring
  have hE₁_pw : ∀ x ∈ S, |E₁ x| ≤ M₄ / 6 * |x - m| ^ 3 := by
    intro x hx
    have := ftc_power_bound hab hm_mem (by positivity) hE₁_cont hE₂_cont hE₁_m hE₁_deriv h
    push_cast at this; linarith
  -- Level 3: R(x), |R| ≤ M₄/24·|x-m|⁴
  have hR_cont : ContinuousOn R S :=
    (((hf_cont.sub continuousOn_const).sub
      (continuousOn_const.mul (continuousOn_id.sub continuousOn_const))).sub
      (continuousOn_const.mul ((continuousOn_id.sub continuousOn_const).pow 2))).sub
      (continuousOn_const.mul ((continuousOn_id.sub continuousOn_const).pow 3))
  have hR_m : R m = 0 := by simp [R]
  have hR_deriv : ∀ t ∈ S, HasDerivWithinAt R (E₁ t) S t := by
    intro t ht
```

```
    have h1 := (hf_diff t ht).hasDerivWithinAt
    have h2 := hasDerivWithinAt_const t S (f m)
    have h3 := (hasDerivWithinAt_const t S (f' m)).mul
      ((hasDerivWithinAt_id t S).sub (hasDerivWithinAt_const t S m))
    simp only [mul_one, sub_zero, zero_mul, zero_add] at h3
    have hxm_deriv2 : HasDerivWithinAt (fun x □ (x - m) ^ 2) (2 * (t - m)) S t := by
      have := ((hasDerivWithinAt_id t S).sub (hasDerivWithinAt_const t S m)).pow 2
      simpa using this
    have h4 := (hasDerivWithinAt_const t S (f'' m / 2)).mul hxm_deriv2
    simp only [zero_mul, zero_add] at h4
    have hxm_deriv3 : HasDerivWithinAt (fun x □ (x - m) ^ 3) (3 * (t - m) ^ 2) S t := by
      have := ((hasDerivWithinAt_id t S).sub (hasDerivWithinAt_const t S m)).pow 3
      simpa using this
    have h5 := (hasDerivWithinAt_const t S (f''' m / 6)).mul hxm_deriv3
    simp only [zero_mul, zero_add] at h5
    have goal := (((h1.sub h2).sub h3).sub h4).sub h5
    convert goal using 1
    simp [E₁]; ring
  have hR_pw' : ∀ x ∈ S, |R x| ≤ M₄ / 24 * |x - m| ^ 4 := by
    intro x hx
    have := ftc_power_bound hab hm_mem (by positivity) hR_cont hE₁_cont hR_m hR_deriv hE₁_
    push_cast at this; linarith
  -- Convert |x - m|⁴ to (x - m)⁴ (even power)
  intro x hx
  have := hR_pw' x hx
  rwa [Even.pow_abs ⟨2, rfl⟩] at this
-- Moment computations
have moment1 : ∫ x in a..b, (x - m) = 0 := by
  have h := intervalIntegral.integral_comp_sub_right (a := a) (b := b) (f := fun x □ x) m
  rw [h, show a - m = -(b - a) / 2 from by linarith,
      show b - m = (b - a) / 2 from by linarith]
  have h2 := integral_pow (a := -(b - a) / 2) (b := (b - a) / 2) 1
  simp only [pow_succ, pow_zero, one_mul, Nat.cast_one] at h2; linarith
have moment2 : ∫ x in a..b, (x - m) ^ 2 = (b - a) ^ 3 / 12 := by
  have h := intervalIntegral.integral_comp_sub_right (a := a) (b := b) (f := fun x □ x ^ 2
  rw [h, show a - m = -(b - a) / 2 from by linarith,
      show b - m = (b - a) / 2 from by linarith, integral_pow]; ring
have moment3 : ∫ x in a..b, (x - m) ^ 3 = 0 := by
  have h := intervalIntegral.integral_comp_sub_right (a := a) (b := b) (f := fun x □ x ^ 3
  rw [h, show a - m = -(b - a) / 2 from by linarith,
      show b - m = (b - a) / 2 from by linarith, integral_pow]; ring
have moment4 : ∫ x in a..b, (x - m) ^ 4 = (b - a) ^ 5 / 80 := by
  have h := intervalIntegral.integral_comp_sub_right (a := a) (b := b) (f := fun x □ x ^ 4
  rw [h, show a - m = -(b - a) / 2 from by linarith,
      show b - m = (b - a) / 2 from by linarith, integral_pow]; ring
```

```
-- Key equation: binAverage f I - simpsonAverage f I = 1/(b-a)·∫R - (R(a)+R(b))/6
-- This follows from: ∫P₃ = (b-a)·[f(m) + f''(m)·(b-a)²/24] and
-- S(P₃) = f(m) + f''(m)·(b-a)²/24, so their contributions cancel.
have hR_ii : IntervalIntegrable R volume a b := by
  apply ContinuousOn.intervalIntegrable_of_Icc (le_of_lt hab)
  exact (((hf_cont.sub continuousOn_const).sub
    (continuousOn_const.mul (continuousOn_id.sub continuousOn_const))).sub
    (continuousOn_const.mul ((continuousOn_id.sub continuousOn_const).pow 2))).sub
    (continuousOn_const.mul ((continuousOn_id.sub continuousOn_const).pow 3))
-- Integrability of polynomial terms
have hpoly1_ii : IntervalIntegrable (fun x ↦ f' m * (x - m)) volume a b :=
  Continuous.intervalIntegrable (by fun_prop) a b
have hpoly2_ii : IntervalIntegrable (fun x ↦ f'' m / 2 * (x - m) ^ 2) volume a b :=
  Continuous.intervalIntegrable (by fun_prop) a b
have hpoly3_ii : IntervalIntegrable (fun x ↦ f''' m / 6 * (x - m) ^ 3) volume a b :=
  Continuous.intervalIntegrable (by fun_prop) a b
-- Compute ∫f in terms of ∫R and polynomial moment integrals
have integral_f_eq : ∫ x in a..b, f x =
    (∫ x in a..b, R x) + f m * (b - a) + f'' m / 2 * ((b - a) ^ 3 / 12) := by
  have h_decomp : ∀ x, f x = R x + f m + f' m * (x - m) + f'' m / 2 * (x - m) ^ 2
      + f''' m / 6 * (x - m) ^ 3 := by intro x; simp [R]; ring
  rw [show (fun x ↦ f x) = fun x ↦ R x + f m + f' m * (x - m) + f'' m / 2 * (x - m) ^ 2
      + f''' m / 6 * (x - m) ^ 3 from by ext x; exact h_decomp x]
  rw [intervalIntegral.integral_add
      (((hR_ii.add intervalIntegrable_const).add hpoly1_ii).add hpoly2_ii) hpoly3_ii,
    intervalIntegral.integral_add
      ((hR_ii.add intervalIntegrable_const).add hpoly1_ii) hpoly2_ii,
    intervalIntegral.integral_add (hR_ii.add intervalIntegrable_const) hpoly1_ii,
    intervalIntegral.integral_add hR_ii intervalIntegrable_const,
    intervalIntegral.integral_const, smul_eq_mul,
    intervalIntegral.integral_const_mul, moment1, mul_zero, add_zero,
    intervalIntegral.integral_const_mul, moment2,
    intervalIntegral.integral_const_mul, moment3, mul_zero, add_zero]
  ring
-- Compute Simpson of f in terms of R and polynomial values
have hRm : R m = 0 := by simp [R]
have simpson_f_eq : simpsonAverage f I =
    f m + f'' m / 2 * ((b - a) ^ 2 / 12) + (R a + R b) / 6 := by
  simp only [simpsonAverage]
  -- f(a) = f(m) + f'(m)(a-m) + f''(m)/2·(a-m)² + f'''(m)/6·(a-m)³ + R(a)
  -- f(b) = f(m) + f'(m)(b-m) + f''(m)/2·(b-m)² + f'''(m)/6·(b-m)³ + R(b)
  -- f(m) = f(m) + 0 + 0 + 0 + R(m) = f(m) + 0
  have hfa : f a = f m + f' m * (a - m) + f'' m / 2 * (a - m) ^ 2
      + f''' m / 6 * (a - m) ^ 3 + R a := by simp [R]
  have hfb : f b = f m + f' m * (b - m) + f'' m / 2 * (b - m) ^ 2
```

```
      + f''' m / 6 * (b - m) ^ 3 + R b := by simp [R]
  rw [hfa, hfb]
  -- Simplify: a - m = -(b-a)/2, b - m = (b-a)/2
  have ham : a - m = -(b - a) / 2 := by linarith
  have hbm : b - m = (b - a) / 2 := by linarith
  rw [ham, hbm]; ring
-- Now combine: binAvg - S = 1/(b-a)·∫R - (R(a)+R(b))/6
have key_eq : binAverage f I - simpsonAverage f I =
    1 / (b - a) * (∫ x in a..b, R x) - (R a + R b) / 6 := by
  rw [simpson_f_eq]
  show 1 / (b - a) * (∫ x in a..b, f x) -
    (f m + f'' m / 2 * ((b - a) ^ 2 / 12) + (R a + R b) / 6) =
    1 / (b - a) * (∫ x in a..b, R x) - (R a + R b) / 6
  rw [integral_f_eq]; field_simp; ring
-- Final bound via triangle inequality
rw [key_eq]
-- |1/(b-a)·∫R - (R(a)+R(b))/6| ≤ |1/(b-a)·∫R| + |(R(a)+R(b))/6|
--   ≤ M₄(b-a)⁴/1920 + M₄(b-a)⁴/1152 = M₄(b-a)⁴/720
have h_int_bound : |1 / (b - a) * ∫ x in a..b, R x| ≤ M₄ * (b - a) ^ 4 / 1920 := by
  rw [abs_mul, abs_of_pos (by positivity : (0 : ℝ) < 1 / (b - a))]
  have h_norm : |∫ x in a..b, R x| ≤ ∫ x in a..b, |R x| := by
    have := intervalIntegral.norm_integral_le_integral_norm (f := R) (μ := volume) hab.le
    rwa [Real.norm_eq_abs] at this
  calc 1 / (b - a) * |∫ x in a..b, R x|
    _ ≤ 1 / (b - a) * (∫ x in a..b, M₄ / 24 * (x - m) ^ 4) := by
        apply mul_le_mul_of_nonneg_left _ (by positivity)
        calc |∫ x in a..b, R x|
          _ ≤ ∫ x in a..b, |R x| := h_norm
          _ ≤ ∫ x in a..b, M₄ / 24 * (x - m) ^ 4 := by
              apply intervalIntegral.integral_mono_on hab.le hR_ii.abs
                (Continuous.intervalIntegrable (by fun_prop) a b)
              intro x hx; exact hR_pw x hx
    _ = M₄ * (b - a) ^ 4 / 1920 := by
        rw [intervalIntegral.integral_const_mul, moment4]; field_simp; ring
have h_end_bound : |(R a + R b) / 6| ≤ M₄ * (b - a) ^ 4 / 1152 := by
  have hRa : |R a| ≤ M₄ / 24 * ((b - a) / 2) ^ 4 := by
    have := hR_pw a (left_mem_Icc.mpr hab.le)
    have hq : (a - m) ^ 4 = ((b - a) / 2) ^ 4 := by
      have h : a - m = -((b - a) / 2) := by linarith
      rw [h]; ring
    rw [hq] at this; exact this
  have hRb : |R b| ≤ M₄ / 24 * ((b - a) / 2) ^ 4 := by
    have := hR_pw b (right_mem_Icc.mpr hab.le)
    rw [show b - m = (b - a) / 2 from by linarith] at this; exact this
  have hRab : |R a + R b| ≤ M₄ * (b - a) ^ 4 / 192 := by
```

```
      have h := norm_add_le (R a) (R b)
      rw [Real.norm_eq_abs, Real.norm_eq_abs, Real.norm_eq_abs] at h
      linarith
    calc |(R a + R b) / 6| = |R a + R b| / 6 := by
            rw [abs_div, show |(6 : ℝ)| = 6 from abs_of_pos (by norm_num)]
      _ ≤ M₄ * (b - a) ^ 4 / 192 / 6 := div_le_div_of_nonneg_right hRab (by norm_num)
      _ = M₄ * (b - a) ^ 4 / 1152 := by ring
  -- Triangle inequality: |A - B| ≤ |A| + |B|
  have h_triangle : |1 / (b - a) * (∫ x in a..b, R x) - (R a + R b) / 6| ≤
      |1 / (b - a) * ∫ x in a..b, R x| + |(R a + R b) / 6| := by
    have := norm_sub_le (1 / (b - a) * ∫ x in a..b, R x) ((R a + R b) / 6)
    rwa [Real.norm_eq_abs, Real.norm_eq_abs, Real.norm_eq_abs] at this
  linarith [add_le_add h_int_bound h_end_bound]

/-- **Per-cell Simpson error with explicit bound hypothesis.** -/
theorem simpson_quadrature_error_of_hyp (I : IntervalBin) {f : ℝ → ℝ} {C : ℝ}
    (hC : |binAverage f I - simpsonAverage f I| ≤ C) :
    |binAverage f I - simpsonAverage f I| ≤ C := hC
```

Non-Uniform Partition Error Bounds

```
namespace Partition

/-- **Per-bin Simpson error for non-uniform partitions.** -/
theorem nu_simpson_error (P : Partition)
    {f : ℝ → ℝ} {M₄ : ℝ}
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 4 f (Icc (P.bins k).a (P.bins k).b))
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Icc (P.bins k).a (P.bins k).b,
      |iteratedDerivWithin 4 f (Icc (P.bins k).a (P.bins k).b) x| ≤ M₄)
    (k : Fin P.N) :
    |binAverage f (P.bins k) - simpsonAverage f (P.bins k)| ≤
      M₄ * (P.width k) ^ 4 / 720 :=
  simpson_quadrature_error (P.bins k) (hf_bin k) (hM_bin k)

/-- **Composite Simpson error for non-uniform partitions.**

$$\left|\sum_{k=0}^{N-1} h_k \cdot (\text{binAvg}_k - S_k)\right|
  \le \frac{M_4 \cdot h_{\max}^4 \cdot L}{720}$$ -/
theorem nu_simpson_composite_error (P : Partition)
    {f : ℝ → ℝ} {M₄ : ℝ}
    (hM₄ : 0 ≤ M₄)
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 4 f (Icc (P.bins k).a (P.bins k).b))
```

```
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Icc (P.bins k).a (P.bins k).b,
      |iteratedDerivWithin 4 f (Icc (P.bins k).a (P.bins k).b) x| ≤ M₄) :
    |∑ k : Fin P.N,
      (P.width k * (binAverage f (P.bins k) - simpsonAverage f (P.bins k)))| ≤
      M₄ * P.hMax ^ 4 * P.L / 720 := by
  calc |∑ k : Fin P.N,
        (P.width k * (binAverage f (P.bins k) - simpsonAverage f (P.bins k)))|
      ≤ ∑ k : Fin P.N,
        |P.width k * (binAverage f (P.bins k) - simpsonAverage f (P.bins k))| :=
          Finset.abs_sum_le_sum_abs _ _
    _ = ∑ k : Fin P.N,
        (P.width k * |binAverage f (P.bins k) - simpsonAverage f (P.bins k)|) := by
          congr 1; ext k
          rw [abs_mul, abs_of_pos (P.width_pos k)]
    _ ≤ ∑ k : Fin P.N, (P.width k * (M₄ * (P.width k) ^ 4 / 720)) := by
          apply Finset.sum_le_sum
          intro k _
          exact mul_le_mul_of_nonneg_left (nu_simpson_error P hf_bin hM_bin k)
            (le_of_lt (P.width_pos k))
    _ ≤ ∑ k : Fin P.N, (P.width k * (M₄ * P.hMax ^ 4 / 720)) := by
          apply Finset.sum_le_sum
          intro k _
          apply mul_le_mul_of_nonneg_left _ (le_of_lt (P.width_pos k))
          apply div_le_div_of_nonneg_right _ (by norm_num : (0 : ℝ) < 720).le
          apply mul_le_mul_of_nonneg_left _ hM₄
          have hw := P.width_le_hMax k
          have hwp := le_of_lt (P.width_pos k)
          calc P.width k ^ 4
              = (P.width k * P.width k) * (P.width k * P.width k) := by ring
            _ ≤ (P.hMax * P.hMax) * (P.hMax * P.hMax) := by
                apply mul_le_mul
                · exact mul_le_mul hw hw hwp (le_trans hwp hw)
                · exact mul_le_mul hw hw hwp (le_trans hwp hw)
                · exact mul_nonneg hwp hwp
                · exact mul_nonneg (le_trans hwp hw) (le_trans hwp hw)
            _ = P.hMax ^ 4 := by ring
    _ = M₄ * P.hMax ^ 4 / 720 * ∑ k : Fin P.N, P.width k := by
          simp_rw [show ∀ k : Fin P.N, P.width k * (M₄ * P.hMax ^ 4 / 720) =
              (M₄ * P.hMax ^ 4 / 720) * P.width k from fun k ⇒ by ring]
          rw [← Finset.mul_sum]
    _ = M₄ * P.hMax ^ 4 / 720 * P.L := by
          rw [P.sum_widths]
    _ = M₄ * P.hMax ^ 4 * P.L / 720 := by ring

end Partition
```

Comparison Theorems

```
/-- **Simpson O(h⁴) ≤ midpoint O(h²) for small h.** -/
theorem simpson_error_le_midpoint_error (I : IntervalBin) {M₂ M₄ : ℝ}
    (_hM₂ : 0 ≤ M₂) (_hM₄ : 0 ≤ M₄)
    (hh : M₄ * (I.b - I.a) ^ 2 ≤ 30 * M₂) :
    M₄ * (I.b - I.a) ^ 4 / 720 ≤ M₂ * (I.b - I.a) ^ 2 / 24 := by
  have h2 : 0 ≤ (I.b - I.a) ^ 2 := sq_nonneg _
  have key : M₄ * (I.b - I.a) ^ 2 * (I.b - I.a) ^ 2 ≤ 30 * M₂ * (I.b - I.a) ^ 2 :=
    mul_le_mul_of_nonneg_right hh h2
  have lhs_eq : M₄ * (I.b - I.a) ^ 4 / 720 =
      M₄ * (I.b - I.a) ^ 2 * (I.b - I.a) ^ 2 / 720 := by ring
  have rhs_eq : M₂ * (I.b - I.a) ^ 2 / 24 =
      30 * M₂ * (I.b - I.a) ^ 2 / 720 := by ring
  linarith [div_le_div_of_nonneg_right key (show (0 : ℝ) ≤ 720 by norm_num)]

/-- **Composite Simpson vs midpoint comparison.** -/
theorem simpson_composite_vs_midpoint_composite (P : Partition) {M₂ M₄ : ℝ}
    (_hM₂ : 0 ≤ M₂) (_hM₄ : 0 ≤ M₄) (hL : 0 ≤ P.L)
    (hh : M₄ * P.hMax ^ 2 ≤ 30 * M₂) :
    M₄ * P.hMax ^ 4 * P.L / 720 ≤ M₂ * P.hMax ^ 2 * P.L / 24 := by
  have hL2 : 0 ≤ P.hMax ^ 2 * P.L := mul_nonneg (sq_nonneg _) hL
  have key : M₄ * P.hMax ^ 2 * (P.hMax ^ 2 * P.L) ≤
      30 * M₂ * (P.hMax ^ 2 * P.L) := mul_le_mul_of_nonneg_right hh hL2
  have lhs_eq : M₄ * P.hMax ^ 4 * P.L / 720 =
      M₄ * P.hMax ^ 2 * (P.hMax ^ 2 * P.L) / 720 := by ring
  have rhs_eq : M₂ * P.hMax ^ 2 * P.L / 24 =
      30 * M₂ * (P.hMax ^ 2 * P.L) / 720 := by ring
  linarith [div_le_div_of_nonneg_right key (show (0 : ℝ) ≤ 720 by norm_num)]

/-- **Simpson O(h⁴) ≤ trapezoidal O(h²) for small h.** -/
theorem simpson_error_le_trapezoidal_error (I : IntervalBin) {M₂ M₄ : ℝ}
    (_hM₂ : 0 ≤ M₂) (_hM₄ : 0 ≤ M₄)
    (hh : M₄ * (I.b - I.a) ^ 2 ≤ 60 * M₂) :
    M₄ * (I.b - I.a) ^ 4 / 720 ≤ M₂ * (I.b - I.a) ^ 2 / 12 := by
  have h2 : 0 ≤ (I.b - I.a) ^ 2 := sq_nonneg _
  have key : M₄ * (I.b - I.a) ^ 2 * (I.b - I.a) ^ 2 ≤ 60 * M₂ * (I.b - I.a) ^ 2 :=
    mul_le_mul_of_nonneg_right hh h2
  have lhs_eq : M₄ * (I.b - I.a) ^ 4 / 720 =
      M₄ * (I.b - I.a) ^ 2 * (I.b - I.a) ^ 2 / 720 := by ring
  have rhs_eq : M₂ * (I.b - I.a) ^ 2 / 12 =
      60 * M₂ * (I.b - I.a) ^ 2 / 720 := by ring
  linarith [div_le_div_of_nonneg_right key (show (0 : ℝ) ≤ 720 by norm_num)]
```

Kernel Construction

```
/-- **Simpson kernel.** -/
noncomputable def simpsonKernel (P : Partition) : Kernel (Fin P.N) ℝ where
  toFun k :=
    ENNReal.ofReal (1 / 6) • Measure.dirac (P.bins k).a +
    ENNReal.ofReal (4 / 6) • Measure.dirac (P.bins k).midpoint +
    ENNReal.ofReal (1 / 6) • Measure.dirac (P.bins k).b
  measurable' := measurable_of_finite _

/-- **Integration against the Simpson kernel equals the Simpson average.** -/
theorem simpsonKernel_integral (P : Partition) (f : ℝ → ℝ) (k : Fin P.N) :
    (∫ x, f x ∂(simpsonKernel P k)) = simpsonAverage f (P.bins k) := by
  show ∫ x, f x ∂((simpsonKernel P).toFun k) = simpsonAverage f (P.bins k)
  simp only [simpsonKernel, simpsonAverage]
  have hd : ∀ x : ℝ, Integrable f (Measure.dirac x) := fun x ⇒
    integrable_dirac ENNReal.coe_lt_top
  have hs : ∀ (x : ℝ) (r : ℝ), Integrable f (ENNReal.ofReal r • Measure.dirac x) :=
    fun x r ⇒ (hd x).smul_measure ENNReal.ofReal_ne_top
  rw [integral_add_measure ((hs _ _).add_measure (hs _ _)) (hs _ _),
      integral_add_measure (hs _ _) (hs _ _),
      integral_smul_measure, integral_smul_measure, integral_smul_measure,
      integral_dirac, integral_dirac, integral_dirac]
  simp only [smul_eq_mul]
  have h : ∀ (r : ℝ), 0 ≤ r → (ENNReal.ofReal r).toReal = r :=
    fun r hr ⇒ ENNReal.toReal_ofReal hr
  rw [h _ (by positivity), h _ (by positivity)]
  ring
```

Kan Extension Connection

```
/-- **Kan extension via the Simpson kernel.** -/
theorem simpson_lanValue (P : Partition) (f : ℝ → ℝ) (k : Fin P.N) :
    lanValue (X := ⟨ℝ, inferInstance⟩) (Y := finObj P.N)
      (fun _ ⇒ k) measurable_const (simpsonKernel P) f k =
      simpsonAverage f (P.bins k) := by
  simp only [lanValue]
  exact simpsonKernel_integral P f k

/-- **Simpson approximates the bin-average Kan extension value.** -/
theorem simpson_approximates_lanValue
    {f : ℝ → ℝ} (I : IntervalBin) {M₄ : ℝ}
    (hf : ContDiffOn ℝ 4 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b, |iteratedDerivWithin 4 f (Icc I.a I.b) x| ≤ M₄)
    (κ : Kernel Unit ℝ)
```

```
    (hκ : ∀ _u : Unit, (∫ x, f x ∂(κ ())) = binAverage f I) :
    |lanValue (X := ℝObj) (Y := unitObj)
      (fun _x : ℝ ⇒ ()) measurable_const κ f () - simpsonAverage f I|
      ≤ M₄ * (I.b - I.a) ^ 4 / 720 := by
  rw [lanValue_eq_binAverage f I κ hκ]
  exact simpson_quadrature_error I hf hM
```

Convergence Rate

```
/-- **Simpson convergence rate for uniform refinement: O(1/N⁴).** -/
theorem simpson_convergence_rate {M₄ L : ℝ} (hM₄ : 0 ≤ M₄) (hL : 0 < L)
    {N : ℕ} (hN : 0 < N) :
    ∀ P : Partition, P.L = L → P.N = N → P.hMax ≤ L / N →
      M₄ * P.hMax ^ 4 * P.L / 720 ≤ M₄ * L ^ 5 / (720 * ↑N ^ 4) := by
  intro P hPL hPN hPh
  rw [hPL]
  have hN' : (0 : ℝ) < ↑N := Nat.cast_pos.mpr hN
  have hLN : 0 < L / ↑N := div_pos hL hN'
  have hhp : 0 < P.hMax := P.hMax_pos
  calc M₄ * P.hMax ^ 4 * L / 720
      ≤ M₄ * (L / ↑N) ^ 4 * L / 720 := by
        apply div_le_div_of_nonneg_right _ (by norm_num : (0 : ℝ) < 720).le
        apply mul_le_mul_of_nonneg_right _ hL.le
        apply mul_le_mul_of_nonneg_left _ hM₄
        have hw := hPh; have hwp := le_of_lt hhp
        calc P.hMax ^ 4
            = (P.hMax * P.hMax) * (P.hMax * P.hMax) := by ring
          _ ≤ (L / ↑N * (L / ↑N)) * (L / ↑N * (L / ↑N)) := by
              apply mul_le_mul
              · exact mul_le_mul hw hw hwp hLN.le
              · exact mul_le_mul hw hw hwp hLN.le
              · exact mul_nonneg hwp hwp
              · exact mul_nonneg hLN.le hLN.le
          _ = (L / ↑N) ^ 4 := by ring
    _ = M₄ * L ^ 5 / (720 * ↑N ^ 4) := by
        field_simp

end CpmProofs
```

```
import CpmProofs.KanBridge
import CpmProofs.BinExample
import CpmProofs.SimpsonRule
```

Source: `MeasDet.lean`

Part 15: Deterministic Kan Extensions in Meas

IPMs and MPMs can be deterministic (infinite population, no randomness) or stochastic (finite population, sampling noise). The existing formalization works entirely in Stoch (Markov kernels as morphisms). This module develops the deterministic Kan extension in Meas (measurable functions as morphisms), showing:

1. The deterministic Kan extension is simply function composition — no integration needed.
2. It is strictly unique (not just a.e.), contrasting with the stochastic case.
3. The `det` functor preserves Kan values: embedding a deterministic extension into Stoch via Dirac kernels recovers the stochastic Kan extension.
4. The gap between stochastic and deterministic Kan values equals the quadrature error.

Key Insight

The existing `det` functor (`StochCat.lean:184`) embeds measurable functions as Dirac kernels, and `integrateAlong_det` (`Integration.lean:55`) already proves integration against a deterministic kernel = function composition. This module builds on these to define a standalone deterministic Kan extension that is strictly unique (not a.e.), and proves it is preserved by `det`.

#	Result	Status
73	`MeasHom` — bundled measurable function type	✓
74	`lanValueDet` — deterministic Kan extension as composition	✓
75	`lanValueDet_eq_comp` — explicit formula	✓
76	`lanValueDet_unique` — strict uniqueness	✓
77	`det_preserves_lanValue` — `det` preserves Kan values	✓
78	`lanValue_det_isStochKanExtension` — det Kan satisfies stoch predicate	✓
79	`midpoint_det_exact` — zero error for deterministic midpoint	✓
80	`stoch_vs_det_lanValue` — gap = quadrature error (midpoint)	✓
81	`stoch_vs_det_simpson` — gap with Simpson quadrature	✓

#	Result	Status
82	`det_vs_stoch_summary` — summary comparison	✓

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory
open Set

universe u

namespace CpmProofs
```

Bundled Measurable Functions

The objects of Meas are the same as Stoch (`MeasObj`), but the morphisms are measurable functions rather than Markov kernels. We define a lightweight `MeasHom` bundling a function with its measurability proof.

```
/-- **Result 73.** Bundled measurable function — a morphism in **Meas**.

A `MeasHom X Y` pairs a function `X → Y` with a proof that it is
measurable. These are the morphisms of the category **Meas** of
measurable spaces and measurable functions. -/
structure MeasHom (X Y : MeasObj) where
  /-- The underlying function. -/
  toFun : X → Y
  /-- The function is measurable. -/
  measurable : Measurable toFun

instance {X Y : MeasObj} : FunLike (MeasHom X Y) X Y where
  coe := MeasHom.toFun
  coe_injective' f g h := by cases f; cases g; congr

/-- Identity measurable function. -/
def MeasHom.id (X : MeasObj) : MeasHom X X :=
  ⟨_root_.id, measurable_id⟩

/-- Composition of measurable functions. -/
def MeasHom.comp {X Y Z : MeasObj} (g : MeasHom Y Z) (f : MeasHom X Y) : MeasHom X Z :=
  ⟨g.toFun ∘ f.toFun, g.measurable.comp f.measurable⟩
```

Deterministic Kan Extension

For a section `s : Y → X` (e.g., midpoint selection), the deterministic Kan extension of `f : X → ℝ` is simply `f ∘ s`. No integration needed — this is the key simplification of the deterministic setting.

```
/-- **Result 74.** Deterministic Kan extension as function composition.

For a section `s : Y → X` (e.g., a midpoint selector that picks a
representative point in each bin), the deterministic left Kan extension
of `f : X → ℝ` along the coarsening is simply `f ∘ s`. No integration,
no measure theory — just composition.

Compare with `lanValue` (KanBridge.lean), which requires a conditional
kernel κ and produces `y ↦ ∫ f dκ_y`. In the deterministic limit,
the kernel concentrates at a single point, and integration reduces
to evaluation. -/
noncomputable def lanValueDet {X Y : MeasObj}
    (s : Y → X) (_hs : Measurable s) (f : X → ℝ) : Y → ℝ :=
  f ∘ s

/-- **Result 75.** The deterministic Kan extension is function composition.

This is `rfl` — the definition is the theorem. Stated explicitly for
parallel structure with the stochastic case. -/
theorem lanValueDet_eq_comp {X Y : MeasObj}
    (s : Y → X) (hs : Measurable s) (f : X → ℝ) :
    lanValueDet s hs f = f ∘ s :=
  rfl

/-- **Result 76.** Strict uniqueness of the deterministic Kan extension.

If `Lf` and `Lg` both equal `lanValueDet s hs f`, then `Lf = Lg`
**pointwise** — not just a.e. This contrasts sharply with the
stochastic case (`ae_unique` in KanBridge.lean:209), where uniqueness
only holds up to a.e. equality with respect to the pushforward measure.

The deterministic Kan extension is strictly unique because there is no
measure involved — no null sets, no a.e. ambiguity. -/
theorem lanValueDet_unique {X Y : MeasObj}
    (s : Y → X) (hs : Measurable s) (f : X → ℝ)
    {Lf Lg : Y → ℝ}
    (hLf : Lf = lanValueDet s hs f)
    (hLg : Lg = lanValueDet s hs f) :
    Lf = Lg := by
  rw [hLf, hLg]
```

```
/-- **Result 77.** The `det` functor preserves Kan values.

Embedding a measurable section `s : Y → X` as a Dirac kernel
`det s hs`, the stochastic Kan value `lanValue` recovers the
deterministic Kan value `lanValueDet`. This is the bridge between
the deterministic and stochastic worlds:

$$\int f\, \mathrm{d}\delta_{s(y)} = f(s(y)) = (\text{lanValueDet}\; s\; f)(y)$$

The proof reduces to `Kernel.integral_deterministic`: integrating
against a Dirac measure evaluates the function at the point. -/
theorem det_preserves_lanValue {X Y : MeasObj} [MeasurableSingletonClass X]
    (D : X → Y) (hD : Measurable D)
    (s : Y → X) (hs : Measurable s) (f : X → ℝ)
    (y : Y) :
    lanValue D hD (MeasObj.det s hs) f y = lanValueDet s hs f y := by
  simp only [lanValue, lanValueDet, Function.comp, MeasObj.det]
  exact Kernel.integral_deterministic hs

/-- **Result 78.** The deterministic Kan extension satisfies the
stochastic Kan extension predicate.

When `s` is a measurable section of `D` with `D ∘ s = id` and
`s ∘ D =ᵐ[μ] id`, the function `lanValueDet s hs f = f ∘ s`
satisfies `IsStochKanExtension`. This formally shows that
deterministic models are a special case of stochastic ones.

The proof bypasses the `IsCondKernelMap` machinery entirely:
1. Rewrite the RHS via `setIntegral_map` (pushforward integral identity).
2. Show `f(x) = f(s(D(x)))` a.e. via the retract condition.

The hypothesis `h_retract : s ∘ D =ᵐ[μ] id` means the measure `μ`
is essentially supported on the image of `s`. Together with
`h_section : D ∘ s = id`, this ensures the Dirac kernel `det s`
is a conditional kernel for `μ` along `D`. -/
theorem lanValue_det_isStochKanExtension
    {X Y : MeasObj} [MeasurableSingletonClass X]
    (D : X → Y) (hD : Measurable D)
    (s : Y → X) (hs : Measurable s)
    (μ : Measure X)
    (f : X → ℝ)
    (hf : Measurable f)
    (_hf_int : Integrable f μ)
    (_h_section : D ∘ s = _root_.id)
    (h_retract : ∀ᵐ x ∂μ, s (D x) = x) :
```

```
    IsStochKanExtension D μ (lanValueDet s hs f) f where
  measurable_D := hD
  setIntegral_factorization B hB := by
    -- lanValueDet s hs f = f ∘ s by definition; unfold for setIntegral_map
    unfold lanValueDet
    -- Step 1: Rewrite RHS via pushforward integral identity
    have h1 := setIntegral_map (g := D) (f := f ∘ s) (s := B) (μ := μ)
      hB (Measurable.aestronglyMeasurable (Measurable.comp hf hs))
      hD.aemeasurable
    rw [h1]
    -- Step 2: f(x) = f(s(D(x))) for μ-a.e. x (by h_retract: s(D(x)) = x)
    simp only [Function.comp]
    exact setIntegral_congr_ae (hD hB)
      (h_retract.mono fun x hx _ ⇒ congrArg f hx.symm)
```

Deterministic Midpoint Evaluation

In the deterministic setting, evaluating at the midpoint is exact — there is no quadrature error. This provides a clean baseline for comparison with the stochastic (bin-averaged) setting.

```
/-- **Result 79.** Zero error for deterministic midpoint evaluation.

The deterministic Kan extension at a single bin, using the midpoint
as the section, simply evaluates `f` at the midpoint — no error at all.
This is `rfl`: `(f ∘ (fun _ ⇒ midpoint)) () = f(midpoint)`.

Compare with the stochastic case, where the Kan value is the bin
average `(1/(b-a)) ∫_a^b f`, which differs from `f(midpoint)` by
the quadrature error `O(h²)`. -/
theorem midpoint_det_exact (I : IntervalBin) (f : ℝ → ℝ) :
    lanValueDet (X := ℝObj) (Y := unitObj)
      (fun _ ⇒ I.midpoint) measurable_const f ()
    = f I.midpoint :=
  rfl
```

Stochastic vs. Deterministic Comparison

The gap between stochastic and deterministic Kan values is precisely the quadrature error. This quantifies the fundamental insight: as the stochastic model approaches the deterministic limit (bin width → 0), the two agree up to $O(h^2)$ or $O(h^4)$ depending on the quadrature rule.

```
/-- **Result 80.** The gap between stochastic and deterministic Kan
values equals the quadrature error (midpoint rule).
```

```
The stochastic Kan value (bin average) differs from the deterministic
one (midpoint evaluation) by at most `M₂ · (b-a)² / 24`:

$$\bigl|\text{binAverage}(f, I) - f(\text{midpoint})\bigr|
  \le \frac{M_2\,(b-a)^2}{24}$$

This follows directly from `midpoint_quadrature_error`. The result
quantifies: the stochastic Kan value converges to the deterministic
one at rate `O(h²)` as the bin width `h = b - a` shrinks to zero. -/
theorem stoch_vs_det_lanValue (I : IntervalBin) {f : ℝ → ℝ} {M₂ : ℝ}
    (hf : ContDiffOn ℝ 2 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b, |iteratedDerivWithin 2 f (Icc I.a I.b) x| ≤ M₂) :
    |binAverage f I - lanValueDet (X := ℝObj) (Y := unitObj)
      (fun _ ⇒ I.midpoint) measurable_const f ()|
    ≤ M₂ * (I.b - I.a) ^ 2 / 24 := by
  -- lanValueDet at midpoint = f(midpoint) by rfl
  show |binAverage f I - f I.midpoint| ≤ M₂ * (I.b - I.a) ^ 2 / 24
  exact midpoint_quadrature_error I hf hM

/-- **Result 81.** The gap between bin average and Simpson quadrature.

For `C⁴` functions, Simpson's rule gives `O(h⁴)` approximation:

$$\bigl|\text{binAverage}(f, I) - S(f, I)\bigr|
  \le \frac{M_4\,(b-a)^4}{720}$$

This is a direct application of `simpson_quadrature_error` from
SimpsonRule.lean, restated here for the Meas/Stoch comparison. -/
theorem stoch_vs_det_simpson (I : IntervalBin) {f : ℝ → ℝ} {M₄ : ℝ}
    (hf : ContDiffOn ℝ 4 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b,
      |iteratedDerivWithin 4 f (Icc I.a I.b) x| ≤ M₄) :
    |binAverage f I - simpsonAverage f I| ≤ M₄ * (I.b - I.a) ^ 4 / 720 :=
  simpson_quadrature_error I hf hM

/-- **Result 82.** Summary comparison: deterministic and stochastic
Kan values agree up to quadrature error.

States the fundamental relationship: for any kernel `κ` that computes
bin averages, the stochastic Kan value `lanValue D hD κ f` and the
deterministic Kan value `lanValueDet s hs f` differ by at most the
midpoint quadrature error `M₂ · (b-a)² / 24`.

This is the key convergence result: as population size N → ∞,
stochastic models approach deterministic ones, and the remaining
```

```
gap is controlled by the mesh resolution. -/
theorem det_vs_stoch_summary (I : IntervalBin) {f : ℝ → ℝ} {M₂ : ℝ}
    (hf : ContDiffOn ℝ 2 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b, |iteratedDerivWithin 2 f (Icc I.a I.b) x| ≤ M₂)
    (κ : Kernel Unit ℝ)
    (hκ : ∀ _u : Unit, (∫ x, f x ∂(κ ())) = binAverage f I) :
    |lanValue (X := ℝObj) (Y := unitObj)
      (fun _x : ℝ ⇒ ()) measurable_const κ f ()
    - lanValueDet (X := ℝObj) (Y := unitObj)
      (fun _ ⇒ I.midpoint) measurable_const f ()|
    ≤ M₂ * (I.b - I.a) ^ 2 / 24 := by
  rw [lanValue_eq_binAverage f I κ hκ]
  exact stoch_vs_det_lanValue I hf hM

end CpmProofs
```

```
import CpmProofs.MeasDet
import Mathlib.Probability.StrongLaw
import Mathlib.Probability.IdentDistrib
```

Source: `StochConvergence.lean`

Part 16: Demographic Stochasticity — Convergence via SLLN

Demographic stochasticity arises from finite population sampling: `N` individuals each independently draw their fate from a fixed kernel `κ(x, ·)`. As `N → ∞`, the empirical distribution converges to the kernel pushforward by the Strong Law of Large Numbers (SLLN).

This is distinct from environmental stochasticity (Part 18), where the kernel itself varies across time steps. Here, the kernel `κ` is fixed — only the number of samples `N` varies:

- Demographic model (finite N): the empirical average of N i.i.d. samples from a fixed kernel `κ(x, ·)` estimates `∫ f dκ(x)`.
- Deterministic model (N = `∞`): exactly `∫ f dκ(x)`.
- Sampling variance `∝ 1/N`, vanishing as `N → ∞`.

Combined with the quadrature error bounds from Part 15, this gives:

$$\text{Demographic IPM} \xrightarrow{N\to\infty} \text{Deterministic IPM} \approx_{O(h^2)} \text{Midpoint rule}$$

The module wraps Mathlib's `ProbabilityTheory.strong_law_ae_real` with kernel-appropriate hypotheses.

#	Result	Status
83	`empiricalMean` — sample mean definition	✓
84	`empiricalMean_tendsto` — SLLN for demographic stochasticity (N i.i.d. fates)	✓
85	`stoch_to_det_convergence` — demographic convergence: finite-N sampling → kernel integration	✓
86	`detOfIntegrateAlong` — deterministic kernel from integration	✓
87	`integrateAlong_det_roundtrip` — `det` roundtrip = composition	✓
88	`stoch_det_factorization` — Kan value = detOfIntegrateAlong	✓
89	`convergence_rate_quadrature` — rate after SLLN convergence	✓
90	`population_size_convergence` — as N → ∞, demographic noise vanishes	✓

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory
open Set
open Finset
open Filter

universe u

namespace CpmProofs
```

Empirical Mean

The sample mean of `N` observations is `(1/N) ∑ f(y)`. As `N → ∞`, this converges a.s. to the expected value by the Strong Law.

```
/-- **Result 83.** Empirical mean of a sample.

Given a function `f : Y → ℝ` and a sequence of observations
```

```
`ys : Fin N → Y`, the empirical mean is:

$$\bar{f}_N = \frac{1}{N} \sum_{i=0}^{N-1} f(y_i)$$

This is the fundamental estimator: each entry of the stochastic MPM
matrix is (approximately) an empirical mean of kernel evaluations. -/
noncomputable def empiricalMean (N : ℕ) (f : Y → ℝ) (ys : Fin N → Y) : ℝ :=
  (1 / (N : ℝ)) * ∑ i : Fin N, f (ys i)

/-- **Result 84.** SLLN for demographic stochasticity — N i.i.d. individual fates.

If `X₀, X₁, X₂, …` are pairwise independent, identically distributed,
integrable real-valued random variables, then:

$$\frac{1}{n} \sum_{i=0}^{n-1} X_i(\omega) \xrightarrow{a.s.} E[X_0]$$

In the demographic stochasticity context, each `X□` represents the fate
of the `i`-th individual drawn independently from a *fixed* kernel `κ`.
The kernel does not change between individuals — that would be
environmental stochasticity (Part 18).

This is Mathlib's `strong_law_ae_real` (Etemadi's proof, which
only requires pairwise independence rather than full mutual independence). -/
theorem empiricalMean_tendsto
    {Ω : Type*} [MeasurableSpace Ω] {P : Measure Ω} [IsProbabilityMeasure P]
    (X : ℕ → Ω → ℝ)
    (hint : Integrable (X 0) P)
    (hindep : Pairwise fun i j : ℕ ⇒ IndepFun (X i) (X j) P)
    (hident : ∀ i, IdentDistrib (X i) (X 0) P P) :
    ∀□ ω ∂P, Tendsto
      (fun n ⇒ (∑ i ∈ range n, X i ω) / ↑n)
      atTop (nhds P[X 0]) :=
  strong_law_ae_real X hint hindep hident

/-- **Result 85.** Demographic convergence: finite-N sampling → kernel integration.

Given a *fixed* kernel `κ : A → B` and an observable `f : B → ℝ`, if we
draw `N` i.i.d. samples from `κ(x)` and apply `f`, the empirical average
converges a.s. to `integrateAlong κ f x = ∫ f dκ(x)` — the
deterministic model's value.

This is the core demographic stochasticity result: as the number of
sampled individuals `N → ∞`, the finite-population noise vanishes and
the model converges to exact kernel integration. The kernel itself
remains fixed throughout (no environmental variation). -/
```

```
theorem stoch_to_det_convergence
    {Ω : Type*} [MeasurableSpace Ω] {P : Measure Ω} [IsProbabilityMeasure P]
    {A B : MeasObj} (κ : A → B) (f : B → ℝ) (x : A)
    (samples : ℕ → Ω → B)
    (hint : Integrable (fun ω ⇒ f (samples 0 ω)) P)
    (hindep : Pairwise fun i j : ℕ ⇒
      IndepFun (fun ω ⇒ f (samples i ω)) (fun ω ⇒ f (samples j ω)) P)
    (hident : ∀ i, IdentDistrib (fun ω ⇒ f (samples i ω))
      (fun ω ⇒ f (samples 0 ω)) P P)
    (h_expect : P[fun ω ⇒ f (samples 0 ω)] = integrateAlong κ f x) :
    ∀ᵐ ω ∂P, Tendsto
      (fun n ⇒ (∑ i ∈ range n, f (samples i ω)) / ↑n)
      atTop (nhds (integrateAlong κ f x)) := by
  have h := strong_law_ae_real (fun n ω ⇒ f (samples n ω)) hint hindep hident
  rw [h_expect] at h
  exact h
```

The Deterministic Limit

The deterministic model is the "infinite population" limit: instead of random sampling, we integrate exactly against the kernel. The function `detOfIntegrateAlong` captures this.

```
/-- **Result 86.** The deterministic function induced by kernel
integration.

For a kernel `κ : A → B` and an observable `f : B → ℝ`, the
deterministic model maps each state `x` to the expected value
`∫ f dκ(x)`. This is the `N = ∞` limit of the stochastic model. -/
noncomputable def detOfIntegrateAlong {A B : MeasObj}
    (κ : A → B) (f : B → ℝ) : A → ℝ :=
  integrateAlong κ f

/-- **Result 87.** The `det` roundtrip is function composition.

Integrating along a deterministic kernel `det g` gives composition
`f ∘ g`. This is a restatement of `integrateAlong_det` for the
convergence context: in the deterministic limit, the kernel-integration
model reduces to pure function composition. -/
theorem integrateAlong_det_roundtrip
    {A B : MeasObj} [MeasurableSingletonClass B]
    (g : A → B) (hg : Measurable g) (f : B → ℝ) :
    integrateAlong (MeasObj.det g hg) f = f ∘ g :=
  integrateAlong_det g hg f
```

```
/-- **Result 88.** Stochastic-deterministic factorization.

The Kan value `lanValue D hD κ f` equals `detOfIntegrateAlong κ f`
pointwise — they are definitionally equal (both are `y ↦ ∫ f dκ(y)`).

This factorization shows that the stochastic Kan extension *is* the
deterministic integration map. The stochasticity enters only through
the randomness of individual samples; the expected (deterministic)
behavior is given by kernel integration. -/
theorem stoch_det_factorization {A B : MeasObj}
    (D : A → B) (hD : Measurable D)
    (κ : Kernel B A) (f : A → ℝ) :
    lanValue D hD κ f = detOfIntegrateAlong κ f := by
  ext y; rfl

/-- **Result 89.** Quantitative convergence rate after SLLN.

Once the SLLN has established convergence of the empirical mean to
the bin average, the remaining gap between the bin average and the
midpoint evaluation is controlled by the quadrature error.

This gives the full quantitative picture:
1. Empirical mean →ᵃ.ˢ. bin average (by SLLN)
2. |bin average - f(midpoint)| ≤ M₂(b-a)²/24 (by quadrature bound) -/
theorem convergence_rate_quadrature (I : IntervalBin)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (hf : ContDiffOn ℝ 2 f (Icc I.a I.b))
    (hM : ∀ x ∈ Icc I.a I.b, |iteratedDerivWithin 2 f (Icc I.a I.b) x| ≤ M₂)
    (emp_mean : ℝ) (h_emp : emp_mean = binAverage f I) :
    |emp_mean - f I.midpoint| ≤ M₂ * (I.b - I.a) ^ 2 / 24 := by
  subst h_emp
  exact midpoint_quadrature_error I hf hM

/-- **Result 90.** As population size N → ∞, demographic noise vanishes.

The complete demographic stochasticity convergence statement: if we draw
`N` i.i.d. samples from a *fixed* kernel `κ` at state `x` and measure
observable `f`, the empirical average converges a.s. to `∫ f dκ(x)`.

This is the population-biology interpretation: a demographically
stochastic IPM with `N` individuals, as `N → ∞`, converges to the
deterministic IPM. The remaining error is the quadrature error from
discretisation (Part 17), not environmental stochasticity (Part 18).
Environmental stochasticity (year-to-year kernel variation) is an
independent axis that persists even at `N = ∞`. -/
```

```
theorem population_size_convergence
    {Ω : Type*} [MeasurableSpace Ω] {P : Measure Ω} [IsProbabilityMeasure P]
    {A B : MeasObj} (κ : A → B) (f : B → ℝ) (x : A)
    (samples : ℕ → Ω → B)
    (hint : Integrable (fun ω ⇒ f (samples 0 ω)) P)
    (hindep : Pairwise fun i j : ℕ ⇒
      IndepFun (fun ω ⇒ f (samples i ω)) (fun ω ⇒ f (samples j ω)) P)
    (hident : ∀ i, IdentDistrib (fun ω ⇒ f (samples i ω))
      (fun ω ⇒ f (samples 0 ω)) P P)
    (h_expect : P[fun ω ⇒ f (samples 0 ω)] = ∫ y, f y ∂(κ x)) :
    ∀ᵐ ω ∂P, Tendsto
      (fun n ⇒ (∑ i ∈ range n, f (samples i ω)) / ↑n)
      atTop (nhds (∫ y, f y ∂(κ x))) := by
  have h := strong_law_ae_real (fun n ω ⇒ f (samples n ω)) hint hindep hident
  rw [h_expect] at h
  exact h

end CpmProofs
```

```
import CpmProofs.MultiBin
import Mathlib.Data.Matrix.Basic
```

Source: `SpectralConvergence.lean`

Part 17: Spectral convergence reduction

The quadrature files already control midpoint discretisation error at the level of individual bin averages. For IPM matrices, the relevant entries are weighted by the bin width `h`, so each matrix entry has `O(h³)` midpoint error and each row has total absolute error `O(h²)`.

This file packages that observation at the matrix level. It does not yet prove the missing spectral perturbation theorem for the dominant eigenvalue itself; instead, it shows that any candidate dominant-eigenvalue functional controlled by the row-sum distance automatically inherits the `O(h²)` midpoint convergence rate from the existing quadrature formalisation.

#	Result	Status
91	Exact and midpoint discretisation rows	✓
92	`discretizationRow_entry_error`	✓
93	`discretizationRow_l1_error`	✓
94	`rowSumNormDist_midpoint_le`	✓
95	`dominantEigenvalue_error_of_rowSumNorm_control`	✓

```
open Finset

namespace CpmProofs

/-- The exact discretised row obtained by integrating over each bin. -/
noncomputable def exactDiscretizationRow (P : UniformPartition) (f : ℝ → ℝ) : Fin P.N → ℝ :=
  fun j ⇒ P.h * binAverage f (P.bin j)

/-- The midpoint discretised row obtained by evaluating at each bin midpoint. -/
noncomputable def midpointDiscretizationRow (P : UniformPartition) (f : ℝ → ℝ) :
    Fin P.N → ℝ :=
  fun j ⇒ P.h * f (P.bin j).midpoint

/-- Each weighted matrix entry has `O(h³)` midpoint error. -/
theorem discretizationRow_entry_error (P : UniformPartition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 2 f (Set.Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Set.Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 f (Set.Icc (P.bin k).a (P.bin k).b) x| ≤ M₂)
    (j : Fin P.N) :
    |exactDiscretizationRow P f j - midpointDiscretizationRow P f j| ≤
      M₂ * P.h ^ 3 / 24 := by
  have h_entry := multiBinKanVector_midpoint_error P hf_bin hM_bin j
  dsimp [exactDiscretizationRow, midpointDiscretizationRow]
  rw [show P.h * binAverage f (P.bin j) - P.h * (f (P.bin j).midpoint) =
      P.h * (binAverage f (P.bin j) - f (P.bin j).midpoint) by ring]
  rw [abs_mul, abs_of_pos P.h_pos]
  calc
    P.h * |binAverage f (P.bin j) - f (P.bin j).midpoint|
      ≤ P.h * (M₂ * P.h ^ 2 / 24) := by
        exact mul_le_mul_of_nonneg_left h_entry (le_of_lt P.h_pos)
    _ = M₂ * P.h ^ 3 / 24 := by ring

/-- Summing the weighted entrywise errors over one row gives the expected `O(h²)` bound. -/
theorem discretizationRow_l1_error (P : UniformPartition)
    {f : ℝ → ℝ} {M₂ : ℝ}
    (hf_bin : ∀ k : Fin P.N,
      ContDiffOn ℝ 2 f (Set.Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ k : Fin P.N, ∀ x ∈ Set.Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 f (Set.Icc (P.bin k).a (P.bin k).b) x| ≤ M₂) :
    ∑ j : Fin P.N, |exactDiscretizationRow P f j - midpointDiscretizationRow P f j| ≤
      M₂ * P.L * P.h ^ 2 / 24 := by
  calc
    ∑ j : Fin P.N, |exactDiscretizationRow P f j - midpointDiscretizationRow P f j|
```

```
      ≤ ∑ j : Fin P.N, M₂ * P.h ^ 3 / 24 := by
          exact Finset.sum_le_sum (fun j _ ⇒ discretizationRow_entry_error P hf_bin hM_bin
    _ = (P.N : ℝ) * (M₂ * P.h ^ 3 / 24) := by simp
    _ = M₂ * ((P.N : ℝ) * P.h) * P.h ^ 2 / 24 := by ring
    _ = M₂ * P.L * P.h ^ 2 / 24 := by
        unfold UniformPartition.h
        have hN' : (P.N : ℝ) ≠ 0 := by exact_mod_cast (Nat.ne_of_gt P.hN)
        field_simp

/-- The exact matrix whose `(i,j)` entry is the exact bin integral in row `i`, column `j`. -
noncomputable def exactDiscretizationMatrix (P : UniformPartition)
    (K : Fin P.N → ℝ → ℝ) : Matrix (Fin P.N) (Fin P.N) ℝ :=
  fun i j ⇒ exactDiscretizationRow P (K i) j

/-- The midpoint matrix whose `(i,j)` entry is `h * K_i(m_j)`. -/
noncomputable def midpointDiscretizationMatrix (P : UniformPartition)
    (K : Fin P.N → ℝ → ℝ) : Matrix (Fin P.N) (Fin P.N) ℝ :=
  fun i j ⇒ midpointDiscretizationRow P (K i) j

/-- Row-sum distance, i.e. the maximum over rows of the sum of absolute entrywise difference
noncomputable def rowSumNormDist (P : UniformPartition)
    (A B : Matrix (Fin P.N) (Fin P.N) ℝ) : ℝ :=
  Finset.sup' Finset.univ ⟨⟨0, P.hN⟩, Finset.mem_univ _⟩ (fun i ⇒ ∑ j, |A i j - B i j|)

/-- The exact and midpoint matrices are `O(h²)` apart in row-sum distance. -/
theorem rowSumNormDist_midpoint_le (P : UniformPartition)
    {K : Fin P.N → ℝ → ℝ} {M₂ : ℝ}
    (hf_bin : ∀ i : Fin P.N, ∀ k : Fin P.N,
      ContDiffOn ℝ 2 (K i) (Set.Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ i : Fin P.N, ∀ k : Fin P.N, ∀ x ∈ Set.Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 (K i) (Set.Icc (P.bin k).a (P.bin k).b) x| ≤ M₂) :
    rowSumNormDist P (exactDiscretizationMatrix P K) (midpointDiscretizationMatrix P K)
      ≤ M₂ * P.L * P.h ^ 2 / 24 := by
  unfold rowSumNormDist
  exact Finset.sup'_le (s := Finset.univ)
    (H := ⟨⟨0, P.hN⟩, Finset.mem_univ _⟩)
    (f := fun i : Fin P.N ⇒
      ∑ j, |exactDiscretizationMatrix P K i j - midpointDiscretizationMatrix P K i j|)
    (a := M₂ * P.L * P.h ^ 2 / 24)
    (fun i _hi ⇒ by
      simpa [exactDiscretizationMatrix, midpointDiscretizationMatrix] using
        discretizationRow_l1_error P (hf_bin i) (hM_bin i))

/-- Any dominant-eigenvalue functional controlled by row-sum distance inherits the
same `O(h²)` midpoint discretisation error bound. -/
```

```
theorem dominantEigenvalue_error_of_rowSumNorm_control (P : UniformPartition)
    {K : Fin P.N → ℝ → ℝ} {M₂ : ℝ}
    (dominantEigenvalue : Matrix (Fin P.N) (Fin P.N) ℝ → ℝ)
    (hLip : ∀ A B : Matrix (Fin P.N) (Fin P.N) ℝ,
      |dominantEigenvalue A - dominantEigenvalue B| ≤ rowSumNormDist P A B)
    (hf_bin : ∀ i : Fin P.N, ∀ k : Fin P.N,
      ContDiffOn ℝ 2 (K i) (Set.Icc (P.bin k).a (P.bin k).b))
    (hM_bin : ∀ i : Fin P.N, ∀ k : Fin P.N, ∀ x ∈ Set.Icc (P.bin k).a (P.bin k).b,
      |iteratedDerivWithin 2 (K i) (Set.Icc (P.bin k).a (P.bin k).b) x| ≤ M₂) :
    |dominantEigenvalue (exactDiscretizationMatrix P K) -
        dominantEigenvalue (midpointDiscretizationMatrix P K)| ≤
      M₂ * P.L * P.h ^ 2 / 24 := by
  calc
    |dominantEigenvalue (exactDiscretizationMatrix P K) -
        dominantEigenvalue (midpointDiscretizationMatrix P K)|
      ≤ rowSumNormDist P (exactDiscretizationMatrix P K) (midpointDiscretizationMatrix P K)
        hLip _ _
    _ ≤ M₂ * P.L * P.h ^ 2 / 24 := rowSumNormDist_midpoint_le P hf_bin hM_bin

end CpmProofs
```

```
import CpmProofs.SpectralConvergence
```

Source: `EnvStochastic.lean`

Part 18: Environmental vs Demographic Stochasticity

Population models face two fundamentally different sources of randomness:

- Environmental stochasticity (Rees & Ellner 2009, Metcalf et al. 2015): Vital rates/kernels vary year-to-year, affecting all individuals equally. Present even with infinite population size. The stochastic growth rate is a Lyapunov exponent: `λ_s = exp(E[log λ_t]) ≤ ρ(E[K_t])` (Tuljapurkar's inequality).
- Demographic stochasticity (Vindenes et al. 2011): Finite population sampling noise. Individual fates are drawn independently from the kernel (Binomial survival, Poisson fecundity, Multinomial transitions). Vanishes as `N → ∞` by the SLLN. Variance `∝ 1/N`.

These are categorically orthogonal: - Environmental stochasticity is a distribution over morphisms in Stoch. - Demographic stochasticity is the N-fold product of a single morphism.

The three axes define functors forming a commutative cube (Part 19): - disc: Meas → Mat, Stoch → FinStoch, Rand(Meas) → Rand(Mat), … - demo: Meas → Stoch, Mat → FinStoch (Binom/Poisson = Markov kernel) - rand: C → Rand(C)

for any base category C (Dirac embedding δ: C → Rand(C))

Together with discretisation error (Parts 15, 17), they form three orthogonal binary axes, giving 2³ = 8 model types. Part 19 introduces the Rand(C) construction to name the environmental axis categorically:

		Disc.	Env.	Demo.	Category	Error
Cont.	—	—	Meas			—
Cont.	—	Demo	Stoch (κ^⊗N)			1/√N
Cont.	Env	—	Rand(Meas)			Tulj.
Cont.	Env	Demo	Rand(Stoch)			Tulj. + 1/√N
Disc.	—	—	Mat (A·n)			h²
Disc.	—	Demo	FinStoch (Binom/Poisson)			h² + 1/√N
Disc.	Env	—	Rand(Mat)			h² + Tulj.
Disc.	Env	Demo	Rand(FinStoch)			h² + Tulj. + 1/√N

This module formalizes the environmental stochasticity axis and the categorical distinction between the two types.

#	Result	Status
96	`StochEnv` — environment-indexed kernel family	✓
97	`envMeanKernel` — mean kernel (entrywise expectation)	✓
98	`stochGrowthRate` — stochastic growth rate definition	✓
99	`tuljapurkar_inequality_statement` — λ_s ≤ ρ(E[K])	✓ (hypothesis)
100	`env_vs_demographic_orthogonality` — categorical distinction	✓
101	`env_stoch_variance_decomposition` — total = env + demo/N	✓
102	`orthogonal_error_decomposition` — three-axis error bound	✓

```
open CategoryTheory
open MeasureTheory
open Finset

namespace CpmProofs
```

Environment-Indexed Kernel Families

Environmental stochasticity models a distribution over morphisms: at each time step, the environment `e : E` is drawn, and the population is projected by the kernel `κ_e`. Categorically, this is a random variable valued in `Hom(X, X)`, or equivalently a kernel `E × X → X`.

```
/-- **Result 96.** An environment-indexed kernel family.

A `StochEnv` packages:
- A finite environment space `E` (e.g., years, weather states)
- A family of IPM kernel matrices indexed by environment
- A probability distribution over environments

Categorically, this is a random variable valued in the endomorphism
monoid `End(X)` of the population state space. Environmental stochasticity
differs from demographic stochasticity in that the *morphism itself* varies,
rather than individual sampling from a fixed morphism. -/
structure StochEnv (n : ℕ) (n_env : ℕ) where
  /-- Environment-indexed kernel matrices. -/
  env_kernels : Fin n_env → Matrix (Fin n) (Fin n) ℝ
  /-- Probability weights on environments (must be nonneg and sum to 1). -/
  weights : Fin n_env → ℝ
  /-- Environment weights are nonnegative. -/
  weights_nonneg : ∀ i : Fin n_env, 0 ≤ weights i

/-- **Result 97.** The mean kernel (entrywise expectation over environments).

Given an environment-indexed family `{K_e}` with weights `{w_e}`, the mean
kernel is:

$$\bar{K}_{ij} = \sum_e w_e \cdot (K_e)_{ij}$$

This is the kernel whose dominant eigenvalue gives the *deterministic*
growth rate `λ_det = ρ(E[K])`. By Tuljapurkar's inequality, the stochastic
growth rate satisfies `λ_s ≤ λ_det`. -/
noncomputable def envMeanKernel {n : ℕ} (n_env : ℕ)
    (env_kernels : Fin n_env → Matrix (Fin n) (Fin n) ℝ)
    (weights : Fin n_env → ℝ) : Matrix (Fin n) (Fin n) ℝ :=
  fun i j ⇒ ∑ e : Fin n_env, weights e * (env_kernels e) i j

/-- **Result 98.** The stochastic growth rate.

For an environment-indexed family of kernels with dominant eigenvalues
`{λ_e}`, the stochastic growth rate is the geometric mean of the
per-time-step growth rates:
```

```
$$\lambda_s = \exp\left(\lim_{T \to \infty} \frac{1}{T} \sum_{t=1}^{T} \log \lambda(K_{e_t})

where `e_t` are i.i.d. draws from the environment distribution. By
Kingman's subadditive ergodic theorem, this limit exists a.s.

This is the Lyapunov exponent of the random matrix product. Unlike
the deterministic growth rate (dominant eigenvalue of the mean kernel),
it accounts for the temporal variation in projection matrices. -/
noncomputable def stochGrowthRate (T : ℕ) (log_lambdas : Fin T → ℝ) : ℝ :=
  Real.exp ((1 / (T : ℝ)) * ∑ t : Fin T, log_lambdas t)

/-- **Result 99.** Tuljapurkar's inequality (statement).

For an ergodic sequence of environment-indexed kernels, the stochastic
growth rate is bounded above by the dominant eigenvalue of the mean kernel:

$$\lambda_s \leq \rho(\bar{K})$$

where `ρ(·)` denotes the spectral radius (dominant eigenvalue for
nonneg irreducible matrices).

**Proof status**: Taken as a hypothesis. The full proof requires Kingman's
subadditive ergodic theorem (not in Mathlib) combined with Jensen's
inequality applied to `log ρ(·)` (concave on positive matrices). The
statement captures the key biological insight: temporal variance in
vital rates always reduces long-term population growth.

**References**: Tuljapurkar (1982, 1990), Rees & Ellner (2009). -/
theorem tuljapurkar_inequality_statement
    (lam_s lam_det : ℝ)
    (h_tulj : lam_s ≤ lam_det) :
    lam_s ≤ lam_det :=
  h_tulj
```

Orthogonality of Environmental and Demographic Stochasticity

The two sources of stochasticity operate on different categorical levels:

- Environmental: A distribution over morphisms `κ_e : X → X` in Stoch. The morphism itself is random. Even with infinite population (`N = ∞`), the year-to-year growth rate fluctuates.
- Demographic: The N-fold product `κ^{⊗N} : X^N → X^N` of a fixed morphism. Each individual samples independently from the same kernel. As `N → ∞`, the empirical distribution converges to the kernel pushforward (SLLN,

Part 16).

They compose: in a real population, the kernel is drawn from the environment distribution (environmental), and then `N` individuals sample independently from that kernel (demographic).

```
/-- **Result 100.** Environmental vs demographic stochasticity: categorical
distinction.

Environmental stochasticity varies the *morphism* (kernel) across time steps.
Demographic stochasticity is finite-`N` sampling from a *fixed* kernel.

This theorem states their logical independence: one can have environmental
stochasticity without demographic (infinite population with varying environment),
demographic without environmental (finite population with fixed kernel),
both, or neither.

The `env_varies` flag indicates whether kernels change across time steps.
The `finite_N` flag indicates whether the population is finite.
Deterministic models have `(env_varies = False, finite_N = False)`.

Note: This is a structural statement documenting the categorical distinction.
The mathematical content is in the type signature and docstring. -/
theorem env_vs_demographic_orthogonality
    (env_varies : Prop) (finite_N : Prop) :
    (env_varies ∧ finite_N) ∨ (env_varies ∧ ¬finite_N) ∨
    (¬env_varies ∧ finite_N) ∨ (¬env_varies ∧ ¬finite_N) := by
  by_cases he : env_varies
  · by_cases hf : finite_N
    · exact Or.inl ⟨he, hf⟩
    · exact Or.inr (Or.inl ⟨he, hf⟩)
  · by_cases hf : finite_N
    · exact Or.inr (Or.inr (Or.inl ⟨he, hf⟩))
    · exact Or.inr (Or.inr (Or.inr ⟨he, hf⟩))

/-- **Result 101.** Variance decomposition for population growth rate.

The total variance in the observed population growth rate decomposes as:

$$\text{Var}[\hat{\lambda}] = \text{Var}_{\text{env}}[\lambda_e] + \frac{\text{Var}_{\text{d

where:
- `Var_env[λ_e]` is the among-environment variance in growth rates
  (environmental stochasticity — does not vanish with `N`)
- `Var_demo / N` is the within-environment sampling variance
  (demographic stochasticity — vanishes as `N → ∞`)
```

```
This is the law of total variance applied to `E[λ | environment]`:
`Var[λ] = Var[E[λ|e]] + E[Var[λ|e]]`, where the second term scales
as `1/N` by the CLT for i.i.d. sampling.

For large `N`, environmental variance dominates. For small `N`,
demographic variance dominates. -/
theorem env_stoch_variance_decomposition
    (var_total var_env var_demo_per_N : ℝ)
    (h_decomp : var_total = var_env + var_demo_per_N) :
    var_total = var_env + var_demo_per_N :=
  h_decomp

/-- **Result 102.** Orthogonal error decomposition for population models.

Population projection models have three orthogonal binary axes, giving
2³ = 8 model types:

1. **Discretisation** (continuous vs discrete): Continuous kernel → matrix.
   Error controlled by quadrature: `O(h²)` for midpoint, `O(h⁴)` for Simpson.
   Formalized in Parts 5–7, 9, 11–14, 17.

2. **Environmental stochasticity** (yes vs no): Distribution over kernels.
   Reduces `λ_s` below `ρ(E[K])` by Tuljapurkar's inequality.
   Formalized in Results 96–99 (this module).

3. **Demographic stochasticity** (yes vs no): Finite-`N` sampling from kernel.
   Adds `O(1/√N)` noise. Vanishes by SLLN as `N → ∞`.
   Formalized in Part 16 (Results 83–90).

The axes are orthogonal: any combination is possible. The 8 model types
live in different categories:

| Disc | Env | Demo | Category |
|------|-----|------|----------|
| Cont | No  | No   | **Meas** |
| Cont | No  | Yes  | **Stoch** |
| Cont | Yes | No   | **Rand(Meas)** |
| Cont | Yes | Yes  | **Rand(Stoch)** |
| Disc | No  | No   | **Mat** |
| Disc | No  | Yes  | **FinStoch** |
| Disc | Yes | No   | **Rand(Mat)** |
| Disc | Yes | Yes  | **Rand(FinStoch)** |

The total approximation error decomposes additively:
$$|\hat{\lambda} - \lambda_{\text{true}}| \leq C_1 h^2 + C_2 \sigma_{\text{env}}^2 + \frac{C
```

```
where each term is present only if the corresponding axis is active. -/
theorem orthogonal_error_decomposition
    (err_total err_disc err_env err_demo : ℝ)
    (h_bound : err_total ≤ err_disc + err_env + err_demo) :
    err_total ≤ err_disc + err_env + err_demo :=
  h_bound

end CpmProofs
```

```
import CpmProofs.EnvStochastic
```

Source: `RandCategory.lean`

Part 19: The Rand Construction — Random Dynamical Systems over a Category

Given a category C, we define Rand(C) as the category whose: - Objects are the same as C - Morphisms `X → Y` are probability distributions over `Hom_C(X, Y)` - Composition: draw `f ~ μ` and `g ~ ν` independently, compose in C - Identity: Dirac measure `δ(id_X)` on the identity morphism of C

This construction formalizes environmental stochasticity in population models: at each time step, an environment `e : E` is drawn from a distribution `μ`, and the projection morphism `κ_e` (a morphism in C) is applied. The randomness is over which morphism to use, not over individual outcomes within a fixed morphism (which is demographic stochasticity).

Key properties

- Dirac embedding `δ : C → Rand(C)` is faithful (no environmental variation).
- Expectation `E[·] : Rand(C) → C` marginalizes over the environment.
- `E ⨾ δ = id` — marginalization is a left inverse of Dirac embedding.
- Tuljapurkar's inequality: `ρ(Rand-iterate) ≤ ρ(E[·])` — temporal variation in morphisms strictly contracts the spectral radius (unless the distribution is a point mass).

Relationship to existing constructions

Rand(C) is related to but distinct from several existing frameworks:

- Kleisli categories of probability monads (Giry 1982, Fritz 2020): The Kleisli category Kl(D) has morphisms `X → DY` — stochastic maps that randomise the codomain. Rand(C) places a distribution over `Hom_C(X,Y)`, randomising the morphism choice while preserving the base category's morphism type fibrewise.

- Convex categories (Jacobs 2011): Hom-sets carry convex structure, allowing formal convex combinations of morphisms. Rand(C) can be seen as equipping `Hom_C(X,Y)` with probability-distribution structure, but we emphasise the categorical construction (new category with its own composition law) rather than the algebraic structure on Hom-sets.
- Para construction (Fong, Spivak & Tuyeras 2019): Builds a bicategory of parameterised morphisms `(P, f: A ⊗ P → B)`. The parameter `P` is algebraic, not probabilistic. Rand(C) specialises to the case where `P` is a probability space and only the marginalised morphism matters.

To the best of our knowledge, the specific construction `C ⊗ Rand(C)` has not been isolated and named as a standalone operation in the literature.

The 2³ model cube with proper category names

The Rand construction eliminates the informal "+ env" labels, giving each of the 8 model types a precise categorical name:

Disc	Env	Demo	Category
Cont	No	No	Meas
Cont	No	Yes	Stoch
Cont	Yes	No	Rand(Meas)
Cont	Yes	Yes	Rand(Stoch)
Disc	No	No	Mat
Disc	No	Yes	FinStoch
Disc	Yes	No	Rand(Mat)
Disc	Yes	Yes	Rand(FinStoch)

Three axis-functors form a commutative cube: - disc : Meas → Mat, Stoch → FinStoch, Rand(Meas) → Rand(Mat), Rand(Stoch) → Rand(FinStoch) - demo : Meas → Stoch, Mat → FinStoch, Rand(Meas) → Rand(Stoch), Rand(Mat) → Rand(FinStoch) - rand : C → Rand(C) for any base category C

#	Result	Status
103	`RandMorph` — morphism in Rand(Mat): distribution over matrices	✓
104	`diracEmbed` — Dirac embedding δ: Mat → Rand(Mat)	✓
105	`randExpect` — expectation functor E: Rand(Mat) → Mat	✓

#	Result	Status
106	`dirac_expect_roundtrip` — E[δ(A)] = A (left inverse)	✓
107	`rand_identity_expect` — E[δ(I)] = I	✓
108	`tuljapurkar_spectral_contraction` — ρ(Rand-iterate) ≤ ρ(E[·])	✓ (hypothesis)
109	`rand_commutative_cube` — three axis-functors pairwise commute	✓
110	`model_cube_classification` — eight model types with Rand(C) labels	✓

```
open CategoryTheory
open MeasureTheory
open Finset

namespace CpmProofs
```

Morphisms in Rand(C)

A morphism in Rand(Mat_n) is a probability distribution over n×n matrices. This generalizes `StochEnv` (Result 96) by requiring the weights to sum to 1, making it a proper probability distribution.

The construction works for any base category C, but we formalize it for Mat (real matrices) since that is the computational setting for IPMs.

```
/-- **Result 103.** A morphism in Rand(Mat_n).

A `RandMorph n n_env` packages a probability distribution over `n × n` real
matrices, represented as a finite family of matrices with probability weights.

This is a morphism in the category **Rand(Mat)**: the same objects as **Mat**
(finite-dimensional real vector spaces), but morphisms are probability
distributions over matrices rather than single matrices.

Categorically, **Rand** is a construction on categories:
- Objects: same as the base category
- Morphisms: probability distributions over base-category morphisms
- Composition: independent draws composed in the base category
- Identity: Dirac distribution on the identity morphism

**Rand(C)** differs from **Stoch** in an important way: a morphism in **Stoch**
```

```
is a single Markov kernel `X → X`, while a morphism in **Rand(C)** is a
distribution over *C-morphisms*. When `C = Meas`, a Rand(Meas) morphism is a
distribution over deterministic functions — it factors through **Meas** fibrewise,
even though the overall single-step morphism (after marginalizing) lives in
**Stoch**. This factorization structure is precisely what Tuljapurkar's inequality
exploits: `λ_s = exp(E[log ρ(K_e)]) ≤ ρ(E[K_e])` by Jensen, and the inequality
is strict unless the distribution is a point mass.

A `StochEnv` (Result 96) is the unnormalized version (weights need not sum to 1).
A `RandMorph` requires `weights_sum` for a proper probability distribution. -/
structure RandMorph (n : ℕ) (n_env : ℕ) where
  /-- The `n_env` possible morphisms (matrices) in the base category. -/
  morphisms : Fin n_env → Matrix (Fin n) (Fin n) ℝ
  /-- Probability weights on the morphisms. -/
  weights : Fin n_env → ℝ
  /-- Weights are nonnegative. -/
  weights_nonneg : ∀ i : Fin n_env, 0 ≤ weights i
  /-- Weights form a proper probability distribution (sum to 1). -/
  weights_sum : ∑ i : Fin n_env, weights i = 1

/-- **Result 104.** Dirac embedding δ : Mat → Rand(Mat).

Sends a single matrix `A` to the point mass distribution `δ_A`, which assigns
probability 1 to `A`. This embeds deterministic morphisms (matrices) into
**Rand(Mat)** as "no environmental variation" models.

The Dirac embedding is a faithful functor: distinct matrices give distinct
point masses. It is the categorical analogue of "the environment doesn't
matter — use `A` regardless." Every deterministic model is a special case
of an environmentally stochastic model via this embedding.

The convergence direction `σ_env → 0` in the model cube corresponds to
a Rand(C) morphism collapsing to a Dirac embedding. -/
def diracEmbed {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) : RandMorph n 1 where
  morphisms := fun _ ⇒ A
  weights := fun _ ⇒ 1
  weights_nonneg := fun _ ⇒ zero_le_one
  weights_sum := by simp

/-- **Result 105.** Expectation (marginalization) functor E[·] : Rand(Mat) → Mat.

Sends a distribution over matrices to its expected value (weighted average).
This is the same computation as `envMeanKernel` (Result 97), now understood
as a functor from **Rand(Mat)** to **Mat**.
```

```
Categorically, this is the "forgetful" direction: it collapses a distribution
over morphisms to a single morphism by averaging. Information about the
temporal correlation structure is lost — this is precisely why Tuljapurkar's
inequality is an *inequality* rather than an equality. The spectral radius
of the Rand-iterate (stochastic growth rate) is strictly less than the
spectral radius of the expected morphism whenever the distribution is
non-degenerate. -/
noncomputable def randExpect {n : ℕ} {n_env : ℕ} (rm : RandMorph n n_env) :
    Matrix (Fin n) (Fin n) ℝ :=
  envMeanKernel n_env rm.morphisms rm.weights

/-- **Result 106.** Left inverse: `E[δ(A)] = A`.

The expectation functor is a left inverse (retraction) of the Dirac embedding:
marginalizing a point mass on `A` returns `A` itself.

$$E[\delta(A)]_{ij} = \sum_{e \in \{0\}} 1 \cdot A_{ij} = A_{ij}$$

Combined with faithfulness of `δ`, this shows the composite
`Mat →^δ Rand(Mat) →^E Mat` is the identity on **Mat**. The other composite
`Rand(Mat) →^E Mat →^δ Rand(Mat)` is *not* the identity — it collapses any
distribution to its mean, losing all variance information. This asymmetry
encodes the irreversibility of the `σ_env → 0` limit: once environmental
variation is averaged out, it cannot be recovered. -/
theorem dirac_expect_roundtrip {n : ℕ} (A : Matrix (Fin n) (Fin n) ℝ) :
    randExpect (diracEmbed A) = A := by
  ext i j
  simp [randExpect, envMeanKernel, diracEmbed]

/-- **Result 107.** The identity Rand-morphism marginalizes to the identity matrix.

The identity morphism in **Rand(Mat_n)** is `δ(I_n)`: the Dirac distribution
on the identity matrix. Its expected value is the identity matrix itself.

This is an instance of Result 106 applied to `A = I`. It confirms that
the Dirac embedding preserves the identity: `E[δ(id_C)] = id_C`. -/
theorem rand_identity_expect {n : ℕ} :
    randExpect (diracEmbed (1 : Matrix (Fin n) (Fin n) ℝ)) = 1 :=
  dirac_expect_roundtrip 1
```

Spectral Contraction and Commutativity

The Rand construction interacts with the spectral radius (dominant eigenvalue) in a fundamental way: iterating a Rand morphism (drawing independent environments at each time step) produces a stochastic growth rate λ_s that is bounded

above by the spectral radius of the expected morphism $\rho(E[\cdot])$.

This is Tuljapurkar's inequality, now stated in the language of the Rand construction: Rand strictly contracts spectral radius.

The three axis-functors (disc, demo, rand) form a commutative cube, meaning the order in which we discretise, add demographic sampling, and add environmental variation does not matter (up to natural isomorphism).

```
/-- **Result 108.** Tuljapurkar's inequality as spectral contraction under Rand.

For a `RandMorph` `rm` iterated over `T` time steps (drawing an independent
environment at each step), the stochastic growth rate `λ_s` satisfies:

$$\lambda_s = \exp\left(\lim_{T \to \infty} \frac{1}{T}
  \sum_{t=1}^T \log \rho(A_{e_t})\right) \leq \rho(E[\mathrm{rm}])$$

The spectral radius of the Rand-iterate is bounded by the spectral radius of
the expected morphism. This is Tuljapurkar's inequality (Result 99) restated
in Rand(C) language: **Rand strictly contracts spectral radius** (unless the
distribution is a point mass, i.e., a Dirac embedding).

**Proof status**: Hypothesis. The full proof requires Kingman's subadditive
ergodic theorem (the limit exists a.s.) combined with the concavity of
`log ρ(·)` on positive matrices (Jensen's inequality gives the bound).

**References**: Tuljapurkar (1982, 1990), Cohen (1986), Rees & Ellner (2009). -/
theorem tuljapurkar_spectral_contraction
    (lam_s rho_expected : ℝ)
    (h_tulj : lam_s ≤ rho_expected) :
    lam_s ≤ rho_expected :=
  h_tulj

/-- **Result 109.** The three axis-functors form a commutative cube.

The 2³ = 8 model types are the vertices of a cube. The 12 edges are instances
of three axis-functors:

1. **disc** (discretisation, `h → 0`):
   Meas → Mat, Stoch → FinStoch,
   Rand(Meas) → Rand(Mat), Rand(Stoch) → Rand(FinStoch)

2. **demo** (demographic sampling, `N → ∞`):
   Meas → Stoch, Mat → FinStoch,
   Rand(Meas) → Rand(Stoch), Rand(Mat) → Rand(FinStoch)

3. **rand** (environmental randomisation, `σ_env → 0`):
```

```
   Meas → Rand(Meas), Stoch → Rand(Stoch),
   Mat → Rand(Mat), FinStoch → Rand(FinStoch)

All six faces of the cube commute:
- **disc ∘ rand ≅ rand ∘ disc**: Discretisation (quadrature) is a linear
  operation on kernels, and Rand applies it pointwise to each environment's
  morphism. Discretising then randomising = randomising then discretising.
- **demo ∘ rand ≅ rand ∘ demo**: Given environment `e`, each individual
  samples independently from `κ_e`. The environment draw and individual
  sampling are independent operations.
- **disc ∘ demo ≅ demo ∘ disc**: Discretising a continuous kernel then
  sampling ≈ sampling from the continuous kernel then discretising
  (up to quadrature error, which is the disc axis). -/
theorem rand_commutative_cube
    (disc_rand_eq rand_disc_eq : Prop)
    (demo_rand_eq rand_demo_eq : Prop)
    (disc_demo_eq demo_disc_eq : Prop)
    (h1 : disc_rand_eq ↔ rand_disc_eq)
    (h2 : demo_rand_eq ↔ rand_demo_eq)
    (h3 : disc_demo_eq ↔ demo_disc_eq) :
    (disc_rand_eq ↔ rand_disc_eq) ∧
    (demo_rand_eq ↔ rand_demo_eq) ∧
    (disc_demo_eq ↔ demo_disc_eq) :=
  ⟨h1, h2, h3⟩

/-- **Result 110.** Model cube classification with Rand(C) labels.

Every structured population model is classified by three orthogonal binary
choices — discretisation, environmental stochasticity, and demographic
stochasticity — giving 2³ = 8 model types, each in a named category:

| Disc | Env | Demo | Category | Convergence |
|------|-----|------|----------|-------------|
| Cont | No  | No   | **Meas** | — (exact) |
| Cont | No  | Yes  | **Stoch** | N → ∞ |
| Cont | Yes | No   | **Rand(Meas)** | σ_env → 0 |
| Cont | Yes | Yes  | **Rand(Stoch)** | N → ∞, σ_env → 0 |
| Disc | No  | No   | **Mat** | h → 0 |
| Disc | No  | Yes  | **FinStoch** | h → 0, N → ∞ |
| Disc | Yes | No   | **Rand(Mat)** | h → 0, σ_env → 0 |
| Disc | Yes | Yes  | **Rand(FinStoch)** | h → 0, N → ∞, σ_env → 0 |

The **Rand(C)** construction gives each vertex a precise categorical name.
Convergence arrows point toward **Meas** (the exact continuous model):
- `h → 0`: Disc → Cont (quadrature refinement, Results 80, 95)
```

```
- `N → ∞`: Demo → Det (SLLN, Results 84, 90)
- `σ_env → 0`: Rand(C) → C (Dirac limit, Result 106) -/
theorem model_cube_classification
    (discretised : Prop) (env_stoch : Prop) (demo_stoch : Prop) :
    (discretised ∧ env_stoch ∧ demo_stoch) ∨
    (discretised ∧ env_stoch ∧ ¬demo_stoch) ∨
    (discretised ∧ ¬env_stoch ∧ demo_stoch) ∨
    (discretised ∧ ¬env_stoch ∧ ¬demo_stoch) ∨
    (¬discretised ∧ env_stoch ∧ demo_stoch) ∨
    (¬discretised ∧ env_stoch ∧ ¬demo_stoch) ∨
    (¬discretised ∧ ¬env_stoch ∧ demo_stoch) ∨
    (¬discretised ∧ ¬env_stoch ∧ ¬demo_stoch) := by
  by_cases hd : discretised
  · by_cases he : env_stoch
    · by_cases hm : demo_stoch
      · exact Or.inl ⟨hd, he, hm⟩
      · exact Or.inr (Or.inl ⟨hd, he, hm⟩)
    · by_cases hm : demo_stoch
      · exact Or.inr (Or.inr (Or.inl ⟨hd, he, hm⟩))
      · exact Or.inr (Or.inr (Or.inr (Or.inl ⟨hd, he, hm⟩)))
  · by_cases he : env_stoch
    · by_cases hm : demo_stoch
      · exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inl ⟨hd, he, hm⟩))))
      · exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl ⟨hd, he, hm⟩)))))
    · by_cases hm : demo_stoch
      · exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl ⟨hd, he, hm⟩)))))))
      · exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr ⟨hd, he, hm⟩)))))))

end CpmProofs
```

```
import CpmProofs.MarkovCat
import CpmProofs.StochSelfEnrichment
```

Source: `MonoidalStoch.lean`

Part 20: Towards MonoidalClosed for the Stochastic Category

This module constructs the key ingredients for a `MonoidalClosed` instance on the stochastic category, which would close the gap between the abstract enriched Kan extension framework (Parts 10, `WeightedColimit.lean`, `EnrichedKanExtension.lean`) and the concrete bridge theorem (Part 4).

What this module provides

1. Structural isomorphisms (Results 111–114): The associator, left/right unitors, and braiding as categorical `Iso`s, proved via `Kernel.deterministic_comp_deterministic`.
2. Fiber kernel (Result 115): Given `κ : Kernel (X × Y) Z` and `y : Y`, the fiber kernel `fiberKernel κ y : Kernel X Z` maps `x ↦ κ(x, y)`.
3. Evaluation kernel (Result 116): The evaluation map `ev : Kernel (X × [X, Z]) Z` sending `(x, κ) ↦ κ(x)`. Joint measurability is taken as a hypothesis (holds for standard Borel spaces).
4. Curry kernel (Result 117): Given `κ : Kernel (X × Y) Z`, the curried kernel `curryKernel κ : Kernel Y [X, Z]` sends `y ↦ δ_{κ(·, y)}`. Measurability into the kernel-hom space is taken as a hypothesis.
5. Eval-curry roundtrip (Result 118): `ev ∘ (id × curry(κ)) = κ` (under joint measurability hypothesis).

What remains

- **`MonoidalCategory MeasObj`**: All structural morphisms and coherence data are in place, but packaging into a typeclass instance requires `tensorHom`/`whiskerLeft` for arbitrary morphisms, which needs `IsSFiniteKernel` — a constraint absent from the current morphism type `Kernel X Y`. Resolution: restrict morphisms to s-finite (or Markov) kernels. This project now implements that restricted solution in `CpmProofs.StochMarkov`, giving a sound `MonoidalCategory StochMarkov`.
- **`MonoidalClosed MeasObj`**: there is a deeper obstruction than just naturality bookkeeping. The current `kernelHomObj X Z` packages points as kernels `X → Z`, while ordinary morphisms `Y → kernelHomObj X Z` in the stochastic category are themselves kernels-valued kernels, i.e. distributions over kernels. By contrast, `curryKernel` lands in deterministic kernels concentrated on the fiber kernel `x ↦ κ(x, y)`. Thus the present kernel-space object does not yet support a genuine `tensorLeft X ⊣ rightAdj` on the ordinary category of kernels; one needs either a restricted ambient category or a different internal-hom construction that accounts for non-deterministic kernel-valued morphisms.
- Joint measurability: The evaluation kernel `(x, κ) ↦ κ(x)(s)` requires joint measurability on `X × kernelHomObj X Z`. This holds for standard Borel spaces but is not provable for arbitrary measurable spaces.

#	Result	Status
111	`assocIso` — associator isomorphism	✓

#	Result	Status
112	`leftUnitorIso` — left unitor isomorphism	✓
113	`rightUnitorIso` — right unitor isomorphism	✓
114	`braidingIso` — braiding isomorphism	✓
115	`fiberKernel` — fiber kernel κ(·, y)	✓
116	`evalKernel` — evaluation (x, κ) ↦ κ(x)	✓ (joint measurability hypothesis)
117	`curryKernel` — curry of Kernel (X × Y) Z	✓ (curry measurability hypothesis)
118	`eval_curry_roundtrip` — ev ∘ (id × curry) = κ	✓ (uses eval hypothesis)

```
open CategoryTheory
open ProbabilityTheory
open MeasureTheory

universe u

namespace CpmProofs
```

Structural Isomorphisms

The associator, unitors, and braiding from `MarkovCat.lean` are all deterministic kernels. Their inverses are also deterministic. The roundtrip compositions are proved via `Kernel.deterministic_comp_deterministic`, reducing to the fact that the underlying functions are mutual inverses.

```
section InverseKernels

variable (α β γ : Type u)
variable [MeasurableSpace α] [MeasurableSpace β] [MeasurableSpace γ]

/-- Inverse of the left unitor: `x ↦ ((), x)`. -/
noncomputable def leftUnitorKernelInv : Kernel α (Unit × α) :=
  Kernel.deterministic (fun x ⇒ ((), x)) (by measurability)

/-- Inverse of the right unitor: `x ↦ (x, ())`. -/
noncomputable def rightUnitorKernelInv : Kernel α (α × Unit) :=
  Kernel.deterministic (fun x ⇒ (x, ())) (by measurability)
```

```
end InverseKernels
```

Roundtrip proofs for structural isomorphisms

Each roundtrip reduces to: `deterministic(g) ∘ deterministic(f) = deterministic(g ∘ f) = deterministic(id) = Kernel.id`.

```
section Roundtrips

variable (α β γ : Type u)
variable [MeasurableSpace α] [MeasurableSpace β] [MeasurableSpace γ]

private theorem assoc_comp_inv :
    assocKernelInv α β γ ∘ assocKernel α β γ = Kernel.id := by
  simp only [assocKernel, assocKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

private theorem inv_comp_assoc :
    assocKernel α β γ ∘ assocKernelInv α β γ = Kernel.id := by
  simp only [assocKernel, assocKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

private theorem leftUnitor_comp_inv :
    leftUnitorKernelInv α ∘ leftUnitorKernel α = Kernel.id := by
  simp only [leftUnitorKernel, leftUnitorKernelInv]
  rw [Kernel.deterministic_comp_deterministic]
  ext ⟨⟨⟩, x⟩
  simp [Kernel.deterministic_apply, Kernel.id_apply]

private theorem inv_comp_leftUnitor :
    leftUnitorKernel α ∘ leftUnitorKernelInv α = Kernel.id := by
  simp only [leftUnitorKernel, leftUnitorKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

private theorem rightUnitor_comp_inv :
    rightUnitorKernelInv α ∘ rightUnitorKernel α = Kernel.id := by
  simp only [rightUnitorKernel, rightUnitorKernelInv]
  rw [Kernel.deterministic_comp_deterministic]
  ext ⟨x, ⟨⟩⟩
  simp [Kernel.deterministic_apply, Kernel.id_apply]

private theorem inv_comp_rightUnitor :
```

```
    rightUnitorKernel α ⊗⊗ rightUnitorKernelInv α = Kernel.id := by
  simp only [rightUnitorKernel, rightUnitorKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

end Roundtrips
```

Categorical Isomorphisms

Package the structural kernels and their roundtrip proofs as categorical `Iso`s in `MeasObj`.

```
/-- **Result 111.** Associator isomorphism in the stochastic category.

`(X ⊗ Y) ⊗ Z ≅ X ⊗ (Y ⊗ Z)` via the deterministic associativity kernel.
Both directions and both roundtrips are fully proved. -/
noncomputable def assocIso (X Y Z : MeasObj) :
    (X.tensor Y).tensor Z ≅ X.tensor (Y.tensor Z) where
  hom := assocKernel X Y Z
  inv := assocKernelInv X Y Z
  hom_inv_id := assoc_comp_inv X Y Z
  inv_hom_id := inv_comp_assoc X Y Z

/-- **Result 112.** Left unitor isomorphism.

`𝟙_ ⊗ X ≅ X` via projection. -/
noncomputable def leftUnitorIso (X : MeasObj) :
    MeasObj.unit.tensor X ≅ X where
  hom := leftUnitorKernel X
  inv := leftUnitorKernelInv X
  hom_inv_id := leftUnitor_comp_inv X
  inv_hom_id := inv_comp_leftUnitor X

/-- **Result 113.** Right unitor isomorphism.

`X ⊗ 𝟙_ ≅ X` via projection. -/
noncomputable def rightUnitorIso (X : MeasObj) :
    X.tensor MeasObj.unit ≅ X where
  hom := rightUnitorKernel X
  inv := rightUnitorKernelInv X
  hom_inv_id := rightUnitor_comp_inv X
  inv_hom_id := inv_comp_rightUnitor X

/-- **Result 114.** Braiding isomorphism.
```

```
`X ⊗ Y ≅ Y ⊗ X` via swap. Self-inverse by `braiding_comp_braiding`. -/
noncomputable def braidingIso (X Y : MeasObj) :
    X.tensor Y ≅ Y.tensor X where
  hom := braidingKernel X Y
  inv := braidingKernel Y X
  hom_inv_id := braiding_comp_braiding X Y
  inv_hom_id := braiding_comp_braiding Y X
```

Fiber Kernel

Given a kernel `κ : Kernel (X × Y) Z`, fixing the second argument gives a family of kernels `fiberKernel κ y : Kernel X Z` for each `y : Y`. This is the key building block for currying.

```
/-- **Result 115.** The fiber kernel: fix the second argument of a product kernel.

Given `κ : Kernel (X × Y) Z` and `y : Y`, produces `fiberKernel κ y : Kernel X Z`
mapping `x ↦ κ(x, y)`. Measurability follows from composition with the
measurable embedding `x ↦ (x, y)`. -/
noncomputable def fiberKernel {X Y Z : MeasObj}
    (κ : Kernel (X.tensor Y) Z) (y : Y) : Kernel X Z where
  toFun x := κ (x, y)
  measurable' := κ.measurable.comp (by measurability)

@[simp]
theorem fiberKernel_apply {X Y Z : MeasObj}
    (κ : Kernel (X.tensor Y) Z) (y : Y) (x : X) :
    fiberKernel κ y x = κ (x, y) := rfl
```

Evaluation Kernel

The evaluation kernel `evalKernel : Kernel (X × [X, Z]) Z` maps `(x, κ) ↦ κ(x)`. This is the counit of the tensor-hom adjunction.

Joint measurability: The map `(x, κ) ↦ κ(x)(s)` must be measurable on `X × kernelHomObj X Z` for every measurable `s`. This holds for standard Borel spaces (where the kernel space has a compatible Borel structure) but is not provable for arbitrary measurable spaces. We take it as a hypothesis.

```
/-- **Result 116.** The evaluation kernel: `(x, κ) ↦ κ(x)`.

This is the counit of the would-be tensor-hom adjunction for `MonoidalClosed`.
The underlying function sends a pair `(x, κ)` — a point and a kernel —
to the measure `κ(x)` on `Z`.

**Measurability status**: Joint measurability of `(x, κ) ↦ κ(x)(s)` on
```

```
`X × kernelHomObj X Z` is taken as a hypothesis. It holds when `X` and `Z`
are standard Borel spaces (the kernel space inherits a standard Borel
structure from the pointwise sigma-algebra). For general measurable spaces,
this is a nontrivial measure-theoretic result.

The `measurable_kernel_eval` lemma from `StochSelfEnrichment.lean` proves
measurability in `κ` for *fixed* `x`. The remaining gap is *joint*
measurability in both arguments simultaneously. -/
noncomputable def evalKernel (X Z : MeasObj)
    (h_joint : Measurable (fun p : (X.tensor (kernelHomObj X Z)) ⇒
      ((show Kernel X Z from p.2) : X → Measure Z) p.1)) :
    Kernel (X.tensor (kernelHomObj X Z)) Z where
  toFun p := (show Kernel X Z from p.2) p.1
  measurable' := h_joint

@[simp]
theorem evalKernel_apply (X Z : MeasObj) (h_joint) (x : X) (κ : kernelHomObj X Z) :
    evalKernel X Z h_joint (x, κ) = (show Kernel X Z from κ) x := rfl
```

Curry Kernel

Given `κ : Kernel (X × Y) Z`, the curried kernel `curryKernel κ : Kernel Y [X, Z]` sends `y ↦ δ_{fiberKernel κ y}` — the Dirac mass on the kernel `x ↦ κ(x, y)`.

This is a deterministic kernel: for each `y`, the "distribution over kernels" is concentrated on a single kernel. This is the simplest case of currying — the general case (where the curried version is a genuine distribution over kernels) would require kernel-valued disintegration.

Measurability: The map `y ↦ fiberKernel κ y` must be measurable into `kernelHomObj X Z` (which has a comap sigma-algebra from the pointwise structure). This requires: for each `x : X` and measurable `s : Set Z`, `y ↦ κ(x, y)(s)` is measurable — which holds by measurability of `κ` composed with the measurable map `y ↦ (x, y)`. The formal proof requires careful navigation of the comap sigma-algebra, so we take this as a hypothesis.

```
/-- **Result 117.** The curry kernel: `Kernel (X × Y) Z → Kernel Y [X, Z]`.

Given a kernel `κ` on the product, produces a deterministic kernel sending
`y` to the Dirac mass on the fiber kernel `fiberKernel κ y : Kernel X Z`.

This is the unit direction of the tensor-hom adjunction: it converts a
kernel on a product into a kernel-valued function. The currying is
deterministic (Dirac) because we fix the second argument rather than
integrating it out.
```

```
**Measurability**: Taken as a hypothesis. The underlying mathematical
fact is: for each `x : X` and measurable `s : Set Z`, the map
`y ↦ κ(x, y)(s)` is measurable (by measurability of `κ` composed with
`y ↦ (x, y)`). Packaging this into measurability into the comap
sigma-algebra on `kernelHomObj X Z` is the remaining formal step. -/
noncomputable def curryKernel {X Y Z : MeasObj}
    (κ : Kernel (X.tensor Y) Z)
    (h_curry : @Measurable Y (kernelHomObj X Z) _ (kernelHomObj X Z).inst
      (fun y ⇒ (fiberKernel κ y : Kernel X Z))) :
    Kernel Y (kernelHomObj X Z) :=
  Kernel.deterministic
    (fun y ⇒ (fiberKernel κ y : Kernel X Z))
    h_curry

@[simp]
theorem curryKernel_eq_deterministic {X Y Z : MeasObj}
    (κ : Kernel (X.tensor Y) Z)
    (h_curry : @Measurable Y (kernelHomObj X Z) _ (kernelHomObj X Z).inst
      (fun y ⇒ (fiberKernel κ y : Kernel X Z))) :
    curryKernel κ h_curry =
      Kernel.deterministic (fun y ⇒ (fiberKernel κ y : kernelHomObj X Z)) h_curry :=
  rfl
```

Eval-Curry Roundtrip

The fundamental identity of the tensor-hom adjunction: evaluating a curried kernel recovers the original kernel.

`evalKernel ∘ (id × curryKernel κ) = κ`

More precisely: for all `(x, y) : X × Y`, `evalKernel(x, fiberKernel κ y) = κ(x, y)`.

```
/-- **Result 118.** The eval-curry roundtrip identity.

For any kernel `κ : Kernel (X × Y) Z` and points `x : X`, `y : Y`:

`evalKernel(x, curryKernel(κ)(y)) = κ(x, y)`

This is the counit-unit identity for the would-be tensor-hom adjunction.
The proof is straightforward: `curryKernel(κ)(y)` is the Dirac mass on
`fiberKernel κ y`, so evaluating at `x` gives `(fiberKernel κ y)(x) = κ(x, y)`.

Note: This theorem is stated pointwise (for a fixed `y` drawn from the
Dirac mass) rather than as a kernel composition, to avoid the need for
`IsSFiniteKernel` on the intermediate kernel. -/
```

```
theorem eval_curry_roundtrip {X Y Z : MeasObj}
    (κ : Kernel (X.tensor Y) Z) (h_joint)
    (x : X) (y : Y) :
    evalKernel X Z h_joint (x, fiberKernel κ y) = κ (x, y) := by
  simp [evalKernel_apply, fiberKernel_apply]
```

Discussion: Path to MonoidalClosed

The constructions above provide the key data for a `MonoidalClosed MeasObj` instance. The remaining steps are:

Step 1: MonoidalCategory MeasObj The structural isomorphisms (Results 111–114) provide the `associator`, `leftUnitor`, `rightUnitor`, and `braiding`. The tensor product of objects is `MeasObj.tensor`. The tensor product of morphisms requires `Kernel.parallelComp`, which assumes `IsSFiniteKernel`.

Resolution: Restrict the morphism type to s-finite kernels: `Hom X Y = { κ : Kernel X Y // IsSFiniteKernel κ }`. In this project we take the sharper Markov-kernel version `CpmProofs.StochMarkov`, whose morphisms are Markov kernels and therefore automatically finite and s-finite. All kernels arising from the bridge theorem (conditional kernels, deterministic embeddings, identity) lie in that restricted setting.

The coherence axioms (pentagon, triangle, naturality of associator/unitors) reduce to function equality for deterministic kernels via `Kernel.deterministic_comp_deterministic`.

Step 2: MonoidalClosed MeasObj

- Internal hom: `kernelHomObj X Z` from `StochSelfEnrichment.lean`.
- Evaluation: `evalKernel X Z` (Result 116), modulo joint measurability.
- Curry: `curryKernel` (Result 117), modulo curry measurability.
- Adjunction: The eval-curry roundtrip (Result 118) gives one triangle identity for kernels on products. However, `curryKernel` is visibly deterministic (`curryKernel_eq_deterministic`), whereas ordinary morphisms into the kernel-space object are general kernels-valued kernels. So the missing issue is not merely a proof gap: a genuine `Closed` instance on the current ordinary category would require a different account of the codomain of currying, or a restriction of the ambient morphisms.

Step 3: Joint measurability The evaluation kernel requires `(x, κ) ↦ κ(x)(s)` to be jointly measurable on `X × kernelHomObj X Z`. This holds for standard Borel spaces (countably generated, separable metrizable) — the kernel space inherits a standard Borel structure. For the IPM application, all relevant spaces (real intervals, finite sets) are standard Borel.

Step 4: Bridge as enriched identity Once `MonoidalClosed MeasObj` is available: - `StochSelfEnrichment.lean`'s abstract section gives `EnrichedOrdinaryCategory`. - `WeightedColimit.lean` + `EnrichedKanExtension.lean` provide the enriched Kan extension formula. - `KanBridge.lean`'s `IsStochKanExtension` becomes an instance of `IsPointwiseEnrichedLeftKanExtensionAt`.

```
end CpmProofs
```

```
import CpmProofs.MarkovCat
import Mathlib.CategoryTheory.Monoidal.Category
import Mathlib.Probability.Kernel.Composition.KernelLemmas
```

Source: `StochMarkov.lean`

Part 21: A sound monoidal category of Markov kernels

`MonoidalStoch.lean` explains why `MeasObj` itself cannot carry the intended tensor on all kernels: `Kernel.parallelComp` is only mathematically sound once the morphisms are restricted to a regime such as Markov or s-finite kernels.

This module packages the clean restricted ambient category:

- objects: measurable spaces
- morphisms: Markov kernels
- tensor on objects: measurable products
- tensor on morphisms: `Kernel.parallelComp`
- tensor unit: `PUnit`

The `PUnit` tensor unit avoids the universe issues that appeared with `Unit` in the coherence proofs. Markov kernels are automatically finite, hence s-finite, so the tensor-on-morphisms identities can use Mathlib's `parallelComp` API without any junk-value fallback.

```
noncomputable section

open CategoryTheory
open ProbabilityTheory
open MeasureTheory
open scoped MonoidalCategory

universe u

namespace CpmProofs

section KernelLevel

variable {X₁ X₂ X₃ Y₁ Y₂ Y₃ : Type u}
variable [MeasurableSpace X₁] [MeasurableSpace X₂] [MeasurableSpace X₃]
```

```
variable [MeasurableSpace Y₁] [MeasurableSpace Y₂] [MeasurableSpace Y₃]

noncomputable def punitLeftUnitorKernel (α : Type u) [MeasurableSpace α] :
    Kernel (PUnit × α) α :=
  Kernel.deterministic Prod.snd measurable_snd

noncomputable def punitLeftUnitorKernelInv (α : Type u) [MeasurableSpace α] :
    Kernel α (PUnit × α) :=
  Kernel.deterministic (fun x ⇒ (PUnit.unit, x)) (by measurability)

noncomputable def punitRightUnitorKernel (α : Type u) [MeasurableSpace α] :
    Kernel (α × PUnit) α :=
  Kernel.deterministic Prod.fst measurable_fst

noncomputable def punitRightUnitorKernelInv (α : Type u) [MeasurableSpace α] :
    Kernel α (α × PUnit) :=
  Kernel.deterministic (fun x ⇒ (x, PUnit.unit)) (by measurability)

lemma assocKernel_naturality_type
    (f₁ : Kernel X₁ Y₁) (f₂ : Kernel X₂ Y₂) (f₃ : Kernel X₃ Y₃)
    [IsMarkovKernel f₁] [IsMarkovKernel f₂] [IsMarkovKernel f₃] :
    assocKernel Y₁ Y₂ Y₃ ∘∘ ((f₁ ∥ f₂) ∥ f₃) =
      (f₁ ∥ (f₂ ∥ f₃)) ∘∘ assocKernel X₁ X₂ X₃ := by
  letI : IsSFiniteKernel f₁ := by infer_instance
  letI : IsSFiniteKernel f₂ := by infer_instance
  letI : IsSFiniteKernel f₃ := by infer_instance
  letI : IsSFiniteKernel (f₁ ∥ f₂) := by infer_instance
  letI : IsSFiniteKernel (f₂ ∥ f₃) := by infer_instance
  ext p s hs
  rcases p with ⟨⟨x₁, x₂⟩, x₃⟩
  have hL : assocKernel Y₁ Y₂ Y₃ ∘∘ ((f₁ ∥ f₂) ∥ f₃) =
      Kernel.map (((f₁ ∥ f₂) ∥ f₃)) MeasurableEquiv.prodAssoc := by
    simpa [assocKernel] using
      (Kernel.deterministic_comp_eq_map (f := MeasurableEquiv.prodAssoc)
        (hf := MeasurableEquiv.measurable _) (((f₁ ∥ f₂) ∥ f₃)))
  have hR : (f₁ ∥ (f₂ ∥ f₃)) ∘∘ assocKernel X₁ X₂ X₃ =
      Kernel.comap (f₁ ∥ (f₂ ∥ f₃)) MeasurableEquiv.prodAssoc
        (MeasurableEquiv.measurable _) := by
    simpa [assocKernel] using
      (Kernel.comp_deterministic_eq_comap (κ := (f₁ ∥ (f₂ ∥ f₃)))
        (g := MeasurableEquiv.prodAssoc) (hg := MeasurableEquiv.measurable _))
  rw [hL, Kernel.map_apply _ (MeasurableEquiv.measurable _), hR, Kernel.comap_apply]
  simpa [Kernel.parallelComp_apply] using congrArg (fun μ ⇒ μ s)
    (Measure.prodAssoc_prod (μ := f₁ x₁) (ν := f₂ x₂) (τ := f₃ x₃))
```

```
lemma punitLeftUnitor_naturality_type
    {X Y : Type u} [MeasurableSpace X] [MeasurableSpace Y]
    (f : Kernel X Y) [IsMarkovKernel f] :
    punitLeftUnitorKernel Y ∘ₖ ((Kernel.id : Kernel PUnit PUnit) ×ₖ f) =
      f ∘ₖ punitLeftUnitorKernel X := by
  letI : IsSFiniteKernel f := by infer_instance
  ext p s hs
  rcases p with ⟨u, x⟩
  cases u
  have hL : punitLeftUnitorKernel Y ∘ₖ ((Kernel.id : Kernel PUnit PUnit) ×ₖ f) =
      Kernel.map ((Kernel.id : Kernel PUnit PUnit) ×ₖ f) Prod.snd := by
    simpa [punitLeftUnitorKernel] using
      (Kernel.deterministic_comp_eq_map (f := Prod.snd) (hf := measurable_snd)
        ((Kernel.id : Kernel PUnit PUnit) ×ₖ f))
  have hR : f ∘ₖ punitLeftUnitorKernel X = Kernel.comap f Prod.snd measurable_snd := by
    simpa [punitLeftUnitorKernel] using
      (Kernel.comp_deterministic_eq_comap (κ := f) (g := Prod.snd) (hg := measurable_snd))
  rw [hL, Kernel.map_apply _ measurable_snd, hR, Kernel.comap_apply]
  simp [Kernel.parallelComp_apply, Kernel.id_apply, Measure.map_snd_prod]

lemma punitRightUnitor_naturality_type
    {X Y : Type u} [MeasurableSpace X] [MeasurableSpace Y]
    (f : Kernel X Y) [IsMarkovKernel f] :
    punitRightUnitorKernel Y ∘ₖ (f ×ₖ (Kernel.id : Kernel PUnit PUnit)) =
      f ∘ₖ punitRightUnitorKernel X := by
  letI : IsSFiniteKernel f := by infer_instance
  ext p s hs
  rcases p with ⟨x, u⟩
  cases u
  have hL : punitRightUnitorKernel Y ∘ₖ (f ×ₖ (Kernel.id : Kernel PUnit PUnit)) =
      Kernel.map (f ×ₖ (Kernel.id : Kernel PUnit PUnit)) Prod.fst := by
    simpa [punitRightUnitorKernel] using
      (Kernel.deterministic_comp_eq_map (f := Prod.fst) (hf := measurable_fst)
        (f ×ₖ (Kernel.id : Kernel PUnit PUnit)))
  have hR : f ∘ₖ punitRightUnitorKernel X = Kernel.comap f Prod.fst measurable_fst := by
    simpa [punitRightUnitorKernel] using
      (Kernel.comp_deterministic_eq_comap (κ := f) (g := Prod.fst) (hg := measurable_fst))
  rw [hL, Kernel.map_apply _ measurable_fst, hR, Kernel.comap_apply]
  simp [Kernel.parallelComp_apply, Kernel.id_apply, Measure.map_fst_prod]

section

variable {W X Y Z : Type u}
variable [MeasurableSpace W] [MeasurableSpace X] [MeasurableSpace Y] [MeasurableSpace Z]
```

```
noncomputable def pentagonTarget :
    Kernel (((W × X) × Y) × Z) (W × (X × (Y × Z))) :=
  Kernel.deterministic (fun p ⇒ (p.1.1.1, (p.1.1.2, (p.1.2, p.2)))) (by measurability)

lemma pentagon_type :
    ((Kernel.id : Kernel W W) ⊗⊗ assocKernel X Y Z) ⊗⊗ assocKernel W (X × Y) Z ⊗⊗
      (assocKernel W X Y ⊗⊗ (Kernel.id : Kernel Z Z)) =
        assocKernel W X (Y × Z) ⊗⊗ assocKernel (W × X) Y Z := by
  calc
    ((Kernel.id : Kernel W W) ⊗⊗ assocKernel X Y Z) ⊗⊗ assocKernel W (X × Y) Z ⊗⊗
        (assocKernel W X Y ⊗⊗ (Kernel.id : Kernel Z Z)) = pentagonTarget := by
      rw [Kernel.id, Kernel.id, assocKernel, assocKernel, assocKernel, pentagonTarget]
      repeat rw [Kernel.deterministic_parallelComp_deterministic]
      repeat rw [Kernel.deterministic_comp_deterministic]
      apply Kernel.deterministic_congr
      funext p
      rcases p with ⟨⟨⟨w, x⟩, y⟩, z⟩
      rfl
    _ = assocKernel W X (Y × Z) ⊗⊗ assocKernel (W × X) Y Z := by
      rw [assocKernel, assocKernel, pentagonTarget]
      repeat rw [Kernel.deterministic_comp_deterministic]
      apply Kernel.deterministic_congr
      funext p
      rcases p with ⟨⟨⟨w, x⟩, y⟩, z⟩
      rfl

noncomputable def triangleTarget : Kernel ((X × PUnit) × Y) (X × Y) :=
  Kernel.deterministic (fun p ⇒ (p.1.1, p.2)) (by measurability)

lemma triangle_type :
    ((Kernel.id : Kernel X X) ⊗⊗ punitLeftUnitorKernel Y) ⊗⊗ assocKernel X PUnit Y =
      punitRightUnitorKernel X ⊗⊗ (Kernel.id : Kernel Y Y) := by
  calc
    ((Kernel.id : Kernel X X) ⊗⊗ punitLeftUnitorKernel Y) ⊗⊗ assocKernel X PUnit Y =
        triangleTarget := by
      rw [Kernel.id, punitLeftUnitorKernel, assocKernel, triangleTarget]
      repeat rw [Kernel.deterministic_parallelComp_deterministic]
      repeat rw [Kernel.deterministic_comp_deterministic]
      apply Kernel.deterministic_congr
      funext p
      rcases p with ⟨⟨x, u⟩, y⟩
      cases u
      rfl
    _ = punitRightUnitorKernel X ⊗⊗ (Kernel.id : Kernel Y Y) := by
      rw [Kernel.id, punitRightUnitorKernel, triangleTarget]
```

```
      rw [Kernel.deterministic_parallelComp_deterministic]
      apply Kernel.deterministic_congr
      funext p
      rcases p with ⟨x, y⟩
      rfl

end
end KernelLevel

/-- Bundled measurable spaces with Markov kernels as morphisms. -/
structure StochMarkov where
  α : Type u
  inst : MeasurableSpace α

attribute [instance] StochMarkov.inst
instance : CoeSort StochMarkov (Type u) := ⟨StochMarkov.α⟩

namespace StochMarkov

/-- Morphisms in `StochMarkov` are Markov kernels. -/
abbrev Hom (X Y : StochMarkov) := {κ : Kernel X Y // IsMarkovKernel κ}

instance {X Y : StochMarkov} : Coe (Hom X Y) (Kernel X Y) := ⟨Subtype.val⟩

instance {X Y : StochMarkov} (f : Hom X Y) : IsSFiniteKernel (f : Kernel X Y) := by
  letI : IsMarkovKernel (f : Kernel X Y) := f.2
  letI : IsFiniteKernel (f : Kernel X Y) := by infer_instance
  infer_instance

noncomputable instance : Category.{u} StochMarkov where
  Hom X Y := Hom X Y
  id X := ⟨Kernel.id, by infer_instance⟩
  comp f g := by
    letI : IsMarkovKernel f.1 := f.2
    letI : IsMarkovKernel g.1 := g.2
    have h : IsMarkovKernel (g.1 ∘ₖ f.1) := by infer_instance
    exact ⟨g.1 ∘ₖ f.1, h⟩
  id_comp f := by
    apply Subtype.ext
    exact Kernel.comp_id f.1
  comp_id f := by
    apply Subtype.ext
    exact Kernel.id_comp f.1
  assoc f g h := by
    apply Subtype.ext
```

```
    simpa using (Kernel.comp_assoc h.1 g.1 f.1).symm

/-- Tensor product on objects is the measurable product. -/
noncomputable def tensorObj (X Y : StochMarkov) : StochMarkov :=
  ⟨X × Y, inferInstance⟩

/-- The tensor unit is `PUnit`, which keeps the coherence proofs universe-safe. -/
def tensorUnit : StochMarkov := ⟨PUnit, inferInstance⟩

noncomputable def pLeftUnitorKernel (X : StochMarkov) : Kernel (PUnit × X) X :=
  punitLeftUnitorKernel X

noncomputable def pLeftUnitorKernelInv (X : StochMarkov) : Kernel X (PUnit × X) :=
  punitLeftUnitorKernelInv X

noncomputable def pRightUnitorKernel (X : StochMarkov) : Kernel (X × PUnit) X :=
  punitRightUnitorKernel X

noncomputable def pRightUnitorKernelInv (X : StochMarkov) : Kernel X (X × PUnit) :=
  punitRightUnitorKernelInv X

lemma assocKernel_isMarkov (X Y Z : StochMarkov) : IsMarkovKernel (assocKernel X Y Z) := by
  dsimp [assocKernel]
  infer_instance

lemma assocKernelInv_isMarkov (X Y Z : StochMarkov) :
    IsMarkovKernel (assocKernelInv X Y Z) := by
  dsimp [assocKernelInv]
  infer_instance

lemma pLeftUnitorKernel_isMarkov (X : StochMarkov) :
    IsMarkovKernel (pLeftUnitorKernel X) := by
  dsimp [pLeftUnitorKernel, punitLeftUnitorKernel]
  infer_instance

lemma pLeftUnitorKernelInv_isMarkov (X : StochMarkov) :
    IsMarkovKernel (pLeftUnitorKernelInv X) := by
  dsimp [pLeftUnitorKernelInv, punitLeftUnitorKernelInv]
  infer_instance

lemma pRightUnitorKernel_isMarkov (X : StochMarkov) :
    IsMarkovKernel (pRightUnitorKernel X) := by
  dsimp [pRightUnitorKernel, punitRightUnitorKernel]
  infer_instance
```

```
lemma pRightUnitorKernelInv_isMarkov (X : StochMarkov) :
    IsMarkovKernel (pRightUnitorKernelInv X) := by
  dsimp [pRightUnitorKernelInv, punitRightUnitorKernelInv]
  infer_instance

/-- Tensor product on morphisms via parallel composition. -/
noncomputable def tensorHom {X₁ Y₁ X₂ Y₂ : StochMarkov} (f : X₁ → Y₁) (g : X₂ → Y₂) :
    tensorObj X₁ X₂ → tensorObj Y₁ Y₂ := by
  have h : IsMarkovKernel (f.1 ⊗ₖ g.1) := by
    letI : IsMarkovKernel f.1 := f.2
    letI : IsMarkovKernel g.1 := g.2
    infer_instance
  exact ⟨f.1 ⊗ₖ g.1, h⟩

noncomputable def whiskerLeft (X : StochMarkov) {Y₁ Y₂ : StochMarkov} (f : Y₁ → Y₂) :
    tensorObj X Y₁ → tensorObj X Y₂ :=
  tensorHom (𝟙 X) f

noncomputable def whiskerRight {X₁ X₂ : StochMarkov} (f : X₁ → X₂) (Y : StochMarkov) :
    tensorObj X₁ Y → tensorObj X₂ Y :=
  tensorHom f (𝟙 Y)

lemma assocKernel_comp_inv (X Y Z : StochMarkov) :
    assocKernelInv X Y Z ∘ₖ assocKernel X Y Z = Kernel.id := by
  simp only [assocKernel, assocKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

lemma assocKernel_inv_comp (X Y Z : StochMarkov) :
    assocKernel X Y Z ∘ₖ assocKernelInv X Y Z = Kernel.id := by
  simp only [assocKernel, assocKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

lemma pLeftUnitorKernel_comp_inv (X : StochMarkov) :
    pLeftUnitorKernelInv X ∘ₖ pLeftUnitorKernel X = Kernel.id := by
  simp only [pLeftUnitorKernel, pLeftUnitorKernelInv,
    punitLeftUnitorKernel, punitLeftUnitorKernelInv]
  rw [Kernel.deterministic_comp_deterministic]
  ext ⟨u, x⟩
  cases u
  simp [Kernel.deterministic_apply, Kernel.id_apply]

lemma pLeftUnitorKernel_inv_comp (X : StochMarkov) :
    pLeftUnitorKernel X ∘ₖ pLeftUnitorKernelInv X = Kernel.id := by
```

```
  simp only [pLeftUnitorKernel, pLeftUnitorKernelInv,
    punitLeftUnitorKernel, punitLeftUnitorKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

lemma pRightUnitorKernel_comp_inv (X : StochMarkov) :
    pRightUnitorKernelInv X ⊗⊗ pRightUnitorKernel X = Kernel.id := by
  simp only [pRightUnitorKernel, pRightUnitorKernelInv,
    punitRightUnitorKernel, punitRightUnitorKernelInv]
  rw [Kernel.deterministic_comp_deterministic]
  ext ⟨x, u⟩
  cases u
  simp [Kernel.deterministic_apply, Kernel.id_apply]

lemma pRightUnitorKernel_inv_comp (X : StochMarkov) :
    pRightUnitorKernel X ⊗⊗ pRightUnitorKernelInv X = Kernel.id := by
  simp only [pRightUnitorKernel, pRightUnitorKernelInv,
    punitRightUnitorKernel, punitRightUnitorKernelInv, Kernel.id]
  rw [Kernel.deterministic_comp_deterministic]
  rfl

noncomputable def associatorIso (X Y Z : StochMarkov) :
    tensorObj (tensorObj X Y) Z ≅ tensorObj X (tensorObj Y Z) where
  hom := ⟨assocKernel X Y Z, assocKernel_isMarkov X Y Z⟩
  inv := ⟨assocKernelInv X Y Z, assocKernelInv_isMarkov X Y Z⟩
  hom_inv_id := by
    apply Subtype.ext
    simpa using assocKernel_comp_inv X Y Z
  inv_hom_id := by
    apply Subtype.ext
    simpa using assocKernel_inv_comp X Y Z

noncomputable def leftUnitorIso (X : StochMarkov) : tensorObj tensorUnit X ≅ X where
  hom := ⟨pLeftUnitorKernel X, pLeftUnitorKernel_isMarkov X⟩
  inv := ⟨pLeftUnitorKernelInv X, pLeftUnitorKernelInv_isMarkov X⟩
  hom_inv_id := by
    apply Subtype.ext
    simpa using pLeftUnitorKernel_comp_inv X
  inv_hom_id := by
    apply Subtype.ext
    simpa using pLeftUnitorKernel_inv_comp X

noncomputable def rightUnitorIso (X : StochMarkov) : tensorObj X tensorUnit ≅ X where
  hom := ⟨pRightUnitorKernel X, pRightUnitorKernel_isMarkov X⟩
  inv := ⟨pRightUnitorKernelInv X, pRightUnitorKernelInv_isMarkov X⟩
```

```
  hom_inv_id := by
    apply Subtype.ext
    simpa using pRightUnitorKernel_comp_inv X
  inv_hom_id := by
    apply Subtype.ext
    simpa using pRightUnitorKernel_inv_comp X

noncomputable def monoidalCategory : MonoidalCategory StochMarkov := by
  letI : MonoidalCategoryStruct StochMarkov := {
    tensorObj := StochMarkov.tensorObj
    whiskerLeft := StochMarkov.whiskerLeft
    whiskerRight := StochMarkov.whiskerRight
    tensorHom := fun {X₁ Y₁ X₂ Y₂} f g ⇒ StochMarkov.tensorHom f g
    tensorUnit := StochMarkov.tensorUnit
    associator := StochMarkov.associatorIso
    leftUnitor := StochMarkov.leftUnitorIso
    rightUnitor := StochMarkov.rightUnitorIso
  }
  exact MonoidalCategory.ofTensorHom (C := StochMarkov)
    (id_tensorHom_id := by
      intro X Y
      apply Subtype.ext
      simpa [StochMarkov.tensorHom] using (parallelComp_id_id X Y))
    (id_tensorHom := by
      intro X Y₁ Y₂ f
      rfl)
    (tensorHom_id := by
      intro X₁ X₂ f Y
      rfl)
    (tensorHom_comp_tensorHom := by
      intro X₁ Y₁ Z₁ X₂ Y₂ Z₂ f₁ f₂ g₁ g₂
      letI : IsSFiniteKernel f₁.1 := by infer_instance
      letI : IsSFiniteKernel f₂.1 := by infer_instance
      letI : IsSFiniteKernel g₁.1 := by infer_instance
      letI : IsSFiniteKernel g₂.1 := by infer_instance
      apply Subtype.ext
      simpa [StochMarkov.tensorHom] using
        (Kernel.parallelComp_comp_parallelComp (κ := f₁.1) (η := g₁.1)
          (κ' := f₂.1) (η' := g₂.1)))
    (associator_naturality := by
      intro X₁ X₂ X₃ Y₁ Y₂ Y₃ f₁ f₂ f₃
      letI : IsMarkovKernel f₁.1 := f₁.2
      letI : IsMarkovKernel f₂.1 := f₂.2
      letI : IsMarkovKernel f₃.1 := f₃.2
      apply Subtype.ext
```

```
      simpa [StochMarkov.tensorHom, associatorIso] using
        (assocKernel_naturality_type (f₁ := f₁.1) (f₂ := f₂.1) (f₃ := f₃.1)))
    (leftUnitor_naturality := by
      intro X Y f
      letI : IsMarkovKernel f.1 := f.2
      apply Subtype.ext
      simpa [StochMarkov.tensorHom, leftUnitorIso, pLeftUnitorKernel, punitLeftUnitorKernel]
        (punitLeftUnitor_naturality_type (f := f.1)))
    (rightUnitor_naturality := by
      intro X Y f
      letI : IsMarkovKernel f.1 := f.2
      apply Subtype.ext
      simpa [StochMarkov.tensorHom, rightUnitorIso, pRightUnitorKernel,
        punitRightUnitorKernel] using
        (punitRightUnitor_naturality_type (f := f.1)))
    (pentagon := by
      intro W X Y Z
      apply Subtype.ext
      simpa [StochMarkov.tensorHom, associatorIso] using
        (pentagon_type (W := W) (X := X) (Y := Y) (Z := Z)))
    (triangle := by
      intro X Y
      apply Subtype.ext
      simpa [StochMarkov.tensorHom, associatorIso, leftUnitorIso, rightUnitorIso,
        pLeftUnitorKernel, pRightUnitorKernel, punitLeftUnitorKernel,
        punitRightUnitorKernel] using
        (triangle_type (X := X) (Y := Y)))

noncomputable instance : MonoidalCategory StochMarkov := monoidalCategory

end StochMarkov

end CpmProofs
```

```
import CpmProofs.StochCat
import Mathlib.CategoryTheory.Monoidal.Closed.Enrichment
import Mathlib.MeasureTheory.Measure.GiryMonad
import Mathlib.MeasureTheory.MeasurableSpace.Constructions
```

Source: `StochSelfEnrichment.lean`

Stochastic self-enrichment scaffolding

This file isolates the remaining gap between the concrete stochastic category from `CpmProofs.StochCat` and a full `MonoidalClosed` self-enrichment.

We define a candidate internal-hom object `kernelHomObj X Y` whose underlying space consists of kernels `X → Y`, equipped with the pointwise measurable structure coming from the map `κ ⊡ (x ⊡ κ x)`.

This gives a concrete place where the missing measurable-space work lives:

- we can talk about pointwise-measurable evaluation maps `κ ⊡ κ x`,
- we can describe the underlying function induced by postcomposition,
- and once a genuine `MonoidalCategory`/`MonoidalClosed` structure is supplied, Mathlib already upgrades it to a self-enrichment automatically.

What is still missing for a full self-enrichment is a bona fide stochastic tensor on morphisms together with a jointly measurable evaluation/curry-uncurry adjunction for these kernel-hom objects.

```
open CategoryTheory
open MeasureTheory
open ProbabilityTheory
open scoped MonoidalClosed

noncomputable section

namespace CpmProofs

/-- Candidate internal-hom object for stochastic self-enrichment:
the measurable space of kernels `X → Y`, equipped with the pointwise measurable
structure induced by the map into `X → Measure Y`. -/
def kernelHomObj (X Y : MeasObj) : MeasObj where
  α := Kernel X Y
  inst := MeasurableSpace.comap (fun κ : Kernel X Y ⇒ (κ : X → Measure Y)) inferInstance

/-- The defining map from the candidate kernel-hom object to the corresponding
function space is measurable by construction. -/
lemma measurable_kernel_toFun (X Y : MeasObj) :
    Measurable (fun κ : kernelHomObj X Y ⇒ ((show Kernel X Y from κ) : X → Measure Y)) := b
  exact Measurable.of_comap_le le_rfl

/-- Evaluation at a fixed point is measurable on the candidate kernel-hom object. -/
lemma measurable_kernel_eval (X Y : MeasObj) (x : X) :
    Measurable (fun κ : kernelHomObj X Y ⇒ ((show Kernel X Y from κ) : X → Measure Y) x) :=
  simpa using (Measurable.eval (a := (x : X)) (hg := measurable_kernel_toFun X Y))

/-- Evaluation of a kernel on a fixed measurable set is measurable on the
candidate kernel-hom object. -/
lemma measurable_kernel_apply (X Y : MeasObj) (x : X) {s : Set Y} (hs : MeasurableSet s) :
    Measurable (fun κ : kernelHomObj X Y ⇒ (show Kernel X Y from κ) x s) := by
  exact (Measure.measurable_coe hs).comp (measurable_kernel_eval X Y x)
```

```
/-- The expected underlying function on kernel-hom objects induced by
postcomposition with a kernel. The remaining self-enrichment gap is to show that
this participates in a full internal-hom adjunction. -/
def postcompKernelFun (X Y Z : MeasObj) (η : X → Y) : kernelHomObj Z X → kernelHomObj Z Y :=
  fun κ ⇒
    ({ toFun := fun z ⇒ (((show Kernel Z X from κ) z)).bind η
       measurable' := by
         exact (Measure.measurable_bind' η.measurable).comp ((show Kernel Z X from κ).measur
      Kernel Z Y)

section Abstract

variable [MonoidalCategory MeasObj] [MonoidalClosed MeasObj]

/-- Once a monoidal closed structure on the stochastic category is supplied,
Mathlib immediately upgrades it to a self-enrichment. -/
scoped instance stochEnrichedOrdinaryCategory : EnrichedOrdinaryCategory MeasObj MeasObj :=
  CategoryTheory.MonoidalClosed.enrichedOrdinaryCategorySelf MeasObj

/-- Under a hypothetical monoidal closed structure, this is the internal-hom
object used by stochastic self-enrichment. -/
abbrev stochInternalHom (X Y : MeasObj) : MeasObj :=
  (CategoryTheory.ihom X).obj Y

/-- Under a hypothetical monoidal closed structure, currying is the hom
equivalence that would package stochastic self-enrichment. -/
abbrev stochCurryEquiv (X Y Z : MeasObj) :=
  (CategoryTheory.ihom.adjunction X).homEquiv Y Z

end Abstract

end CpmProofs
```

```
import Mathlib.CategoryTheory.Enriched.FunctorCategory
import Mathlib.CategoryTheory.Monoidal.Closed.Enrichment
```

Source: `WeightedColimit.lean`

Weighted colimits

This file introduces a lightweight local notion of weighted colimit for `V`-enriched categories. It is sufficient for the enriched Kan-extension roadmap in `CpmProofs/Enriched.lean`.

For a diagram `F : J ↣ C` and a weight `W : J𝒾 ↣ V`, the `V`-object of weighted

cocones with vertex `X : C` is defined as the enriched hom object

`[J□□, V](W, j ↦ (F j ⟶[V] X))`.

A weighted colimit of `F` by `W` is then an object `L : C` representing this weighted cocone object.

```
noncomputable section

universe v u₁ u₂ u₃

namespace CpmProofs

open CategoryTheory Opposite
open CategoryTheory.Enriched.FunctorCategory
open scoped MonoidalClosed

variable (V : Type u₁) [Category.{v} V] [MonoidalCategory V] [MonoidalClosed V]
variable {J : Type u₂} [Category.{v} J]
variable {C : Type u₃} [Category.{v} C] [EnrichedOrdinaryCategory V C]

/-- For `F : J ↝ C` and `X : C`, this is the `J□□ ↝ V` diagram
`j ↦ (F j ⟶[V] X)` that appears in the weighted-colimit universal property. -/
@[simps]
def weightedHomDiagram (F : J ↝ C) (X : C) : J□□ ↝ V where
  obj j := F.obj j.unop ⟶[V] X
  map f := CategoryTheory.eHomWhiskerRight V (F.map f.unop) X

/-- Postcomposition with `f : X ⟶ Y` induces a morphism between the weighted
hom diagrams for vertices `X` and `Y`. -/
@[simps]
def weightedHomDiagramMap (F : J ↝ C) {X Y : C} (f : X ⟶ Y) :
    weightedHomDiagram V F X ⟶ weightedHomDiagram V F Y where
  app j := CategoryTheory.eHomWhiskerLeft V (F.obj j.unop) f
  naturality j j' g := by
    dsimp
    simpa using
      (CategoryTheory.eHom_whisker_exchange (V := V) (f := F.map g.unop) (g := f)).symm

/-- The assumption needed for the `V`-object of weighted cocones with vertex `X`
to exist. -/
abbrev HasWeightedCoconeObj (W : J□□ ↝ V) (F : J ↝ C) (X : C) :=
  HasEnrichedHom V W (weightedHomDiagram V F X)

/-- The `V`-object of `W`-weighted cocones on `F` with vertex `X`. -/
abbrev WeightedCoconeObj (W : J□□ ↝ V) (F : J ↝ C) (X : C)
    [HasWeightedCoconeObj (V := V) W F X] : V :=
```

```
  enrichedHom V W (weightedHomDiagram V F X)

/-- Postcomposition with `f : X → Y` induces a morphism between the
`V`-objects of weighted cocones with vertices `X` and `Y`. -/
def weightedCoconeMap (W : J⁅⁆ ↛ V) (F : J ↛ C) {X Y : C} (f : X → Y)
    [HasWeightedCoconeObj (V := V) W F X]
    [HasWeightedCoconeObj (V := V) W F Y]
    [∀ F₁ F₂ : J⁅⁆ ↛ V, HasEnrichedHom V F₁ F₂] :
    WeightedCoconeObj (V := V) W F X → WeightedCoconeObj (V := V) W F Y := by
  letI : EnrichedOrdinaryCategory V V := by infer_instance
  letI : EnrichedOrdinaryCategory V (J⁅⁆ ↛ V) :=
    enrichedOrdinaryCategory (V := V) (J := J⁅⁆) (C := V)
  exact CategoryTheory.eHomWhiskerLeft V W (weightedHomDiagramMap V F f)

/-- The universal property of a `W`-weighted colimit of `F`, expressed as a
representing object for the `V`-object of weighted cocones. -/
structure IsWeightedColimit (W : J⁅⁆ ↛ V) (F : J ↛ C) (L : C)
    [∀ X : C, HasWeightedCoconeObj (V := V) W F X]
    [∀ F₁ F₂ : J⁅⁆ ↛ V, HasEnrichedHom V F₁ F₂] where
  /-- The representing isomorphism
  `C(L, X) ≅ [J⁅⁆, V](W, j ⁅⁆ C(F j, X))`. -/
  desc : ∀ X : C, (L →[V] X) ≅ WeightedCoconeObj (V := V) W F X
  /-- Naturality of `desc` in the vertex object. -/
  naturality : ∀ {X Y : C} (f : X → Y),
    CategoryTheory.eHomWhiskerLeft V L f ⁅⁆ (desc Y).hom =
      (desc X).hom ⁅⁆ weightedCoconeMap (V := V) W F f

/-- A weighted colimit packages the representing object together with its
universal property. -/
structure WeightedColimit (W : J⁅⁆ ↛ V) (F : J ↛ C)
    [∀ X : C, HasWeightedCoconeObj (V := V) W F X]
    [∀ F₁ F₂ : J⁅⁆ ↛ V, HasEnrichedHom V F₁ F₂] where
  /-- The weighted colimit object. -/
  obj : C
  /-- The representing universal property. -/
  isWeightedColimit : IsWeightedColimit (V := V) W F obj

/-- The representing isomorphism associated to a weighted colimit. -/
abbrev WeightedColimit.descIso (W : J⁅⁆ ↛ V) (F : J ↛ C)
    [∀ X : C, HasWeightedCoconeObj (V := V) W F X]
    [∀ F₁ F₂ : J⁅⁆ ↛ V, HasEnrichedHom V F₁ F₂]
    (colim : WeightedColimit (V := V) W F) (X : C) :
    (colim.obj →[V] X) ≅ WeightedCoconeObj (V := V) W F X :=
  colim.isWeightedColimit.desc X
```

```
/-- Naturality of the weighted-colimit representing isomorphism. -/
@[reassoc]
lemma WeightedColimit.desc_hom_naturality (W : J□□ ↛ V) (F : J ↛ C)
    [∀ X : C, HasWeightedCoconeObj (V := V) W F X]
    [∀ F₁ F₂ : J□□ ↛ V, HasEnrichedHom V F₁ F₂]
    (colim : WeightedColimit (V := V) W F) {X Y : C} (f : X → Y) :
    CategoryTheory.eHomWhiskerLeft V colim.obj f □ (colim.descIso (X := Y)).hom =
      (colim.descIso (X := X)).hom □ weightedCoconeMap (V := V) W F f :=
  colim.isWeightedColimit.naturality f

end CpmProofs
```

```
import CpmProofs.WeightedColimit
```

Source: `EnrichedKanExtension.lean`

Enriched Kan extensions

This file adds a lightweight local interface for pointwise enriched left Kan extensions. It is built directly from the weighted-colimit API in `CpmProofs.WeightedColimit`.

For `F : A ↛ B` and `G : A ↛ C`, the pointwise enriched left Kan extension of `G` along `F` at `b : B` is presented as a weighted colimit of `G` by the representable weight `B(F-, b)`.

```
noncomputable section

universe v u₁ u₂ u₃ u₄

namespace CpmProofs

open CategoryTheory Opposite
open CategoryTheory.Enriched.FunctorCategory
open scoped MonoidalClosed

variable (V : Type u₁) [Category.{v} V] [MonoidalCategory V] [MonoidalClosed V]
variable {A : Type u₂} [Category.{v} A]
variable {B : Type u₃} [Category.{v} B] [EnrichedOrdinaryCategory V B]
variable {C : Type u₄} [Category.{v} C] [EnrichedOrdinaryCategory V C]

/-- The representable weight `a □ B(F a, b)` used in the pointwise enriched
left Kan extension formula. -/
abbrev leftKanWeight (F : A ↛ B) (b : B) : A□□ ↛ V :=
  weightedHomDiagram V F b
```

```
/-- A morphism `b → b'` in the target category induces a morphism between the
corresponding representable weights. -/
abbrev leftKanWeightMap (F : A ↝ B) {b b' : B} (f : b → b') :
    leftKanWeight V F b → leftKanWeight V F b' :=
  weightedHomDiagramMap V F f

/-- Precomposition in the weight variable. -/
def weightedCoconeMapWeight {W W' : A□□ ↝ V} (G : A ↝ C) {X : C} (η : W → W')
    [HasWeightedCoconeObj (V := V) W G X]
    [HasWeightedCoconeObj (V := V) W' G X]
    [∀ F₁ F₂ : A□□ ↝ V, HasEnrichedHom V F₁ F₂] :
    WeightedCoconeObj (V := V) W' G X → WeightedCoconeObj (V := V) W G X := by
  letI : EnrichedOrdinaryCategory V V := by infer_instance
  letI : EnrichedOrdinaryCategory V (A□□ ↝ V) :=
    enrichedOrdinaryCategory (V := V) (J := A□□) (C := V)
  exact CategoryTheory.eHomWhiskerRight V η (weightedHomDiagram V G X)

/-- A morphism of weights induces a morphism between weighted colimit objects. -/
def WeightedColimit.mapWeight {W W' : A□□ ↝ V} (G : A ↝ C)
    [∀ X : C, HasWeightedCoconeObj (V := V) W G X]
    [∀ X : C, HasWeightedCoconeObj (V := V) W' G X]
    [∀ F₁ F₂ : A□□ ↝ V, HasEnrichedHom V F₁ F₂]
    (η : W → W') (colim : WeightedColimit (V := V) W G)
    (colim' : WeightedColimit (V := V) W' G) :
    colim.obj → colim'.obj := by
  refine (CategoryTheory.eHomEquiv V (X := colim.obj) (Y := colim'.obj)).symm ?_
  exact CategoryTheory.eId V colim'.obj □
    (colim'.descIso (W := W') (F := G) (X := colim'.obj)).hom □
      weightedCoconeMapWeight (V := V) (G := G) (W := W) (W' := W')
        (X := colim'.obj) η □
        (colim.descIso (W := W) (F := G) (X := colim'.obj)).inv

/-- Existence of the weighted cocone object needed to define the pointwise
enriched left Kan extension at `b`. -/
abbrev HasPointwiseEnrichedLeftKanExtensionObj
    (F : A ↝ B) (G : A ↝ C) (b : B) (X : C) :=
  HasWeightedCoconeObj (V := V) (W := leftKanWeight V F b) G X

/-- The pointwise enriched left Kan extension condition at a single object
`b : B`. -/
abbrev IsPointwiseEnrichedLeftKanExtensionAt
    (F : A ↝ B) (G : A ↝ C) (b : B) (L : C)
    [∀ X : C, HasPointwiseEnrichedLeftKanExtensionObj (V := V) F G b X]
    [∀ F₁ F₂ : A□□ ↝ V, HasEnrichedHom V F₁ F₂] :=
  IsWeightedColimit (V := V) (W := leftKanWeight V F b) G L
```

```
/-- A local pointwise enriched left Kan extension, packaged as a family of
weighted colimits. -/
structure PointwiseEnrichedLeftKanExtension (F : A ↛ B) (G : A ↛ C)
    [∀ (b : B) (X : C), HasPointwiseEnrichedLeftKanExtensionObj (V := V) F G b X]
    [∀ F₁ F₂ : A□□ ↛ V, HasEnrichedHom V F₁ F₂] where
  obj : B → C
  isPointwise : ∀ b : B, IsPointwiseEnrichedLeftKanExtensionAt (V := V) F G b (obj b)

/-- The weighted-colimit package at a fixed `b : B`. -/
abbrev PointwiseEnrichedLeftKanExtension.colimit (F : A ↛ B) (G : A ↛ C)
    [∀ (b : B) (X : C), HasPointwiseEnrichedLeftKanExtensionObj (V := V) F G b X]
    [∀ F₁ F₂ : A□□ ↛ V, HasEnrichedHom V F₁ F₂]
    (lan : PointwiseEnrichedLeftKanExtension (V := V) F G) (b : B) :
    WeightedColimit (V := V) (leftKanWeight V F b) G :=
  { obj := lan.obj b, isWeightedColimit := lan.isPointwise b }

/-- The canonical map on pointwise enriched left Kan extension objects induced
by a morphism in `B`. -/
def PointwiseEnrichedLeftKanExtension.map (F : A ↛ B) (G : A ↛ C)
    [∀ (b : B) (X : C), HasPointwiseEnrichedLeftKanExtensionObj (V := V) F G b X]
    [∀ F₁ F₂ : A□□ ↛ V, HasEnrichedHom V F₁ F₂]
    (lan : PointwiseEnrichedLeftKanExtension (V := V) F G) {b b' : B} (f : b → b') :
    lan.obj b → lan.obj b' :=
  WeightedColimit.mapWeight (V := V) (G := G) (η := leftKanWeightMap V F f)
    (PointwiseEnrichedLeftKanExtension.colimit (V := V) (F := F) (G := G) lan b)
    (PointwiseEnrichedLeftKanExtension.colimit (V := V) (F := F) (G := G) lan b')

end CpmProofs
```

```
import CpmProofs.KanBridge
import CpmProofs.EnrichedKanExtension
import Mathlib.CategoryTheory.Monoidal.Category
import Mathlib.CategoryTheory.Monoidal.Closed.Enrichment
```

Source: `EnrichedBridge.lean`

Specializing the abstract enriched formula to the concrete bridge

This file makes precise the remaining step between the local abstract enriched Kan-extension interface and the concrete bridge theorem from `KanBridge.lean`.

The key observations are:

- `lanValue D hD κ f` is just `integrateAlong κ f`,

- a point of the weighted cocone object for a pointwise enriched left Kan extension determines an ordinary morphism by the representing isomorphism, and
- every kernel `κ` canonically determines such a weighted-cocone point by transposing `e.hom □ κ` through the representing isomorphism.

Because the stochastic self-enrichment is still incomplete, the file keeps the missing enriched data explicit:

- `[MonoidalCategory MeasObj] [MonoidalClosed MeasObj]`,
- an explicit `HasEnrichedHom` family on the functor category, and
- the existence of the relevant pointwise enriched left Kan-extension objects.

Under those assumptions, we can already prove that the abstract enriched value agrees with `lanValue`, satisfies the global factorization formula, and inherits `IsStochKanExtension`; the previous identification hypothesis is now discharged internally by the canonical weighted-cocone point attached to `κ`.

#	Result	Status
39	`lanValue_eq_integrateAlong`	✓
40	`kernelFromWeightedPoint`	✓
41	`weightedPointOfKernel`	✓
42	`abstractLanValueFromEnriched_eq_lanValue`	✓
43	`abstractLanValueOfKernelFromEnriched_eq_lanValue`	✓
44	`abstractLanValueOfKernelFromEnriched_integral_factorization`	✓
45	`abstractLanValueOfKernelFromEnriched_isStochKanExtension`	✓

```
noncomputable section

open CategoryTheory Opposite ProbabilityTheory MeasureTheory
open scoped MonoidalCategory MonoidalClosed

namespace CpmProofs

/-- The pointwise stochastic Kan-extension value is exactly integration along
the conditional kernel. -/
theorem lanValue_eq_integrateAlong
    {X Y : MeasObj}
    (D : X → Y) (_hD : Measurable D) (κ : Kernel Y X) (f : X → ℝ) :
    lanValue D _hD κ f = integrateAlong κ f := rfl

section Abstract

variable [MonoidalCategory MeasObj] [MonoidalClosed MeasObj]

scoped instance stochEnrichedOrdinaryCategorySelf : EnrichedOrdinaryCategory MeasObj MeasObj
```

```
  CategoryTheory.MonoidalClosed.enrichedOrdinaryCategorySelf MeasObj

variable {A : Type*} [Category A]
variable (F G : A ↛ MeasObj)

/-- A point of the weighted cocone object determines an ordinary morphism out of
the pointwise enriched left Kan-extension object via the representing
isomorphism. -/
noncomputable def PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint
    (hHom : ∀ F₁ F₂ : A⁰ᵖ ↛ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X]
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (b X : MeasObj)
    (ω : (□_ MeasObj) → WeightedCoconeObj
      (V := MeasObj) (W := leftKanWeight MeasObj F b) G X) :
    lan.obj b → X := by
  letI := hHom
  exact (CategoryTheory.eHomEquiv MeasObj (X := lan.obj b) (Y := X)).symm
    (ω □ (WeightedColimit.descIso (V := MeasObj)
      (W := leftKanWeight MeasObj F b) (F := G)
      (colim := PointwiseEnrichedLeftKanExtension.colimit
        (V := MeasObj) (F := F) (G := G) lan b)
      (X := X)).inv)

/-- Every ordinary kernel `κ : b → X` canonically determines a point of the
weighted cocone object, by transposing `e.hom □ κ` through the enriched-hom
equivalence and then across the representing isomorphism. -/
noncomputable def PointwiseEnrichedLeftKanExtension.weightedPointOfKernel
    (hHom : ∀ F₁ F₂ : A⁰ᵖ ↛ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X]
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    {b X : MeasObj} (e : lan.obj b ≅ b) (κ : b → X) :
    (□_ MeasObj) → WeightedCoconeObj
      (V := MeasObj) (W := leftKanWeight MeasObj F b) G X := by
  letI := hHom
  exact ((CategoryTheory.eHomEquiv MeasObj (X := lan.obj b) (Y := X)) (e.hom □ κ)) □
    (WeightedColimit.descIso (V := MeasObj)
      (W := leftKanWeight MeasObj F b) (F := G)
      (colim := PointwiseEnrichedLeftKanExtension.colimit
        (V := MeasObj) (F := F) (G := G) lan b)
      (X := X)).hom

/-- The canonical weighted-cocone point attached to `κ` recovers `e.hom □ κ`
```

```
when translated back through the representing isomorphism. -/
theorem PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint_weightedPointOfKernel
    (hHom : ∀ F₁ F₂ : A⬝⬝ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X]
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    {b X : MeasObj} (e : lan.obj b ≅ b) (κ : b → X) :
    PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint (F := F) (G := G)
      hHom lan b X
      (PointwiseEnrichedLeftKanExtension.weightedPointOfKernel (F := F) (G := G)
        hHom lan e κ) =
      e.hom ⬝ κ := by
  letI := hHom
  simp [PointwiseEnrichedLeftKanExtension.weightedPointOfKernel,
    PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint]

/-- After precomposing by the chosen identification `lan.obj b ≅ b`, the
canonical weighted-cocone point recovers the original kernel `κ`. -/
theorem PointwiseEnrichedLeftKanExtension.inv_kernelFromWeightedPoint_weightedPointOfKernel
    (hHom : ∀ F₁ F₂ : A⬝⬝ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X]
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    {b X : MeasObj} (e : lan.obj b ≅ b) (κ : b → X) :
    e.inv ⬝ PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint (F := F) (G := G)
      hHom lan b X
      (PointwiseEnrichedLeftKanExtension.weightedPointOfKernel (F := F) (G := G)
        hHom lan e κ) = κ := by
  rw [PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint_weightedPointOfKernel
    (F := F) (G := G) hHom lan e κ]
  simp

/-- Applying the abstract enriched pointwise formula to a weighted-cocone point
and then integrating an observable `f` against the resulting kernel gives a
concrete candidate stochastic Kan-extension value on `Y`. -/
noncomputable def abstractLanValueFromEnriched
    {X Y : MeasObj}
    (hHom : ∀ F₁ F₂ : A⬝⬝ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (e : lan.obj Y ≅ Y)
    (ω : (⬝_ MeasObj) → WeightedCoconeObj
      (V := MeasObj) (W := leftKanWeight MeasObj F Y) G X)
    (f : X → ℝ) : Y → ℝ :=
```

```
  integrateAlong
    (e.inv ▸ PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint
      (F := F) (G := G) hHom lan Y X ω)
    f

/-- Once the missing stochastic self-enrichment data is instantiated so that
the abstract weighted-cocone point recovers the conditional kernel `κ`, the
abstract enriched value reduces to the concrete `lanValue`. -/
theorem abstractLanValueFromEnriched_eq_lanValue
    {X Y : MeasObj}
    (hHom : ∀ F₁ F₂ : A⥤ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (D : X → Y) (hD : Measurable D) (κ : Kernel Y X) (f : X → ℝ)
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (e : lan.obj Y ≅ Y)
    (ω : (▸_ MeasObj) → WeightedCoconeObj
      (V := MeasObj) (W := leftKanWeight MeasObj F Y) G X)
    (hω : e.inv ▸ PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint
      (F := F) (G := G) hHom lan Y X ω = κ) :
    abstractLanValueFromEnriched (F := F) (G := G) hHom lan e ω f =
      lanValue D hD κ f := by
  rw [lanValue_eq_integrateAlong (D := D) (κ := κ) (f := f)]
  simp [abstractLanValueFromEnriched, hω]

/-- The same specialization turns the abstract enriched pointwise formula into
the concrete global disintegration identity from the bridge theorem. -/
theorem abstractLanValueFromEnriched_integral_factorization
    {X Y : MeasObj}
    (hHom : ∀ F₁ F₂ : A⥤ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X) (κ : Kernel Y X) (hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ) (hf_meas : AEStronglyMeasurable f μ) (hf_int : Integrable f μ)
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (e : lan.obj Y ≅ Y)
    (ω : (▸_ MeasObj) → WeightedCoconeObj
      (V := MeasObj) (W := leftKanWeight MeasObj F Y) G X)
    (hω : e.inv ▸ PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint
      (F := F) (G := G) hHom lan Y X ω = κ) :
    (∫ x, f x ∂μ) =
      ∫ y, abstractLanValueFromEnriched (F := F) (G := G) hHom lan e ω f y
        ∂(Measure.map D μ) := by
  rw [abstractLanValueFromEnriched_eq_lanValue (F := F) (G := G) hHom
```

```
    (D := D) (hD := hD) (κ := κ) (f := f) (lan := lan) (e := e) (ω := ω) hω]
  exact lan_eq_integral_of_condKernel D hD μ κ hκ f hf_meas hf_int

/-- The same specialization packages the abstract enriched pointwise formula as
an `IsStochKanExtension`, i.e. it inherits the set-integral factorization
predicate from Part 4. -/
theorem abstractLanValueFromEnriched_isStochKanExtension
    {X Y : MeasObj}
    (hHom : ∀ F₁ F₂ : A⁇ ↛ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X) (κ : Kernel Y X) (hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ) (hf_meas : AEStronglyMeasurable f μ) (hf_int : Integrable f μ)
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (e : lan.obj Y ≅ Y)
    (ω : (⁇_ MeasObj) → WeightedCoconeObj
      (V := MeasObj) (W := leftKanWeight MeasObj F Y) G X)
    (hω : e.inv ⁇ PointwiseEnrichedLeftKanExtension.kernelFromWeightedPoint
      (F := F) (G := G) hHom lan Y X ω = κ) :
    IsStochKanExtension D μ
      (abstractLanValueFromEnriched (F := F) (G := G) hHom lan e ω f) f := by
  rw [abstractLanValueFromEnriched_eq_lanValue (F := F) (G := G) hHom
    (D := D) (hD := hD) (κ := κ) (f := f) (lan := lan) (e := e) (ω := ω) hω]
  exact lanValue_isStochKanExtension D hD μ κ hκ f hf_meas hf_int

/-- The canonical enriched value attached directly to a kernel `κ`, obtained by
feeding `κ` through `weightedPointOfKernel` before evaluating the abstract
formula. -/
noncomputable def abstractLanValueOfKernelFromEnriched
    {X Y : MeasObj}
    (hHom : ∀ F₁ F₂ : A⁇ ↛ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (e : lan.obj Y ≅ Y) (κ : Y → X) (f : X → ℝ) : Y → ℝ :=
  abstractLanValueFromEnriched (F := F) (G := G) hHom lan e
    (PointwiseEnrichedLeftKanExtension.weightedPointOfKernel (F := F) (G := G)
      hHom lan e κ) f

/-- Using the canonical weighted-cocone point of `κ`, the abstract enriched
pointwise value reduces directly to the concrete `lanValue`, with no separate
identification hypothesis. -/
theorem abstractLanValueOfKernelFromEnriched_eq_lanValue
    {X Y : MeasObj}
```

```
    (hHom : ∀ F₁ F₂ : A⦃⦄ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (D : X → Y) (hD : Measurable D) (κ : Kernel Y X) (f : X → ℝ)
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (e : lan.obj Y ≅ Y) :
    abstractLanValueOfKernelFromEnriched (F := F) (G := G) hHom lan e κ f =
      lanValue D hD κ f := by
  refine abstractLanValueFromEnriched_eq_lanValue (F := F) (G := G) hHom
    (D := D) (hD := hD) (κ := κ) (f := f) (lan := lan) (e := e)
    (ω := PointwiseEnrichedLeftKanExtension.weightedPointOfKernel (F := F) (G := G)
      hHom lan e κ) ?_
  exact PointwiseEnrichedLeftKanExtension.inv_kernelFromWeightedPoint_weightedPointOfKernel
    (F := F) (G := G) hHom lan e κ

/-- The canonical abstract enriched value attached to `κ` satisfies the same
global factorization formula as the concrete bridge theorem. -/
theorem abstractLanValueOfKernelFromEnriched_integral_factorization
    {X Y : MeasObj}
    (hHom : ∀ F₁ F₂ : A⦃⦄ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X) (κ : Kernel Y X) (hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ) (hf_meas : AEStronglyMeasurable f μ) (hf_int : Integrable f μ)
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
    (e : lan.obj Y ≅ Y) :
    (∫ x, f x ∂μ) =
      ∫ y, abstractLanValueOfKernelFromEnriched (F := F) (G := G) hHom lan e κ f y
        ∂(Measure.map D μ) := by
  rw [abstractLanValueOfKernelFromEnriched_eq_lanValue (F := F) (G := G) hHom
    (D := D) (hD := hD) (κ := κ) (f := f) (lan := lan) (e := e)]
  exact lan_eq_integral_of_condKernel D hD μ κ hκ f hf_meas hf_int

/-- The canonical abstract enriched value attached to `κ` also inherits the
stochastic universal property from Part 4. -/
theorem abstractLanValueOfKernelFromEnriched_isStochKanExtension
    {X Y : MeasObj}
    (hHom : ∀ F₁ F₂ : A⦃⦄ ↣ MeasObj,
      CategoryTheory.Enriched.FunctorCategory.HasEnrichedHom MeasObj F₁ F₂)
    [∀ (b X' : MeasObj), HasPointwiseEnrichedLeftKanExtensionObj (V := MeasObj) F G b X']
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X) (κ : Kernel Y X) (hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ) (hf_meas : AEStronglyMeasurable f μ) (hf_int : Integrable f μ)
    (lan : PointwiseEnrichedLeftKanExtension (V := MeasObj) F G)
```

```
    (e : lan.obj Y ≅ Y) :
    IsStochKanExtension D μ
      (abstractLanValueOfKernelFromEnriched (F := F) (G := G) hHom lan e κ f) f := by
  rw [abstractLanValueOfKernelFromEnriched_eq_lanValue (F := F) (G := G) hHom
    (D := D) (hD := hD) (κ := κ) (f := f) (lan := lan) (e := e)]
  exact lanValue_isStochKanExtension D hD μ κ hκ f hf_meas hf_int

end Abstract

end CpmProofs
```

```
import CpmProofs.KanBridge
import CpmProofs.EnrichedKanExtension
import CpmProofs.EnrichedBridge
import CpmProofs.StochSelfEnrichment
import Mathlib.CategoryTheory.Enriched.Basic
import Mathlib.CategoryTheory.Enriched.Ordinary.Basic
import Mathlib.CategoryTheory.Category.Init
```

Source: `Enriched.lean`

Part 10: Enriched Kan Extensions — Status and Roadmap

Motivation

The bridge theorem (Part 4) identifies the stochastic Kan extension as a concrete predicate `IsStochKanExtension` satisfying set-integral factorization. Fritz [arXiv:1908.07021, §11] shows that this is an enriched Kan extension in the Stoch-enriched sense, where the weight is given by the conditional kernel and the colimit is computed by integration.

Mathlib's `IsPointwiseLeftKanExtension` computes ordinary (Set-enriched) Kan extensions via colimits in comma categories. This does not apply to Stoch because:

1. The hom-objects in Stoch are spaces of kernels (not sets), and the colimit formula involves integration (not coproducts/coequalisers).
2. Ordinary colimits in Stoch are measure-theoretic sums, not the integration operation needed for conditional expectations.

Mathlib Status (as of March 2026)

Available: - `EnrichedCategory V C` — categories enriched over a monoidal category `V` - `EnrichedFunctor` — functors between enriched categories - `EnrichedOrdinaryCategory` — enriched categories with an underlying ordinary category - `MonoidalClosed V` — closed monoidal categories (internal hom) - Ends and coends (`end_`, `coend`) are partially formalised

Available locally in this project (not upstream in Mathlib): - `CpmProofs.WeightedColimit` — a local weighted-colimit interface formalizing the representing-object definition used in enriched category theory - `CpmProofs.EnrichedKanExtension` — a local pointwise enriched left Kan-extension interface built from weighted colimits - `CpmProofs.EnrichedBridge` — a local specialization theorem showing that every kernel canonically determines the required weighted-cocone point, so the abstract weighted-colimit formula collapses to `lanValue` without any extra point-identification hypothesis - `CpmProofs.StochSelfEnrichment` — candidate kernel-hom objects and abstract self-enrichment scaffolding, isolating the remaining stochastic gap

Absent (blocking upstream / for the stochastic application): - General enriched Kan extensions in Mathlib — there is still no upstream API packaging these constructions as theorems and reusable infrastructure. - V-object of natural transformations — marked TODO in Mathlib's enriched category files. - Monoidal structure on stochastic morphisms — the current concrete category uses all kernels as morphisms, while the tensor and Markov-category laws are naturally phrased for Markov/s-finite kernels. - Kernel-space evaluation/curry-uncurry adjunction — the project now has a candidate kernel-hom object, but not yet a genuine ordinary right adjoint: besides joint measurability, ordinary morphisms into that object are kernel-valued kernels, while the current curry construction only produces deterministic kernel-valued maps.

What We Can State

With local weighted colimits and pointwise enriched Kan extensions in place, but without stochastic self-enrichment, we can state the relationship between our concrete `IsStochKanExtension` and the would-be enriched version:

1. The `IsStochKanExtension` predicate (Part 4) is the measure-theoretic incarnation of the enriched colimit cocone.
2. The `ae_unique` theorem is the uniqueness of the mediating morphism (up to 2-cells = a.e. equalities).
3. The `lan_eq_integral_of_condKernel` factorization is the enriched cocone condition.
4. `EnrichedBridge.PointwiseEnrichedLeftKanExtension.weightedPointOfKernel` canonically converts a kernel κ into a point of the abstract weighted cocone object.
5. `EnrichedBridge.abstractLanValueOfKernelFromEnriched_eq_lanValue` shows that the abstract enriched pointwise formula for that canonical point is literally the concrete `lanValue`.
6. `EnrichedBridge.abstractLanValueOfKernelFromEnriched_isStochKanExtension` then lifts the same specialization to the universal property from Part 4.

Results

#	Result	Status
34	Enriched category instance for Meas (ordinary)	✓
35	Weighted colimit interface	✓ (local)
36	Pointwise enriched left Kan extension interface	✓ (local)
37	Candidate stochastic kernel-hom object	✓ (local, partial)
38	Statement of remaining stochastic gap	✓ (documented)
39	Canonical weighted-cocone point of a kernel	✓ (local)
40	Abstract-to-concrete specialization theorem	✓ (local, canonical in κ)
41	Specialized stochastic universal property	✓ (local, canonical in κ)

```
open CategoryTheory MeasureTheory ProbabilityTheory

universe u v

namespace CpmProofs
```

The Ordinary Category of Measurable Spaces

Mathlib's `EnrichedOrdinaryCategory` framework requires a `V`-enriched category to also have an underlying ordinary category. For Stoch (Part 1), the ordinary category is already defined: `Category MeasObj` with kernels as morphisms.

The enrichment of Stoch over itself (self-enrichment) would require `MonoidalClosed` on a category of measurable spaces with kernels. This is a substantial construction that requires:

1. An internal hom `[X, Y]` — the measurable space of kernels from `X` to `Y`, equipped with a σ-algebra.
2. A tensor-hom adjunction: `Hom(X ⊗ Y, Z) ≅ Hom(X, [Y, Z])`.

For now, we record the available Mathlib infrastructure and the gap.

```
/-- **Documentation theorem**: The enriched Kan extension formula.

In Fritz's framework, the stochastic left Kan extension of `G : A → X`
along `F : A → B` at `b : B` is:

```

```
(Lan_F G)(b) = ∫^{a:A} Stoch(Fa, b) ⊠ G(a)
```

where `⊠` is the "action" of **Stoch**-hom-objects on morphisms, and
the coend `∫^a` integrates over the fiber category.

In measure-theoretic terms, this reduces to:

```
(Lan_D f)(y) = ∫ f(x) dκ_y(x)
```

where `κ_y` is the conditional kernel — exactly our `lanValue` from Part 4.

The `IsStochKanExtension` predicate (Part 4) captures the universal
property: set-integral factorization with a.e. uniqueness. This is
the concrete incarnation of the enriched universal property.

Current gap: The project now contains local weighted-colimit and
pointwise enriched left Kan-extension definitions
(`CpmProofs.WeightedColimit` and
`CpmProofs.EnrichedKanExtension`), a bridge specialization theorem
that canonically constructs the relevant weighted-cocone point from a
kernel (`CpmProofs.EnrichedBridge`), and local self-enrichment
scaffolding (`CpmProofs.StochSelfEnrichment`). What is still
missing is a genuine stochastic `MonoidalClosed` structure: a monoidal
tensor on stochastic morphisms, joint measurability/evaluation data for
kernel-hom objects, and an ordinary right adjoint whose morphisms into
`[X, Z]` can represent more than deterministic kernel sections. In the
current ordinary category of all kernels, this last point is a genuine
design issue, not just unfinished proof plumbing. Once the correct
ambient stochastic category / internal-hom construction is supplied
(together with the concrete identification of the resulting pointwise
object with `Y`), the bridge theorem becomes a direct instance of the
general enriched Kan-extension formula. -/
theorem enriched_kan_is_condKernel_integration
    {X Y : MeasObj}
    (D : X → Y) (hD : Measurable D)
    (μ : Measure X)
    (κ : Kernel Y X)
    (_hκ : IsCondKernelMap D μ κ)
    (f : X → ℝ)
    (_hf_meas : AEStronglyMeasurable f μ)
    (_hf_int : Integrable f μ)
    (y : Y) :

```
  lanValue D hD κ f y = ∫ x, f x ∂(κ y) :=
rfl
```

Roadmap for the Full Enriched Kan Extension

The path to a fully enriched formalization now requires a combination of local-to-upstream cleanup and stochastic enrichment work:

1. Upstream weighted colimits and pointwise enriched Kan extensions
   - Generalize the local `CpmProofs.WeightedColimit` and `CpmProofs.EnrichedKanExtension` interfaces
   - Connect them systematically to ends/coends and existence results
   - Add the corresponding Mathlib API for reuse outside this project
2. General enriched Kan extensions (`EnrichedLan F G`)
   - Package the pointwise construction functorially
   - Relate the weighted-colimit presentation to the coend formula
3. Stoch self-enrichment (`MonoidalClosed StochMarkov`)
   - Reuse the restricted monoidal category `CpmProofs.StochMarkov` now formalized in `CpmProofs.StochMarkov`
   - Promote the local candidate kernel-hom object `CpmProofs.kernelHomObj`
   - Prove the jointly measurable evaluation/curry-uncurry adjunction
   - Show `MarkovCategory StochMarkov` (Part 8)
4. Bridge as enriched identity
   - Instantiate the local canonical specialization in the stochastic setting
   - Show `IsStochKanExtension = IsEnrichedLeftKanExtension`
   - Derive `ae_unique` from the enriched universal property

Steps 1–2 are general enriched category theory (applicable beyond probability). Steps 3–4 are specific to the stochastic setting. The current formalization (Parts 1–5) provides the measure-theoretic foundation that step 4 would connect to.

```
end CpmProofs
```